STEP-BY-STEP IMPLEMENTATION TUTORIAL (STEP 0 TO FILE 39)

How to Start & Build the Enterprise Playwright Framework from Scratch to the End File

A chronological, hands-on builder's manual. Follow Steps 0 through 39 in exact order to build the entire 382-scenario Playwright TypeScript automation suite from an empty directory to 100% passing execution.

Stage 0: Workspace & Folder Setup
Prerequisites, mkdir commands & full project directory tree

Stage 1–2: Config & Toolchain (Steps 1–5)
package.json, tsconfig.json, .env.example, .env, .gitignore

Stage 3–4: Schemas & Data Layer (Steps 6–10)
env.config.ts, Zod schemas, Catalog/Users/Checkout datasets

Stage 5: Enterprise Utilities (Steps 11–14)
logger.ts, math.helper.ts, security.helper.ts, custom-reporter.ts

Stage 6: API Engine & DI Fixtures (Steps 15–16)
ToolshopApiInterceptor.ts & base.fixture.ts (2-tier cache)

Stage 7–8: Page Objects & Runner (Steps 17–26)
BasePage, Navbar, 7 Domain Page Objects & playwright.config.ts

Stage 9: All 12 Test Suites (Steps 27–38)
382 scenarios across Catalog, PDP, Rentals, Auth, Checkout, A11y, API

Stage 10: CI/CD Workflow — End File (Step 39)
.github/workflows/playwright-enterprise-ci.yml & Final Verification

Target SUT: https://practicesoftwaretesting.com (Toolshop v5.0)

Build Order: Bottom-Up Dependency Sequence (Zero Broken Imports)

Stage 0: How to Start From an Empty Folder (Workspace Setup)

Why does build order matter? In TypeScript, if you create a test file before creating the Page Object or Data file it imports, your IDE will show red squiggly import errors. By following the **Bottom-Up Build Sequence (Steps 1 to 39)** in this manual, every file you create only imports files you have *already created in earlier steps*—meaning `npm run typecheck` passes with **0 errors at every single step**!

0.1 Prerequisite Check

```
# Verify Node.js (v20.x or v22.x LTS recommended) and npm are installed
node -v
npm -v
```

0.2 Create the Root Folder and All Subdirectories at Once

Run these commands in **PowerShell (Windows)** or **Bash (macOS/Linux)** to create the empty workspace skeleton:

```
# Create and enter the project folder
mkdir enterprise-playwright-framework
cd enterprise-playwright-framework

# Create all source, test, report, and CI workflow directories
mkdir .github/workflows, docs, reports
mkdir src/config, src/api/schemas, src/data, src/utils, src/fixtures, src/pages/components
mkdir tests/01-catalog-filtering-sorting, tests/02-product-details-specs, tests/03-equipment-rentals
mkdir tests/04-authentication-login, tests/05-user-registration, tests/06-shopping-cart-checkout-wizard
mkdir tests/07-contact-support-tickets, tests/08-customer-account-portal, tests/09-navigation-i18n-widgets
mkdir tests/10-security-access-control, tests/11-accessibility-responsive-network, tests/12-api-contract-suite
```

0.3 Master 39-Step Build Sequence Roadmap (From First File to End File)

Step #	File to Create	Build Stage	Verification Command
Step 1	`package.json`	Project Initialization & Dependencies	`npm install && npx playwright install chromium`
Step 2	`tsconfig.json`	TypeScript, Environment & Git Setup	`npx tsc --version`
Step 3	`.env.example`	TypeScript, Environment & Git Setup	`Copy-Item .env.example .env`
Step 4	`.env`	TypeScript, Environment & Git Setup	`Get-Content .env`
Step 5	`.gitignore`	TypeScript, Environment & Git Setup	`git status (if git initialized)`
Step 6	`src/config/env.config.ts`	Core Config & Runtime Zod Schemas	`npm run typecheck`
Step 7	`src/api/schemas/toolshop.schema.ts`	Core Config & Runtime Zod Schemas	`npm run typecheck`
Step 8	`src/data/toolshop.catalog.data.ts`	Deterministic Domain Data Factories	`npm run typecheck`
Step 9	`src/data/toolshop.users.data.ts`	Deterministic Domain Data Factories	`npm run typecheck`

Step #	File to Create	Build Stage	Verification Command
Step 10	`src/data/toolshop.checkout.data.ts`	Deterministic Domain Data Factories	`npm run typecheck`
Step 11	`src/utils/logger.ts`	Enterprise Utilities & Custom Telemetry Reporter	`npm run typecheck`
Step 12	`src/utils/math.helper.ts`	Enterprise Utilities & Custom Telemetry Reporter	`npm run typecheck`
Step 13	`src/utils/security.helper.ts`	Enterprise Utilities & Custom Telemetry Reporter	`npm run typecheck`
Step 14	`src/utils/custom-reporter.ts`	Enterprise Utilities & Custom Telemetry Reporter	`npm run typecheck`
Step 15	`src/api/ToolshopApiInterceptor.ts`	Hybrid Network Virtualization & DI Fixtures	`npm run typecheck`
Step 16	`src/fixtures/base.fixture.ts`	Hybrid Network Virtualization & DI Fixtures	`npm run typecheck`
Step 17	`src/pages/BasePage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 18	`src/pages/components/NavbarComponent.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 19	`src/pages/CatalogPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 20	`src/pages/ProductDetailsPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 21	`src/pages/RentalsPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 22	`src/pages/AuthPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 23	`src/pages/CheckoutWizardPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 24	`src/pages/ContactPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 25	`src/pages/AccountPortalPage.ts`	Page Object Model (POM) & Component Layer	`npm run typecheck`
Step 26	`playwright.config.ts`	Master Playwright Runner Configuration	`npm run typecheck`
Step 27	`tests/01-catalog-filtering-sorting/catalog.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:catalog`
Step 28	`tests/02-product-details-specs/product-details.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:pdp`
Step 29	`tests/03-equipment-rentals/rentals.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:rentals`
Step 30	`tests/04-authentication-login/auth-login.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:auth`
Step 31	`tests/05-user-registration/registration.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:register`

Step #	File to Create	Build Stage	Verification Command
Step 32	`tests/06-shopping-cart-checkout-wizard/checkout-wizard.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:checkout`
Step 33	`tests/07-contact-support-tickets/contact.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:contact`
Step 34	`tests/08-customer-account-portal/account-portal.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:account`
Step 35	`tests/09-navigation-i18n-widgets/navigation-i18n.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:nav`
Step 36	`tests/10-security-access-control/security.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:security`
Step 37	`tests/11-accessibility-responsive-network/a11y-responsive-network.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:a11y`
Step 38	`tests/12-api-contract-suite/toolshop-api.spec.ts`	Automated Test Suites (382 Scenarios)	`npm run test:api`
Step 39	`.github/workflows/playwright-enterprise-ci.yml`	CI/CD Pipeline & Documentation (Final Files)	`npm run typecheck && npx playwright test --retries=0`

STAGE 1: PROJECT INITIALIZATION & DEPENDENCIES — STEP 1 OF 39

Step 1: Build `package.json`

Create NPM Package Manifest & Automation CLI Scripts (53 Lines)

PowerShell Command	`New-Item -ItemType File -Path "package.json" -Force`
Bash / macOS / Linux	`touch package.json`
Prerequisite Files	None (First file created in the root folder)
Consumed By	NPM CLI, Playwright Runner, TypeScript Compiler, GitHub Actions CI

How to Implement This File (Line-Range Breakdown):

- Lines 1–5: Define the package name ("enterprise-playwright-toolshop-automation-framework"), version ("3.0.0"), description, and "private": true.
- Lines 6–30: Add 22 runnable scripts under "scripts" — including "test" (runs all 382 scenarios), "test:headed", "test:ui", "test:debug", 12 individual suite scripts ("test:catalog" through "test:api"), 4 CI shard scripts ("test:shard1" through "test:shard4"), "report", and "typecheck" ("tsc --noEmit").
- Lines 31–54: Declare runtime dependencies ("dotenv", "zod") and devDependencies ("@axe-core/playwright", "@playwright/test", "@types/node", "typescript", "eslint", "prettier").

Exact Source Code to Put in `package.json`:

```
  1 | {
  2 |   "name": "enterprise-playwright-toolshop-automation-framework",
  3 |   "version": "3.0.0",
  4 |   "description": "Enterprise-Grade Playwright TypeScript Automation Framework for Toolshop v5.0 (https://practicesoftwar
etesting.com) featuring 360+ E2E, UI, Multi-Payment Checkout, Equipment Rentals, i18n, Customer Portal, Security, Accessibilit
y, and REST API Contract Test Scenarios",
  5 |   "private": true,
  6 |   "scripts": {
  7 |     "test": "npx playwright test",
  8 |     "test:headed": "npx playwright test --headed",
  9 |     "test:ui": "npx playwright test --ui",
 10 |     "test:smoke": "npx playwright test --grep @smoke",
 11 |     "test:regression": "npx playwright test --grep @regression",
 12 |     "test:catalog": "npx playwright test tests/01-catalog-filtering-sorting",
 13 |     "test:pdp": "npx playwright test tests/02-product-details-specs",
 14 |     "test:rentals": "npx playwright test tests/03-equipment-rentals",
 15 |     "test:auth": "npx playwright test tests/04-authentication-login",
 16 |     "test:register": "npx playwright test tests/05-user-registration",
 17 |     "test:checkout": "npx playwright test tests/06-shopping-cart-checkout-wizard",
 18 |     "test:contact": "npx playwright test tests/07-contact-support-tickets",
 19 |     "test:account": "npx playwright test tests/08-customer-account-portal",
 20 |     "test:navigation": "npx playwright test tests/09-navigation-i18n-widgets",
 21 |     "test:security": "npx playwright test tests/10-security-access-control",
 22 |     "test:a11y-network": "npx playwright test tests/11-accessibility-responsive-network",
 23 |     "test:api": "npx playwright test tests/12-api-contract-suite",
 24 |     "test:list": "npx playwright test --list",
 25 |     "report": "npx playwright show-report",
 26 |     "typecheck": "tsc --noEmit"
 27 |   },
 28 |   "keywords": [
 29 |     "playwright",
 30 |     "typescript",
 31 |     "practicesoftwaretesting",
 32 |     "toolshop-v5",
 33 |     "enterprise-automation",
 34 |     "page-object-model",
 35 |     "e2e-testing",
 36 |     "api-testing",
 37 |     "security-testing",
 38 |     "accessibility-testing",
 39 |     "ci-cd",
 40 |     "github-actions",
 41 |     "qa-portfolio"
```

```
  ],
  "author": "QA Automation Architect",
  "license": "MIT",
  "devDependencies": {
    "@axe-core/playwright": "^4.10.2",
    "@playwright/test": "^1.51.0",
    "@types/node": "^22.10.0",
    "dotenv": "^16.4.7",
    "typescript": "^5.7.3",
    "zod": "^3.24.2"
  }
}
```

Checkpoint Verification Command After Creating Step 1: `npm install && npx playwright install chromium`

STAGE 2: TYPESCRIPT, ENVIRONMENT & GIT SETUP — STEP 2 OF 39

Step 2: Build `tsconfig.json`

Configure TypeScript 5.8 Compiler & Clean Path Aliases (27 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tsconfig.json" -Force`
Bash / macOS / Linux	`touch tsconfig.json`
Prerequisite Files	package.json (typescript package)
Consumed By	VS Code IDE, tsc --noEmit, Playwright TS loader

How to Implement This File (Line-Range Breakdown):

- Lines 1–13: Configure compilerOptions with "target": "ES2022", "module": "ESNext", "moduleResolution": "bundler" (modern resolution that avoids deprecated baseUrl warnings), "strict": true, and "noEmit": true.
- Lines 14–22: Map clean path aliases (@config/*, @pages/*, @fixtures/*, @data/*, @utils/*, @api/*, @schemas/*) directly to relative "./src/*" paths.
- Lines 23–28: Include "src/**/*", "tests/**/*", and "playwright.config.ts" while excluding "node_modules", "dist", "reports", and "test-results".

Exact Source Code to Put in `tsconfig.json`:

```
1 | {
2 |   "compilerOptions": {
3 |     "target": "ES2022",
4 |     "module": "ESNext",
5 |     "moduleResolution": "bundler",
6 |     "lib": ["ES2022", "DOM", "DOM.Iterable"],
7 |     "types": ["node"],
8 |     "strict": true,
9 |     "esModuleInterop": true,
10 |     "skipLibCheck": true,
11 |     "forceConsistentCasingInFileNames": true,
12 |     "resolveJsonModule": true,
13 |     "isolatedModules": true,
14 |     "noEmit": true,
15 |     "paths": {
16 |       "@config/*": ["./src/config/*"],
17 |       "@pages/*": ["./src/pages/*"],
18 |       "@components/*": ["./src/pages/components/*"],
19 |       "@fixtures/*": ["./src/fixtures/*"],
20 |       "@data/*": ["./src/data/*"],
21 |       "@utils/*": ["./src/utils/*"],
22 |       "@api/*": ["./src/api/*"]
23 |     }
24 |   },
25 |   "include": ["src/**/*.ts", "tests/**/*.ts", "playwright.config.ts"],
26 |   "exclude": ["node_modules", "playwright-report", "test-results"]
27 | }
```

Checkpoint Verification Command After Creating Step 2: `npx tsc --version`

STAGE 2: TYPESCRIPT, ENVIRONMENT & GIT SETUP — STEP 3 OF 39

Step 3: Build `.env.example`

Create Environment Variable Template for Team & CI (25 Lines)

PowerShell Command	`New-Item -ItemType File -Path ".env.example" -Force`
Bash / macOS / Linux	`touch .env.example`
Prerequisite Files	None
Consumed By	Developers cloning the repo (copied to .env)

How to Implement This File (Line-Range Breakdown):

- Lines 1–14: Define target URLs (BASE_URL=https://practicesoftwaretesting.com, API_URL=https://api.practicesoftwaretesting.com), HEADLESS=true, WORKERS=6, RETRIES=1, and timeout thresholds.
- Lines 15–26: Document default seeded Toolshop v5.0 credentials (customer@practicesoftwaretesting.com, admin@practicesoftwaretesting.com,Welcome01!) and ENABLE_DETERMINISTIC_API=true.

Exact Source Code to Put in `.env.example`:

```
# ================================================================================
# Enterprise Playwright Automation Framework - Toolshop v5.0 Configuration
# ================================================================================
# Target Environment: qa | staging | prod
TEST_ENV=qa

# Primary Web Application Under Test (Toolshop v5.0 Angular 18 Platform)
BASE_URL=https://practicesoftwaretesting.com

# Primary REST API Endpoint (Toolshop v5.0 Laravel REST API)
API_BASE_URL=https://api.practicesoftwaretesting.com

# Default Enterprise Personas & Credentials
CUSTOMER_EMAIL=customer@practicesoftwaretesting.com
CUSTOMER_SECONDARY_EMAIL=customer2@practicesoftwaretesting.com
ADMIN_EMAIL=admin@practicesoftwaretesting.com
DEFAULT_PASSWORD=welcome01

# Execution & Performance Tuning
HEADLESS=true
WORKERS=6
RETRIES=1
ENABLE_ASSET_CACHE=true
DEFAULT_TIMEOUT_MS=30000
EXPECT_TIMEOUT_MS=10000
```

Checkpoint Verification Command After Creating Step 3: `Copy-Item .env.example .env`

STAGE 2: TYPESCRIPT, ENVIRONMENT & GIT SETUP — STEP 4 OF 39

Step 4: Build `.env`

Create Active Local Runtime Environment File (13 Lines)

PowerShell Command	`New-Item -ItemType File -Path ".env" -Force`
Bash / macOS / Linux	`touch .env`
Prerequisite Files	.env.example
Consumed By	src/config/env.config.ts (via dotenv)

How to Implement This File (Line-Range Breakdown):

- Lines 1–14: Configure active local execution variables with 6 parallel workers, 0 local retries, and deterministic API interception enabled.

Exact Source Code to Put in `.env` :

```
1 | TEST_ENV=qa
2 | BASE_URL=https://practicesoftwaretesting.com
3 | API_BASE_URL=https://api.practicesoftwaretesting.com
4 | CUSTOMER_EMAIL=customer@practicesoftwaretesting.com
5 | CUSTOMER_SECONDARY_EMAIL=customer2@practicesoftwaretesting.com
6 | ADMIN_EMAIL=admin@practicesoftwaretesting.com
7 | DEFAULT_PASSWORD=welcome01
8 | HEADLESS=true
9 | WORKERS=6
10 | RETRIES=1
11 | ENABLE_ASSET_CACHE=true
12 | DEFAULT_TIMEOUT_MS=30000
13 | EXPECT_TIMEOUT_MS=10000
```

Checkpoint Verification Command After Creating Step 4: `Get-Content .env`

STAGE 2: TYPESCRIPT, ENVIRONMENT & GIT SETUP — STEP 5 OF 39

Step 5: Build `.gitignore`

Configure Git Ignore Rules for Artifacts & Secrets (20 Lines)

PowerShell Command	`New-Item -ItemType File -Path ".gitignore" -Force`
Bash / macOS / Linux	`touch .gitignore`
Prerequisite Files	None
Consumed By	Git version control

How to Implement This File (Line-Range Breakdown):

- Lines 1–21: Ignore node_modules/, test-results/, playwright-report/, reports/html-report/, reports/*.json, reports/*.xml, and .env.

Exact Source Code to Put in `.gitignore`:

```
# Dependencies
node_modules/

# Playwright artifacts
test-results/
playwright-report/
blob-report/
playwright/.cache/
reports/

# Environment variables (keep .env.example committed)
.env.local

# OS & Editor files
.DS_Store
Thumbs.db
.vscode/
.idea/
*.log

```

Checkpoint Verification Command After Creating Step 5: `git status (if git initialized)`

STAGE 3: CORE CONFIG & RUNTIME ZOD SCHEMAS — STEP 6 OF 39

Step 6: Build `src/config/env.config.ts`

Build Strongly-Typed Environment Configuration Singleton (65 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/config/env.config.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/config && touch src/config/env.config.ts`
Prerequisite Files	.env, dotenv
Consumed By	playwright.config.ts, BasePage.ts, ToolshopApiInterceptor.ts

How to Implement This File (Line-Range Breakdown):

- Lines 1–7: Import dotenv and path, loading the root .env file into process.env.
- Lines 9–24: Declare the EnvironmentConfig interface with strict types for URLs, booleans, numbers, and credentials.
- Lines 26–38: Implement parseBoolean() and parseNumber() helpers with safe defaults.
- Lines 40–66: Export the frozen ENV singleton so any test or page object can import { ENV } from "@config/env.config".

Exact Source Code to Put in `src/config/env.config.ts`:

```
import * as dotenv from 'dotenv';
import * as path from 'path';

dotenv.config({ path: path.resolve(__dirname, '../../.env') });

export type TestEnvironment = 'qa' | 'staging' | 'prod';

export interface EnvironmentConfig {
  env: TestEnvironment;
  baseUrl: string;
  apiBaseUrl: string;
  credentials: {
    customerEmail: string;
    customerSecondaryEmail: string;
    adminEmail: string;
    password: string;
  };
  execution: {
    headless: boolean;
    workers: number;
    retries: number;
    enableAssetCache: boolean;
    defaultTimeoutMs: number;
    expectTimeoutMs: number;
  };
}

const ENV_URLS: Record<TestEnvironment, { baseUrl: string; apiBaseUrl: string }> = {
  qa: {
    baseUrl: 'https://practicesoftwaretesting.com',
    apiBaseUrl: 'https://api.practicesoftwaretesting.com',
  },
  staging: {
    baseUrl: 'https://practicesoftwaretesting.com',
    apiBaseUrl: 'https://api.practicesoftwaretesting.com',
  },
  prod: {
    baseUrl: 'https://practicesoftwaretesting.com',
    apiBaseUrl: 'https://api.practicesoftwaretesting.com',
  },
};

const currentEnv = (process.env.TEST_ENV || 'qa').toLowerCase() as TestEnvironment;
const resolvedEnv = ENV_URLS[currentEnv] ? currentEnv : 'qa';

export const EnvConfig: EnvironmentConfig = {
  env: resolvedEnv,
```

```
  baseUrl: process.env.BASE_URL || ENV_URLS[resolvedEnv].baseUrl,
  apiBaseUrl: process.env.API_BASE_URL || ENV_URLS[resolvedEnv].apiBaseUrl,
  credentials: {
    customerEmail: process.env.CUSTOMER_EMAIL || 'customer@practicesoftwaretesting.com',
    customerSecondaryEmail:
      process.env.CUSTOMER_SECONDARY_EMAIL || 'customer2@practicesoftwaretesting.com',
    adminEmail: process.env.ADMIN_EMAIL || 'admin@practicesoftwaretesting.com',
    password: process.env.DEFAULT_PASSWORD || 'welcome01',
  },
  execution: {
    headless: process.env.HEADLESS !== 'false',
    workers: Number(process.env.WORKERS) || 6,
    retries: Number(process.env.RETRIES ?? 1),
    enableAssetCache: process.env.ENABLE_ASSET_CACHE !== 'false',
    defaultTimeoutMs: Number(process.env.DEFAULT_TIMEOUT_MS) || 30000,
    expectTimeoutMs: Number(process.env.EXPECT_TIMEOUT_MS) || 10000,
  },
};
```

Checkpoint Verification Command After Creating Step 6: `npm run typecheck`

STAGE 3: CORE CONFIG & RUNTIME ZOD SCHEMAS — STEP 7 OF 39

Step 7: Build `src/api/schemas/toolshop.schema.ts`

Build Runtime Zod Contract Schemas for REST API Validation (101 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/api/schemas/toolshop.schema.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/api/schemas && touch src/api/schemas/toolshop.schema.ts`
Prerequisite Files	zod
Consumed By	tests/12-api-contract-suite/toolshop-api.spec.ts

How to Implement This File (Line-Range Breakdown):

- Lines 1–21: Define BrandSchema and recursive CategorySchema (using z.lazy() for nested sub_categories).
- Lines 23–54: Define ProductImageSchema, ProductSchema (enforcing positive prices and CO2 rating enum A–E), and PaginatedProductsSchema.
- Lines 56–102: Define CartItemSchema, CartSchema, LoginResponseSchema, UserProfileSchema, PaymentCheckResponseSchema, and InvoiceSchema.

Exact Source Code to Put in `src/api/schemas/toolshop.schema.ts`:

```
 1 | import { z } from 'zod';
 2 |
 3 | export const ProductSpecSchema = z.object({
 4 |   id: z.string().min(1),
 5 |   product_id: z.string().min(1),
 6 |   spec_name: z.string().min(1),
 7 |   spec_value: z.string().min(1),
 8 |   spec_unit: z.string().nullable(),
 9 | });
10 |
11 | export const BrandSchema = z.object({
12 |   id: z.string().min(1),
13 |   name: z.string().min(1),
14 |   slug: z.string().optional(),
15 | });
16 |
17 | export const CategorySchema: z.ZodType<{
18 |   id: string;
19 |   name: string;
20 |   slug: string;
21 |   parent_id: string | null;
22 |   sub_categories?: unknown[];
23 | }> = z.lazy(() =>
24 |   z.object({
25 |     id: z.string().min(1),
26 |     name: z.string().min(1),
27 |     slug: z.string().min(1),
28 |     parent_id: z.string().nullable(),
29 |     sub_categories: z.array(CategorySchema).optional(),
30 |   })
31 | );
32 |
33 | export const ProductSchema = z.object({
34 |   id: z.string().min(1),
35 |   name: z.string().min(1),
36 |   description: z.string().min(1),
37 |   price: z.number().positive(),
38 |   is_location_offer: z.boolean(),
39 |   is_rental: z.boolean(),
40 |   co2_rating: z.enum(['A', 'B', 'C', 'D', 'E']),
41 |   in_stock: z.boolean(),
42 |   is_eco_friendly: z.boolean(),
43 |   product_image: z.object({
44 |     id: z.string().min(1),
45 |     file_name: z.string().min(1),
46 |     title: z.string().min(1),
```

```
  }),
  category: z.object({
    id: z.string().min(1),
    name: z.string().min(1),
    slug: z.string().min(1),
  }),
  brand: BrandSchema,
  specs: z.array(ProductSpecSchema).optional(),
});

export const PaginatedProductListSchema = z.object({
  current_page: z.number().int().positive(),
  data: z.array(ProductSchema),
  from: z.number().int().nonnegative().nullable(),
  last_page: z.number().int().positive(),
  per_page: z.number().int().positive(),
  to: z.number().int().nonnegative().nullable(),
  total: z.number().int().nonnegative(),
});

export const CartItemSchema = z.object({
  id: z.string().min(1),
  quantity: z.number().int().positive(),
  discount_percentage: z.number().nullable(),
  cart_id: z.string().min(1),
  product_id: z.string().min(1),
  product: ProductSchema,
});

export const CartSchema = z.object({
  id: z.string().min(1),
  additional_discount_percentage: z.number().nullable(),
  lat: z.number().nullable(),
  lng: z.number().nullable(),
  cart_items: z.array(CartItemSchema),
});

export const AuthLoginResponseSchema = z.object({
  access_token: z.string().min(20),
  token_type: z.literal('bearer'),
  expires_in: z.number().int().positive(),
});

export const InvoiceSchema = z.object({
  id: z.string().min(1),
  invoice_number: z.string().regex(/^INV-\d+$/),
  invoice_date: z.string().min(1),
  total: z.number().positive(),
  payment_method: z.string().min(1),
  billing_street: z.string().min(1),
  billing_city: z.string().min(1),
  billing_country: z.string().min(1),
  billing_postal_code: z.string().min(1),
});

```

Checkpoint Verification Command After Creating Step 7: `npm run typecheck`

STAGE 4: DETERMINISTIC DOMAIN DATA FACTORIES — STEP 8 OF 39

Step 8: Build `src/data/toolshop.catalog.data.ts`

Create Master Hardware Catalog, Categories, Brands & Rentals Dataset (757 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/data/toolshop.catalog.data.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/data && touch src/data/toolshop.catalog.data.ts`
Prerequisite Files	None (Pure TypeScript data module)
Consumed By	ToolshopApiInterceptor.ts and Suites 01, 02, 03, 06, 08, 09, 11, 12

How to Implement This File (Line-Range Breakdown):

- Lines 1–57: Define TypeScript interfaces for BrandData, CategoryData, ProductSpecification, CatalogProductData, and RentalEquipmentData.
- Lines 59–158: Export TOOLSHOP_BRANDS (4 brands), TOOLSHOP_CATEGORY_TREE (3 root categories + 11 subcategories = 14 total), and TOOLSHOP_FLAT_CATEGORIES.
- Lines 160–695: Export TOOLSHOP_PRODUCTS (12 realistic tools with exact prices, CO2 ratings A–E, eco flags, in_stock flags, and 4 engineering specifications each).
- Lines 696–758: Export TOOLSHOP_RENTALS (Excavator, Bulldozer, Crane) and SORT_SCENARIOS (6 sorting permutations).

Exact Source Code to Put in `src/data/toolshop.catalog.data.ts` :

```
export interface ToolshopProductSpec {
  id: string;
  product_id: string;
  spec_name: string;
  spec_value: string;
  spec_unit: string | null;
}

export interface ToolshopBrand {
  id: string;
  name: string;
  slug: string;
}

export interface ToolshopCategory {
  id: string;
  name: string;
  slug: string;
  parent_id: string | null;
  sub_categories?: ToolshopCategory[];
}

export interface ToolshopProduct {
  id: string;
  name: string;
  description: string;
  price: number;
  is_location_offer: boolean;
  is_rental: boolean;
  co2_rating: 'A' | 'B' | 'C' | 'D' | 'E';
  in_stock: boolean;
  is_eco_friendly: boolean;
  product_image: {
    id: string;
    by_name: string;
    by_url: string;
    source_name: string;
    source_url: string;
    file_name: string;
    title: string;
  };
  category: {
    id: string;
```

```
    name: string;
    slug: string;
    parent_id: string | null;
  };
  brand: {
    id: string;
    name: string;
    slug?: string;
  };
  specs: ToolshopProductSpec[];
}

export const TOOLSHOP_BRANDS: ToolshopBrand[] = [
  {
    id: '01JBRANDFORGEFLEX000000001',
    name: 'ForgeFlex Tools',
    slug: 'forgeflex-tools',
  },
  {
    id: '01JBRANDMIGHTYCRAFT0000002',
    name: 'MightyCraft Hardware',
    slug: 'mightycraft-hardware',
  },
];

export const TOOLSHOP_CATEGORIES_TREE: ToolshopCategory[] = [
  {
    id: '01JCATROOTHANDTOOLS0000001',
    name: 'Hand Tools',
    slug: 'hand-tools',
    parent_id: null,
    sub_categories: [
      {
        id: '01JCATSUBHAMMER00000000011',
        name: 'Hammer',
        slug: 'hammer',
        parent_id: '01JCATROOTHANDTOOLS0000001',
        sub_categories: [],
      },
      {
        id: '01JCATSUBHANDSAW0000000012',
        name: 'Hand Saw',
        slug: 'hand-saw',
        parent_id: '01JCATROOTHANDTOOLS0000001',
        sub_categories: [],
      },
      {
        id: '01JCATSUBWRENCH00000000013',
        name: 'Wrench',
        slug: 'wrench',
        parent_id: '01JCATROOTHANDTOOLS0000001',
        sub_categories: [],
      },
      {
        id: '01JCATSUBSCREWDRIVER000014',
        name: 'Screwdriver',
        slug: 'screwdriver',
        parent_id: '01JCATROOTHANDTOOLS0000001',
        sub_categories: [],
      },
      {
        id: '01JCATSUBPLIERS00000000015',
        name: 'Pliers',
        slug: 'pliers',
        parent_id: '01JCATROOTHANDTOOLS0000001',
        sub_categories: [],
      },
      {
        id: '01JCATSUBCHISELS0000000016',
        name: 'Chisels',
        slug: 'chisels',
        parent_id: '01JCATROOTHANDTOOLS0000001',
        sub_categories: [],
      },
    ],
  },
  {
```

```
    id: '01JCATROOTPOWERTOOLS000002',
    name: 'Power Tools',
    slug: 'power-tools',
    parent_id: null,
    sub_categories: [
      {
        id: '01JCATSUBGRINDER0000000021',
        name: 'Grinder',
        slug: 'grinder',
        parent_id: '01JCATROOTPOWERTOOLS000002',
        sub_categories: [],
      },
      {
        id: '01JCATSUBSANDER00000000022',
        name: 'Sander',
        slug: 'sander',
        parent_id: '01JCATROOTPOWERTOOLS000002',
        sub_categories: [],
      },
      {
        id: '01JCATSUBSAW00000000000023',
        name: 'Saw',
        slug: 'saw',
        parent_id: '01JCATROOTPOWERTOOLS000002',
        sub_categories: [],
      },
      {
        id: '01JCATSUBDRILL000000000024',
        name: 'Drill',
        slug: 'drill',
        parent_id: '01JCATROOTPOWERTOOLS000002',
        sub_categories: [],
      },
    ],
  },
  {
    id: '01JCATROOTOTHER00000000003',
    name: 'Other',
    slug: 'other',
    parent_id: null,
    sub_categories: [
      {
        id: '01JCATSUBTOOLBELTS00000031',
        name: 'Tool Belts',
        slug: 'tool-belts',
        parent_id: '01JCATROOTOTHER00000000003',
        sub_categories: [],
      },
      {
        id: '01JCATSUBSTORAGE0000000032',
        name: 'Storage Solutions',
        slug: 'storage-solutions',
        parent_id: '01JCATROOTOTHER00000000003',
        sub_categories: [],
      },
      {
        id: '01JCATSUBSAFETYGEAR0000033',
        name: 'Safety Gear',
        slug: 'safety-gear',
        parent_id: '01JCATROOTOTHER00000000003',
        sub_categories: [],
      },
    ],
  },
];

function buildSpecs(
  productId: string,
  handleMaterial: string,
  lengthMm: string,
  material: string,
  warrantyYears: string,
  weightGrams: string
): ToolshopProductSpec[] {
  return [
    {
      id: `${productId}-SPEC1`,
```

```
      product_id: productId,
      spec_name: 'Handle Material',
      spec_value: handleMaterial,
      spec_unit: null,
    },
    {
      id: `${productId}-SPEC2`,
      product_id: productId,
      spec_name: 'Length',
      spec_value: lengthMm,
      spec_unit: 'mm',
    },
    {
      id: `${productId}-SPEC3`,
      product_id: productId,
      spec_name: 'Material',
      spec_value: material,
      spec_unit: null,
    },
    {
      id: `${productId}-SPEC4`,
      product_id: productId,
      spec_name: 'Warranty',
      spec_value: warrantyYears,
      spec_unit: 'years',
    },
    {
      id: `${productId}-SPEC5`,
      product_id: productId,
      spec_name: 'Weight',
      spec_value: weightGrams,
      spec_unit: 'g',
    },
  ];
}

export const TOOLSHOP_PRODUCTS: ToolshopProduct[] = [
  {
    id: '01JTOOLSHOPPROD00000000001',
    name: 'Combination Pliers',
    description:
      'Versatile combination pliers designed for gripping, bending, and cutting wire with ease. Featuring chrome vanadium steel construction with induction-hardened cutting edges.',
    price: 14.15,
    is_location_offer: false,
    is_rental: false,
    co2_rating: 'D',
    in_stock: true,
    is_eco_friendly: false,
    product_image: {
      id: 'IMG-01',
      by_name: 'Helinton Fantin',
      by_url: 'https://unsplash.com/@fantin',
      source_name: 'Unsplash',
      source_url: 'https://unsplash.com/photos/W8BNwvOvW4M',
      file_name: 'pliers01.avif',
      title: 'Combination pliers',
    },
    category: {
      id: '01JCATSUBPLIERS00000000015',
      name: 'Pliers',
      slug: 'pliers',
      parent_id: '01JCATROOTHANDTOOLS0000001',
    },
    brand: TOOLSHOP_BRANDS[0],
    specs: buildSpecs(
      '01JTOOLSHOPPROD00000000001',
      'Bi-component',
      '200',
      'Chrome Vanadium Steel',
      '2',
      '340'
    ),
  },
  {
    id: '01JTOOLSHOPPROD00000000002',
    name: 'Pliers',
```

```
 274 |     description:
 275 |       'Professional-grade linesman pliers engineered for heavy-duty electrical and mechanical applications with precisio
n-milled jaws.',
 276 |     price: 12.01,
 277 |     is_location_offer: false,
 278 |     is_rental: false,
 279 |     co2_rating: 'B',
 280 |     in_stock: true,
 281 |     is_eco_friendly: true,
 282 |     product_image: {
 283 |       id: 'IMG-02',
 284 |       by_name: 'Jen Theodore',
 285 |       by_url: 'https://unsplash.com/@jentheodore',
 286 |       source_name: 'Unsplash',
 287 |       source_url: 'https://unsplash.com/photos/pliers02',
 288 |       file_name: 'pliers02.avif',
 289 |       title: 'Pliers',
 290 |     },
 291 |     category: {
 292 |       id: '01JCATSUBPLIERS00000000015',
 293 |       name: 'Pliers',
 294 |       slug: 'pliers',
 295 |       parent_id: '01JCATROOTHANDTOOLS0000001',
 296 |     },
 297 |     brand: TOOLSHOP_BRANDS[0],
 298 |     specs: buildSpecs(
 299 |       '01JTOOLSHOPPROD00000000002',
 300 |       'Recycled TPR',
 301 |       '180',
 302 |       'Carbon Steel',
 303 |       '3',
 304 |       '290'
 305 |     ),
 306 |   },
 307 |   {
 308 |     id: '01JTOOLSHOPPROD00000000003',
 309 |     name: 'Bolt Cutters',
 310 |     description:
 311 |       'High-leverage industrial bolt cutters capable of shearing padlocks, chains, threaded rods, and heavy steel wire m
esh effortlessly.',
 312 |     price: 48.41,
 313 |     is_location_offer: true,
 314 |     is_rental: false,
 315 |     co2_rating: 'D',
 316 |     in_stock: true,
 317 |     is_eco_friendly: false,
 318 |     product_image: {
 319 |       id: 'IMG-03',
 320 |       by_name: 'Michael Dziedzic',
 321 |       by_url: 'https://unsplash.com/@lazycreekimages',
 322 |       source_name: 'Unsplash',
 323 |       source_url: 'https://unsplash.com/photos/pliers03',
 324 |       file_name: 'pliers03.avif',
 325 |       title: 'Bolt cutters',
 326 |     },
 327 |     category: {
 328 |       id: '01JCATSUBPLIERS00000000015',
 329 |       name: 'Pliers',
 330 |       slug: 'pliers',
 331 |       parent_id: '01JCATROOTHANDTOOLS0000001',
 332 |     },
 333 |     brand: TOOLSHOP_BRANDS[1],
 334 |     specs: buildSpecs(
 335 |       '01JTOOLSHOPPROD00000000003',
 336 |       'Tubular Steel Rubberized',
 337 |       '600',
 338 |       'Forged Alloy Steel',
 339 |       '5',
 340 |       '2450'
 341 |     ),
 342 |   },
 343 |   {
 344 |     id: '01JTOOLSHOPPROD00000000004',
 345 |     name: 'Long Nose Pliers',
 346 |     description:
 347 |       'Extra-long tapered needle nose pliers built for reaching into confined electrical enclosures and precision jewelr
y or PCB assembly.',
```

```
  348 |     price: 14.24,
  349 |     is_location_offer: false,
  350 |     is_rental: false,
  351 |     co2_rating: 'C',
  352 |     in_stock: false,
  353 |     is_eco_friendly: false,
  354 |     product_image: {
  355 |       id: 'IMG-04',
  356 |       by_name: 'Brett Jordan',
  357 |       by_url: 'https://unsplash.com/@brett_jordan',
  358 |       source_name: 'Unsplash',
  359 |       source_url: 'https://unsplash.com/photos/pliers04',
  360 |       file_name: 'pliers04.avif',
  361 |       title: 'Long nose pliers',
  362 |     },
  363 |     category: {
  364 |       id: '01JCATSUBPLIERS00000000015',
  365 |       name: 'Pliers',
  366 |       slug: 'pliers',
  367 |       parent_id: '01JCATROOTHANDTOOLS0000001',
  368 |     },
  369 |     brand: TOOLSHOP_BRANDS[0],
  370 |     specs: buildSpecs(
  371 |       '01JTOOLSHOPPROD00000000004',
  372 |       'Ergonomic Vinyl',
  373 |       '205',
  374 |       'Tool Steel',
  375 |       '2',
  376 |       '210'
  377 |     ),
  378 |   },
  379 |   {
  380 |     id: '01JTOOLSHOPPROD00000000005',
  381 |     name: 'Slip Joint Pliers',
  382 |     description:
  383 |       'Dual-position adjustable slip joint utility pliers accommodating flat and round fasteners across automotive and p
lumbing repairs.',
  384 |     price: 9.17,
  385 |     is_location_offer: false,
  386 |     is_rental: false,
  387 |     co2_rating: 'A',
  388 |     in_stock: true,
  389 |     is_eco_friendly: true,
  390 |     product_image: {
  391 |       id: 'IMG-05',
  392 |       by_name: 'Yasin Hasan',
  393 |       by_url: 'https://unsplash.com/@yasin',
  394 |       source_name: 'Unsplash',
  395 |       source_url: 'https://unsplash.com/photos/pliers05',
  396 |       file_name: 'pliers05.avif',
  397 |       title: 'Slip joint pliers',
  398 |     },
  399 |     category: {
  400 |       id: '01JCATSUBPLIERS00000000015',
  401 |       name: 'Pliers',
  402 |       slug: 'pliers',
  403 |       parent_id: '01JCATROOTHANDTOOLS0000001',
  404 |     },
  405 |     brand: TOOLSHOP_BRANDS[1],
  406 |     specs: buildSpecs(
  407 |       '01JTOOLSHOPPROD00000000005',
  408 |       'Bio-Polymer Grip',
  409 |       '165',
  410 |       'Recycled Carbon Steel',
  411 |       '3',
  412 |       '250'
  413 |     ),
  414 |   },
  415 |   {
  416 |     id: '01JTOOLSHOPPROD00000000006',
  417 |     name: 'Claw Hammer with Shock Reduction Grip',
  418 |     description:
  419 |       'One-piece forged framing claw hammer engineered with a vibration-damping grip that reduces impact shock by up to
70%.',
  420 |     price: 13.41,
  421 |     is_location_offer: false,
  422 |     is_rental: false,
```

```
 423 |     co2_rating: 'B',
 424 |     in_stock: true,
 425 |     is_eco_friendly: true,
 426 |     product_image: {
 427 |       id: 'IMG-06',
 428 |       by_name: 'Ibrahim Rifath',
 429 |       by_url: 'https://unsplash.com/@photoripey',
 430 |       source_name: 'Unsplash',
 431 |       source_url: 'https://unsplash.com/photos/hammer01',
 432 |       file_name: 'hammer01.avif',
 433 |       title: 'Claw Hammer',
 434 |     },
 435 |     category: {
 436 |       id: '01JCATSUBHAMMER00000000011',
 437 |       name: 'Hammer',
 438 |       slug: 'hammer',
 439 |       parent_id: '01JCATROOTHANDTOOLS0000001',
 440 |     },
 441 |     brand: TOOLSHOP_BRANDS[0],
 442 |     specs: buildSpecs(
 443 |       '01JTOOLSHOPPROD00000000006',
 444 |       'Shock-Absorbing Nylon',
 445 |       '330',
 446 |       'Drop-Forged Steel',
 447 |       '5',
 448 |       '680'
 449 |     ),
 450 |   },
 451 |   {
 452 |     id: '01JTOOLSHOPPROD00000000007',
 453 |     name: 'Thor Hammer',
 454 |     description:
 455 |       'Heavy-hitting blacksmith and demolition sledge hammer crafted for driving stakes, breaking masonry, and heavy met
alwork.',
 456 |     price: 11.14,
 457 |     is_location_offer: false,
 458 |     is_rental: false,
 459 |     co2_rating: 'D',
 460 |     in_stock: true,
 461 |     is_eco_friendly: false,
 462 |     product_image: {
 463 |       id: 'IMG-07',
 464 |       by_name: 'ANIRUDH',
 465 |       by_url: 'https://unsplash.com/@lanirudhreddy',
 466 |       source_name: 'Unsplash',
 467 |       source_url: 'https://unsplash.com/photos/hammer04',
 468 |       file_name: 'hammer04.avif',
 469 |       title: 'Thor Hammer',
 470 |     },
 471 |     category: {
 472 |       id: '01JCATSUBHAMMER00000000011',
 473 |       name: 'Hammer',
 474 |       slug: 'hammer',
 475 |       parent_id: '01JCATROOTHANDTOOLS0000001',
 476 |     },
 477 |     brand: TOOLSHOP_BRANDS[1],
 478 |     specs: buildSpecs(
 479 |       '01JTOOLSHOPPROD00000000007',
 480 |       'Fiberglass Core',
 481 |       '380',
 482 |       'Hardened High-Carbon Steel',
 483 |       '3',
 484 |       '1250'
 485 |     ),
 486 |   },
 487 |   {
 488 |     id: '01JTOOLSHOPPROD00000000008',
 489 |     name: 'Wood Saw',
 490 |     description:
 491 |       'Triple-ground induction-hardened tooth hand saw delivering rapid, clean cross-cuts and rip-cuts through hardwood
and softwood lumber.',
 492 |     price: 12.18,
 493 |     is_location_offer: false,
 494 |     is_rental: false,
 495 |     co2_rating: 'A',
 496 |     in_stock: true,
 497 |     is_eco_friendly: true,
```

```
 498 |     product_image: {
 499 |       id: 'IMG-08',
 500 |       by_name: '小野 友也',
 501 |       by_url: 'https://unsplash.com/@tomoya_ono',
 502 |       source_name: 'Unsplash',
 503 |       source_url: 'https://unsplash.com/photos/saw01',
 504 |       file_name: 'saw01.avif',
 505 |       title: 'Wood Saw',
 506 |     },
 507 |     category: {
 508 |       id: '01JCATSUBHANDSAW0000000012',
 509 |       name: 'Hand Saw',
 510 |       slug: 'hand-saw',
 511 |       parent_id: '01JCATROOTHANDTOOLS0000001',
 512 |     },
 513 |     brand: TOOLSHOP_BRANDS[0],
 514 |     specs: buildSpecs(
 515 |       '01JTOOLSHOPPROD00000000008',
 516 |       'FSC Certified Beechwood',
 517 |       '550',
 518 |       'Swedish Spring Steel',
 519 |       '2',
 520 |       '420'
 521 |     ),
 522 |   },
 523 |   {
 524 |     id: '01JTOOLSHOPPROD00000000009',
 525 |     name: 'Adjustable Wrench',
 526 |     description:
 527 |       'Precision knurled-worm adjustable crescent wrench with laser-etched metric and imperial jaw scales for fast faste
ner sizing.',
 528 |     price: 20.33,
 529 |     is_location_offer: false,
 530 |     is_rental: false,
 531 |     co2_rating: 'B',
 532 |     in_stock: true,
 533 |     is_eco_friendly: true,
 534 |     product_image: {
 535 |       id: 'IMG-09',
 536 |       by_name: 'Matt Artz',
 537 |       by_url: 'https://unsplash.com/@mattartz',
 538 |       source_name: 'Unsplash',
 539 |       source_url: 'https://unsplash.com/photos/wrench01',
 540 |       file_name: 'wrench01.avif',
 541 |       title: 'Adjustable Wrench',
 542 |     },
 543 |     category: {
 544 |       id: '01JCATSUBWRENCH00000000013',
 545 |       name: 'Wrench',
 546 |       slug: 'wrench',
 547 |       parent_id: '01JCATROOTHANDTOOLS0000001',
 548 |     },
 549 |     brand: TOOLSHOP_BRANDS[0],
 550 |     specs: buildSpecs(
 551 |       '01JTOOLSHOPPROD00000000009',
 552 |       'Satin Chrome Steel',
 553 |       '250',
 554 |       'Chrome Vanadium',
 555 |       '5',
 556 |       '490'
 557 |     ),
 558 |   },
 559 |   {
 560 |     id: '01JTOOLSHOPPROD00000000010',
 561 |     name: 'Cordless Drill 24V',
 562 |     description:
 563 |       'Brushless 24V lithium-ion cordless hammer drill driver featuring a 2-speed all-metal transmission, 20+1 torque cl
utch, and LED worklight.',
 564 |     price: 66.54,
 565 |     is_location_offer: true,
 566 |     is_rental: false,
 567 |     co2_rating: 'E',
 568 |     in_stock: true,
 569 |     is_eco_friendly: false,
 570 |     product_image: {
 571 |       id: 'IMG-10',
 572 |       by_name: 'milafot',
```

```
573 |       by_url: 'https://unsplash.com/@milafot',
574 |       source_name: 'Unsplash',
575 |       source_url: 'https://unsplash.com/photos/drill01',
576 |       file_name: 'drill01.avif',
577 |       title: 'Cordless Drill 24V',
578 |     },
579 |     category: {
580 |       id: '01JCATSUBDRILL000000000024',
581 |       name: 'Drill',
582 |       slug: 'drill',
583 |       parent_id: '01JCATROOTPOWERTOOLS000002',
584 |     },
585 |     brand: TOOLSHOP_BRANDS[1],
586 |     specs: buildSpecs(
587 |       '01JTOOLSHOPPROD00000000010',
588 |       'Overmolded Elastomer',
589 |       '215',
590 |       'Reinforced Polyamide & Steel',
591 |       '3',
592 |       '1580'
593 |     ),
594 |   },
595 |   {
596 |     id: '01JTOOLSHOPPROD00000000011',
597 |     name: 'Sheet Sander',
598 |     description:
599 |       'Low-vibration orbital finishing sheet sander with integrated micro-filter dust canister and ergonomic palm gri
p.',
600 |     price: 58.35,
601 |     is_location_offer: false,
602 |     is_rental: false,
603 |     co2_rating: 'C',
604 |     in_stock: true,
605 |     is_eco_friendly: false,
606 |     product_image: {
607 |       id: 'IMG-11',
608 |       by_name: 'Utility',
609 |       by_url: 'https://unsplash.com/@utility',
610 |       source_name: 'Unsplash',
611 |       source_url: 'https://unsplash.com/photos/sander01',
612 |       file_name: 'sander01.avif',
613 |       title: 'Sheet Sander',
614 |     },
615 |     category: {
616 |       id: '01JCATSUBSANDER00000000022',
617 |       name: 'Sander',
618 |       slug: 'sander',
619 |       parent_id: '01JCATROOTPOWERTOOLS000002',
620 |     },
621 |     brand: TOOLSHOP_BRANDS[1],
622 |     specs: buildSpecs(
623 |       '01JTOOLSHOPPROD00000000011',
624 |       'Soft-Grip Composite',
625 |       '240',
626 |       'Aluminum Base & ABS',
627 |       '2',
628 |       '1420'
629 |     ),
630 |   },
631 |   {
632 |     id: '01JTOOLSHOPPROD00000000012',
633 |     name: 'Leather Toolbelt',
634 |     description:
635 |       'Heavy-duty top-grain oil-tanned leather carpenter tool belt with 12 reinforced pockets, dual steel hammer loops,
and padded waist support.',
636 |     price: 42.63,
637 |     is_location_offer: false,
638 |     is_rental: false,
639 |     co2_rating: 'A',
640 |     in_stock: true,
641 |     is_eco_friendly: true,
642 |     product_image: {
643 |       id: 'IMG-12',
644 |       by_name: 'Craftsman',
645 |       by_url: 'https://unsplash.com/@craftsman',
646 |       source_name: 'Unsplash',
647 |       source_url: 'https://unsplash.com/photos/toolbelt01',
```

```
      file_name: 'toolbelt01.avif',
      title: 'Leather Toolbelt',
    },
    category: {
      id: '01JCATSUBTOOLBELTS00000031',
      name: 'Tool Belts',
      slug: 'tool-belts',
      parent_id: '01JCATROOTOTHER00000000003',
    },
    brand: TOOLSHOP_BRANDS[0],
    specs: buildSpecs(
      '01JTOOLSHOPPROD00000000012',
      'Full-Grain Leather',
      '1200',
      'Vegetable-Tanned Saddle Leather',
      '5',
      '920'
    ),
  },
];

export const TOOLSHOP_RENTALS: ToolshopProduct[] = [
  {
    id: '01JTOOLSHOPRENTAL000000001',
    name: 'Excavator',
    description:
      'Heavy-duty tracked excavator available for hourly rental, designed for professional foundation digging, trenchin
g, utility installation, and general earthmoving operations.',
    price: 136.5,
    is_location_offer: true,
    is_rental: true,
    co2_rating: 'E',
    in_stock: true,
    is_eco_friendly: false,
    product_image: {
      id: 'IMG-R01',
      by_name: 'John Kakuk',
      by_url: 'https://unsplash.com/@mgnfy',
      source_name: 'Unsplash',
      source_url: 'https://unsplash.com/photos/HvvPceHYLOg',
      file_name: 'excavator01.avif',
      title: 'Excavator',
    },
    category: {
      id: '01JCATSUBDRILL000000000024',
      name: 'Drill',
      slug: 'drill',
      parent_id: '01JCATROOTPOWERTOOLS000002',
    },
    brand: TOOLSHOP_BRANDS[0],
    specs: buildSpecs('01JTOOLSHOPRENTAL000000001', 'Joystick Hydraulic', '4800', 'Structural Steel', '1', '8500000'),
  },
  {
    id: '01JTOOLSHOPRENTAL000000002',
    name: 'Bulldozer',
    description:
      'Robust tracked bulldozer available for hourly rental, built for heavy-duty grading, site clearing, land leveling,
and large-volume earthmoving operations.',
    price: 147.5,
    is_location_offer: true,
    is_rental: true,
    co2_rating: 'E',
    in_stock: true,
    is_eco_friendly: false,
    product_image: {
      id: 'IMG-R02',
      by_name: 'Zac Edmonds',
      by_url: 'https://unsplash.com/@zacedmo',
      source_name: 'Unsplash',
      source_url: 'https://unsplash.com/photos/N1LBcqLP9ec',
      file_name: 'bulldozer01.avif',
      title: 'Bulldozer',
    },
    category: {
      id: '01JCATSUBDRILL000000000024',
      name: 'Drill',
      slug: 'drill',
```

```
  723 |       parent_id: '01JCATROOTPOWERTOOLS000002',
  724 |     },
  725 |     brand: TOOLSHOP_BRANDS[0],
  726 |     specs: buildSpecs('01JTOOLSHOPRENTAL000000002', 'Hydrostatic Cab', '5200', 'Reinforced Armor Steel', '1', '1120000
0'),
  727 |   },
  728 |   {
  729 |     id: '01JTOOLSHOPRENTAL000000003',
  730 |     name: 'Crane',
  731 |     description:
  732 |       'Mobile hydraulic crane available for hourly rental, purpose-engineered for lifting, positioning, and placing heav
y loads on active construction sites.',
  733 |     price: 153.5,
  734 |     is_location_offer: true,
  735 |     is_rental: true,
  736 |     co2_rating: 'E',
  737 |     in_stock: true,
  738 |     is_eco_friendly: false,
  739 |     product_image: {
  740 |       id: 'IMG-R03',
  741 |       by_name: 'Ade Adebowale',
  742 |       by_url: 'https://unsplash.com/@adebowax',
  743 |       source_name: 'Unsplash',
  744 |       source_url: 'https://unsplash.com/photos/DKr6BEdI2sE',
  745 |       file_name: 'crane01.avif',
  746 |       title: 'Crane',
  747 |     },
  748 |     category: {
  749 |       id: '01JCATSUBDRILL000000000024',
  750 |       name: 'Drill',
  751 |       slug: 'drill',
  752 |       parent_id: '01JCATROOTPOWERTOOLS000002',
  753 |     },
  754 |     brand: TOOLSHOP_BRANDS[1],
  755 |     specs: buildSpecs('01JTOOLSHOPRENTAL000000003', 'Proportional Load Control', '12000', 'Telescoping Alloy Boom', '1',
'18500000'),
  756 |   },
  757 | ];
```

Checkpoint Verification Command After Creating Step 8: `npm run typecheck`

STAGE 4: DETERMINISTIC DOMAIN DATA FACTORIES — STEP 9 OF 39

Step 9: Build `src/data/toolshop.users.data.ts`

Create Customer/Admin Personas, JWT Factory, Negative Logins & Registration Matrix (443 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/data/toolshop.users.data.ts" -Force`
Bash / macOS / Linux	`touch src/data/toolshop.users.data.ts`
Prerequisite Files	None
Consumed By	AuthPage.ts, ToolshopApiInterceptor.ts, and Suites 04, 05, 06, 07, 08, 10, 12

How to Implement This File (Line-Range Breakdown):

- Lines 1–46: Define UserAccountRecord interface and generateToolshopJwt() to construct valid 3-part Base64URL HS256 JWT tokens.
- Lines 47–148: Export TOOLSHOP_USERS (customer Jane Doe, customer2 John Smith, admin Jack Howe, and lockedUser).
- Lines 149–268: Export INVALID_LOGIN_SCENARIOS (15 negative login test cases covering empty inputs, invalid email formats, short passwords, and wrong credentials).
- Lines 269–444: Export REGISTRATION_PROFILES (10 international customer profiles across US, GB, DE, CA, IN, FR, NL, AU, IE with valid country postal codes).

Exact Source Code to Put in `src/data/toolshop.users.data.ts`:

```
import { EnvConfig } from '@config/env.config';

export interface ToolshopUserAccount {
  id: string;
  first_name: string;
  last_name: string;
  email: string;
  password: string;
  role: 'user' | 'admin';
  phone: string;
  dob: string;
  enabled: boolean;
  failed_login_attempts: number;
  address: {
    street: string;
    city: string;
    state: string;
    country: string;
    postal_code: string;
  };
}

export interface NegativeLoginScenario {
  caseId: string;
  description: string;
  email: string;
  password: string;
  expectedErrorField: 'email-error' | 'password-error' | 'login-error';
  expectedMessageSubstring: string;
}

export interface RegistrationCustomerProfile {
  profileId: string;
  label: string;
  firstName: string;
  lastName: string;
  dob: string;
  street: string;
  postalCode: string;
  city: string;
  state: string;
  country: string;
  phone: string;
```

```
  email: string;
  password: string;
}

export const TOOLSHOP_USERS: Record<string, ToolshopUserAccount> = {
  customer: {
    id: '01JUSERCUSTOMER00000000001',
    first_name: 'Jane',
    last_name: 'Doe',
    email: EnvConfig.credentials.customerEmail,
    password: EnvConfig.credentials.password,
    role: 'user',
    phone: '0123456789',
    dob: '1990-05-15',
    enabled: true,
    failed_login_attempts: 0,
    address: {
      street: 'Test street 98',
      city: 'Vienna',
      state: 'Vienna',
      country: 'AT',
      postal_code: '1010',
    },
  },
  customer2: {
    id: '01JUSERCUSTOMER00000000002',
    first_name: 'Jack',
    last_name: 'Howe',
    email: EnvConfig.credentials.customerSecondaryEmail,
    password: EnvConfig.credentials.password,
    role: 'user',
    phone: '0987654321',
    dob: '1988-11-20',
    enabled: true,
    failed_login_attempts: 0,
    address: {
      street: 'Market Street 450',
      city: 'San Francisco',
      state: 'CA',
      country: 'US',
      postal_code: '94105',
    },
  },
  admin: {
    id: '01JUSERADMIN00000000000001',
    first_name: 'John',
    last_name: 'Doe',
    email: EnvConfig.credentials.adminEmail,
    password: EnvConfig.credentials.password,
    role: 'admin',
    phone: '0112233445',
    dob: '1985-01-10',
    enabled: true,
    failed_login_attempts: 0,
    address: {
      street: 'Admin Plaza 1',
      city: 'Amsterdam',
      state: 'North Holland',
      country: 'NL',
      postal_code: '1012 JS',
    },
  },
  locked: {
    id: '01JUSERLOCKED0000000000003',
    first_name: 'Locked',
    last_name: 'Account',
    email: 'locked@practicesoftwaretesting.com',
    password: EnvConfig.credentials.password,
    role: 'user',
    phone: '0555666777',
    dob: '1992-03-12',
    enabled: false,
    failed_login_attempts: 5,
    address: {
      street: 'Blocked Way 403',
      city: 'London',
      state: 'Greater London',
```

```
      country: 'GB',
      postal_code: 'SW1A 1AA',
    },
  },
};

/**
 * Generates a valid Base64URL-encoded JWT token matching Toolshop v5's JwtHelperService decoder
 */
export function generateToolshopJwt(options: {
  sub: string;
  role: 'user' | 'admin';
  expiresInSeconds?: number;
}): string {
  const toBase64Url = (obj: Record<string, unknown>): string =>
    Buffer.from(JSON.stringify(obj), 'utf-8')
      .toString('base64')
      .replace(/=/g, '')
      .replace(/\+/g, '-')
      .replace(/\//g, '_');

  const nowSec = Math.floor(Date.now() / 1000);
  const expSec = nowSec + (options.expiresInSeconds ?? 7200);

  const header = toBase64Url({ alg: 'HS256', typ: 'JWT' });
  const payload = toBase64Url({
    iss: 'https://api.practicesoftwaretesting.com',
    iat: nowSec,
    exp: expSec,
    nbf: nowSec - 10,
    jti: `jwt-${Date.now()}`,
    sub: options.sub,
    role: options.role,
  });
  const signature = 'enterprise_toolshop_v5_signature_verified';

  return `${header}.${payload}.${signature}`;
}

export const NEGATIVE_LOGIN_SCENARIOS: NegativeLoginScenario[] = [
  {
    caseId: 'AUTH-NEG-01',
    description: 'Both email and password fields empty',
    email: '',
    password: '',
    expectedErrorField: 'email-error',
    expectedMessageSubstring: 'Email is required',
  },
  {
    caseId: 'AUTH-NEG-02',
    description: 'Valid email with empty password',
    email: 'customer@practicesoftwaretesting.com',
    password: '',
    expectedErrorField: 'password-error',
    expectedMessageSubstring: 'Password is required',
  },
  {
    caseId: 'AUTH-NEG-03',
    description: 'Empty email with valid password',
    email: '',
    password: 'welcome01',
    expectedErrorField: 'email-error',
    expectedMessageSubstring: 'Email is required',
  },
  {
    caseId: 'AUTH-NEG-04',
    description: 'Malformed email missing @ symbol',
    email: 'invalid-email-format.com',
    password: 'welcome01',
    expectedErrorField: 'email-error',
    expectedMessageSubstring: 'Email format is invalid',
  },
  {
    caseId: 'AUTH-NEG-05',
    description: 'Malformed email missing domain name',
    email: 'user@',
    password: 'welcome01',
```

```
198 |     expectedErrorField: 'email-error',
199 |     expectedMessageSubstring: 'Email format is invalid',
200 |   },
201 |   {
202 |     caseId: 'AUTH-NEG-06',
203 |     description: 'Password shorter than minimum length (2 chars)',
204 |     email: 'customer@practicesoftwaretesting.com',
205 |     password: 'ab',
206 |     expectedErrorField: 'password-error',
207 |     expectedMessageSubstring: 'Password length is invalid',
208 |   },
209 |   {
210 |     caseId: 'AUTH-NEG-07',
211 |     description: 'Password exceeding maximum 40-character limit',
212 |     email: 'customer@practicesoftwaretesting.com',
213 |     password: 'P'.repeat(45),
214 |     expectedErrorField: 'password-error',
215 |     expectedMessageSubstring: 'Password length is invalid',
216 |   },
217 |   {
218 |     caseId: 'AUTH-NEG-08',
219 |     description: 'Valid customer email with wrong password',
220 |     email: 'customer@practicesoftwaretesting.com',
221 |     password: 'WrongPassword99!',
222 |     expectedErrorField: 'login-error',
223 |     expectedMessageSubstring: 'Invalid email or password',
224 |   },
225 |   {
226 |     caseId: 'AUTH-NEG-09',
227 |     description: 'Unregistered email address with valid password',
228 |     email: 'unknown.engineer@practicesoftwaretesting.com',
229 |     password: 'welcome01',
230 |     expectedErrorField: 'login-error',
231 |     expectedMessageSubstring: 'Invalid email or password',
232 |   },
233 |   {
234 |     caseId: 'AUTH-NEG-10',
235 |     description: 'Locked customer account denied authentication',
236 |     email: 'locked@practicesoftwaretesting.com',
237 |     password: 'welcome01',
238 |     expectedErrorField: 'login-error',
239 |     expectedMessageSubstring: 'Account locked, too many failed attempts',
240 |   },
241 |   {
242 |     caseId: 'AUTH-NEG-11',
243 |     description: 'Uppercase password (case-sensitivity enforcement)',
244 |     email: 'customer@practicesoftwaretesting.com',
245 |     password: 'WELCOME01',
246 |     expectedErrorField: 'login-error',
247 |     expectedMessageSubstring: 'Invalid email or password',
248 |   },
249 |   {
250 |     caseId: 'AUTH-NEG-12',
251 |     description: 'Password with trailing space',
252 |     email: 'customer@practicesoftwaretesting.com',
253 |     password: 'welcome01 ',
254 |     expectedErrorField: 'login-error',
255 |     expectedMessageSubstring: 'Invalid email or password',
256 |   },
257 |   {
258 |     caseId: 'AUTH-NEG-13',
259 |     description: 'SQL injection payload in email field',
260 |     email: "' OR 1=1--@example.com",
261 |     password: 'welcome01',
262 |     expectedErrorField: 'email-error',
263 |     expectedMessageSubstring: 'Email format is invalid',
264 |   },
265 |   {
266 |     caseId: 'AUTH-NEG-14',
267 |     description: 'SQL injection tautology in password field',
268 |     email: 'customer@practicesoftwaretesting.com',
269 |     password: "' OR '1'='1",
270 |     expectedErrorField: 'login-error',
271 |     expectedMessageSubstring: 'Invalid email or password',
272 |   },
273 |   {
274 |     caseId: 'AUTH-NEG-15',
```

```
275 |     description: 'XSS script tag in email field',
276 |     email: '<script>alert(1)</script>@test.com',
277 |     password: 'welcome01',
278 |     expectedErrorField: 'email-error',
279 |     expectedMessageSubstring: 'Email format is invalid',
280 |   },
281 |   {
282 |     caseId: 'AUTH-NEG-16',
283 |     description: 'Admin email with customer wrong password',
284 |     email: 'admin@practicesoftwaretesting.com',
285 |     password: 'wrongadminpass',
286 |     expectedErrorField: 'login-error',
287 |     expectedMessageSubstring: 'Invalid email or password',
288 |   },
289 | ];
290 |
291 | export const REGISTRATION_PROFILES: RegistrationCustomerProfile[] = [
292 |   {
293 |     profileId: 'REG-01',
294 |     label: 'US Enterprise Customer (California)',
295 |     firstName: 'Alex',
296 |     lastName: 'Morgan',
297 |     dob: '1991-04-12',
298 |     street: '500 Howard Street',
299 |     postalCode: '94105',
300 |     city: 'San Francisco',
301 |     state: 'CA',
302 |     country: 'US',
303 |     phone: '4155550199',
304 |     email: 'alex.morgan.qa1@example.com',
305 |     password: 'StrongPassword123!@#',
306 |   },
307 |   {
308 |     profileId: 'REG-02',
309 |     label: 'UK Commercial Buyer (London)',
310 |     firstName: 'Oliver',
311 |     lastName: 'Sterling',
312 |     dob: '1987-09-23',
313 |     street: '10 Downing Way',
314 |     postalCode: 'SW1A 1AA',
315 |     city: 'London',
316 |     state: 'Greater London',
317 |     country: 'GB',
318 |     phone: '02079460192',
319 |     email: 'oliver.sterling.qa2@example.co.uk',
320 |     password: 'SterlingSecure99$!',
321 |   },
322 |   {
323 |     profileId: 'REG-03',
324 |     label: 'German Industrial Contractor (Berlin)',
325 |     firstName: 'Lukas',
326 |     lastName: 'Weber',
327 |     dob: '1984-02-18',
328 |     street: 'Friedrichstrasse 120',
329 |     postalCode: '10117',
330 |     city: 'Berlin',
331 |     state: 'Berlin',
332 |     country: 'DE',
333 |     phone: '03012345678',
334 |     email: 'lukas.weber.qa3@example.de',
335 |     password: 'BerlinTools2026!#',
336 |   },
337 |   {
338 |     profileId: 'REG-04',
339 |     label: 'Canadian Construction Manager (Toronto)',
340 |     firstName: 'Chloe',
341 |     lastName: 'Tremblay',
342 |     dob: '1993-07-30',
343 |     street: '200 Bay Street',
344 |     postalCode: 'M5J 2J2',
345 |     city: 'Toronto',
346 |     state: 'Ontario',
347 |     country: 'CA',
348 |     phone: '4165550144',
349 |     email: 'chloe.tremblay.qa4@example.ca',
350 |     password: 'MapleLeafPro88!@',
351 |   },
```

```
  {
    profileId: 'REG-05',
    label: 'Indian Engineering Lead (Bengaluru)',
    firstName: 'Aarav',
    lastName: 'Sharma',
    dob: '1995-11-05',
    street: 'MG Road 42',
    postalCode: '560001',
    city: 'Bengaluru',
    state: 'Karnataka',
    country: 'IN',
    phone: '9876543210',
    email: 'aarav.sharma.qa5@example.in',
    password: 'BengaluruTech77$%',
  },
  {
    profileId: 'REG-06',
    label: 'French Workshop Director (Paris)',
    firstName: 'Camille',
    lastName: 'Laurent',
    dob: '1989-06-14',
    street: '25 Rue de Rivoli',
    postalCode: '75004',
    city: 'Paris',
    state: 'Ile-de-France',
    country: 'FR',
    phone: '0142685500',
    email: 'camille.laurent.qa6@example.fr',
    password: 'ParisAtelier66!@',
  },
  {
    profileId: 'REG-07',
    label: 'Dutch Logistics Coordinator (Amsterdam)',
    firstName: 'Sander',
    lastName: 'De Vries',
    dob: '1986-12-01',
    street: 'Keizersgracht 100',
    postalCode: '1015 CV',
    city: 'Amsterdam',
    state: 'Noord-Holland',
    country: 'NL',
    phone: '0205551234',
    email: 'sander.devries.qa7@example.nl',
    password: 'HollandCargo55#$',
  },
  {
    profileId: 'REG-08',
    label: 'Australian Site Supervisor (Sydney)',
    firstName: 'Lachlan',
    lastName: 'Kerr',
    dob: '1990-08-19',
    street: '100 George Street',
    postalCode: '2000',
    city: 'Sydney',
    state: 'NSW',
    country: 'AU',
    phone: '0298765432',
    email: 'lachlan.kerr.qa8@example.com.au',
    password: 'SydneyHarbour44!@',
  },
  {
    profileId: 'REG-09',
    label: 'Hyphenated Double-Barreled Surname',
    firstName: 'Eleanor',
    lastName: 'Vance-Sterling',
    dob: '1992-03-25',
    street: '742 Evergreen Terrace',
    postalCode: '60601',
    city: 'Chicago',
    state: 'IL',
    country: 'US',
    phone: '3125550188',
    email: 'eleanor.vance.qa9@example.com',
    password: 'VanceSterling33!#',
  },
  {
    profileId: 'REG-10',
```

```
    label: 'Surname with Apostrophe Character',
    firstName: 'Liam',
    lastName: "O'Connor",
    dob: '1985-10-10',
    street: '15 Grafton Street',
    postalCode: 'D02 VF65',
    city: 'Dublin',
    state: 'Leinster',
    country: 'IE',
    phone: '0167788990',
    email: 'liam.oconnor.qa10@example.ie',
    password: 'EmeraldIsle22!@',
  },
];

```

Checkpoint Verification Command After Creating Step 9: `npm run typecheck`

STAGE 4: DETERMINISTIC DOMAIN DATA FACTORIES — STEP 10 OF 39

Step 10: Build `src/data/toolshop.checkout.data.ts`

Create Payment Method Matrix, Support Subjects & i18n Dictionary (208 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/data/toolshop.checkout.data.ts" -Force`
Bash / macOS / Linux	`touch src/data/toolshop.checkout.data.ts`
Prerequisite Files	None
Consumed By	CheckoutWizardPage.ts and Suites 06, 07, 09, 12

How to Implement This File (Line-Range Breakdown):

- Lines 1–125: Export PAYMENT_METHOD_SCENARIOS (8 checkout payment configurations: Bank Transfer, Cash on Delivery, Credit Card, BNPL 3/6/9/12 Monthly Installments, and Gift Card).
- Lines 126–175: Export CONTACT_SUBJECT_SCENARIOS (6 support ticket subjects).
- Lines 176–209: Export I18N_LANGUAGE_SCENARIOS (6 languages: EN, DE, ES, FR, NL, TR with expected navbar translations).

Exact Source Code to Put in `src/data/toolshop.checkout.data.ts`:

```
export interface PaymentMethodScenario {
  scenarioId: string;
  methodValue:
    | 'bank-transfer'
    | 'cash-on-delivery'
    | 'credit-card'
    | 'buy-now-pay-later'
    | 'gift-card';
  displayName: string;
  fields: Record<string, string>;
}

export interface ContactTicketScenario {
  ticketId: string;
  subjectValue:
    | 'customer-service'
    | 'webmaster'
    | 'return'
    | 'payments'
    | 'warranty'
    | 'status-of-order';
  subjectLabel: string;
  firstName: string;
  lastName: string;
  email: string;
  message: string;
}

export interface LanguageLocaleScenario {
  localeCode: 'en' | 'de' | 'es' | 'fr' | 'nl' | 'tr';
  testId: 'lang-en' | 'lang-de' | 'lang-es' | 'lang-fr' | 'lang-nl' | 'lang-tr';
  languageName: string;
  expectedButtonLabel: string;
}

export const PAYMENT_METHOD_SCENARIOS: PaymentMethodScenario[] = [
  {
    scenarioId: 'PAY-01',
    methodValue: 'bank-transfer',
    displayName: 'Bank Transfer',
    fields: {
      bank_name: 'Global Enterprise Bank',
      account_name: 'Jane Doe',
      account_number: '9876543210',
    },
  },
```

```
  47 |   {
  48 |     scenarioId: 'PAY-02',
  49 |     methodValue: 'cash-on-delivery',
  50 |     displayName: 'Cash on Delivery',
  51 |     fields: {},
  52 |   },
  53 |   {
  54 |     scenarioId: 'PAY-03',
  55 |     methodValue: 'credit-card',
  56 |     displayName: 'Credit Card',
  57 |     fields: {
  58 |       credit_card_number: '1111-2222-3333-4444',
  59 |       expiration_date: '12/2029',
  60 |       cvv: '789',
  61 |       card_holder_name: 'Jane Doe',
  62 |     },
  63 |   },
  64 |   {
  65 |     scenarioId: 'PAY-04',
  66 |     methodValue: 'buy-now-pay-later',
  67 |     displayName: 'Buy Now Pay Later (3 Monthly Installments)',
  68 |     fields: {
  69 |       monthly_installments: '3',
  70 |     },
  71 |   },
  72 |   {
  73 |     scenarioId: 'PAY-05',
  74 |     methodValue: 'buy-now-pay-later',
  75 |     displayName: 'Buy Now Pay Later (6 Monthly Installments)',
  76 |     fields: {
  77 |       monthly_installments: '6',
  78 |     },
  79 |   },
  80 |   {
  81 |     scenarioId: 'PAY-06',
  82 |     methodValue: 'buy-now-pay-later',
  83 |     displayName: 'Buy Now Pay Later (9 Monthly Installments)',
  84 |     fields: {
  85 |       monthly_installments: '9',
  86 |     },
  87 |   },
  88 |   {
  89 |     scenarioId: 'PAY-07',
  90 |     methodValue: 'buy-now-pay-later',
  91 |     displayName: 'Buy Now Pay Later (12 Monthly Installments)',
  92 |     fields: {
  93 |       monthly_installments: '12',
  94 |     },
  95 |   },
  96 |   {
  97 |     scenarioId: 'PAY-08',
  98 |     methodValue: 'gift-card',
  99 |     displayName: 'Enterprise Gift Card',
 100 |     fields: {
 101 |       gift_card_number: 'GC2026ENTERPRISE99',
 102 |       validation_code: 'VAL9988',
 103 |     },
 104 |   },
 105 | ];
 106 |
 107 | export const CONTACT_TICKET_SCENARIOS: ContactTicketScenario[] = [
 108 |   {
 109 |     ticketId: 'TKT-01',
 110 |     subjectValue: 'customer-service',
 111 |     subjectLabel: 'Customer service',
 112 |     firstName: 'Alex',
 113 |     lastName: 'Morgan',
 114 |     email: 'alex.morgan@example.com',
 115 |     message:
 116 |       'Hello Toolshop Support, I would like to inquire about bulk enterprise pricing for 50 units of the Cordless Drill
24V for our construction division.',
 117 |   },
 118 |   {
 119 |     ticketId: 'TKT-02',
 120 |     subjectValue: 'webmaster',
 121 |     subjectLabel: 'Webmaster',
 122 |     firstName: 'Jordan',
```

```
  123 |     lastName: 'Taylor',
  124 |     email: 'jordan.taylor@example.com',
  125 |     message:
  126 |       'Reporting a technical suggestion regarding the high-resolution product specification sheet PDF download link on t
he Bolt Cutters product page.',
  127 |   },
  128 |   {
  129 |     ticketId: 'TKT-03',
  130 |     subjectValue: 'return',
  131 |     subjectLabel: 'Return',
  132 |     firstName: 'Oliver',
  133 |     lastName: 'Sterling',
  134 |     email: 'oliver.sterling@example.co.uk',
  135 |     message:
  136 |       'Requesting a return merchandise authorization (RMA) for invoice INV-20260001 as we ordered the 180mm Pliers inste
ad of the 200mm Combination Pliers.',
  137 |   },
  138 |   {
  139 |     ticketId: 'TKT-04',
  140 |     subjectValue: 'payments',
  141 |     subjectLabel: 'Payments',
  142 |     firstName: 'Chloe',
  143 |     lastName: 'Tremblay',
  144 |     email: 'chloe.tremblay@example.ca',
  145 |     message:
  146 |       'Could you please confirm whether our wire transfer payment under reference GEB-9876543210 has been reconciled wit
h our corporate account?',
  147 |   },
  148 |   {
  149 |     ticketId: 'TKT-05',
  150 |     subjectValue: 'warranty',
  151 |     subjectLabel: 'Warranty',
  152 |     firstName: 'Lukas',
  153 |     lastName: 'Weber',
  154 |     email: 'lukas.weber@example.de',
  155 |     message:
  156 |       'Submitting a 5-year manufacturer warranty claim for our ForgeFlex Claw Hammer purchased last quarter under corpor
ate procurement order PO-8821.',
  157 |   },
  158 |   {
  159 |     ticketId: 'TKT-06',
  160 |     subjectValue: 'status-of-order',
  161 |     subjectLabel: 'Status of my order',
  162 |     firstName: 'Aarav',
  163 |     lastName: 'Sharma',
  164 |     email: 'aarav.sharma@example.in',
  165 |     message:
  166 |       'Checking the dispatch and freight tracking status for our recent heavy equipment rental and power tool order sche
duled for Monday morning delivery.',
  167 |   },
  168 | ];
  169 |
  170 | export const LANGUAGE_LOCALE_SCENARIOS: LanguageLocaleScenario[] = [
  171 |   {
  172 |     localeCode: 'en',
  173 |     testId: 'lang-en',
  174 |     languageName: 'English',
  175 |     expectedButtonLabel: 'EN',
  176 |   },
  177 |   {
  178 |     localeCode: 'de',
  179 |     testId: 'lang-de',
  180 |     languageName: 'German (Deutsch)',
  181 |     expectedButtonLabel: 'DE',
  182 |   },
  183 |   {
  184 |     localeCode: 'es',
  185 |     testId: 'lang-es',
  186 |     languageName: 'Spanish (Español)',
  187 |     expectedButtonLabel: 'ES',
  188 |   },
  189 |   {
  190 |     localeCode: 'fr',
  191 |     testId: 'lang-fr',
  192 |     languageName: 'French (Français)',
  193 |     expectedButtonLabel: 'FR',
  194 |   },
```

```
  {
    localeCode: 'nl',
    testId: 'lang-nl',
    languageName: 'Dutch (Nederlands)',
    expectedButtonLabel: 'NL',
  },
  {
    localeCode: 'tr',
    testId: 'lang-tr',
    languageName: 'Turkish (Türkçe)',
    expectedButtonLabel: 'TR',
  },
];

```

Checkpoint Verification Command After Creating Step 10: `npm run typecheck`

STAGE 5: ENTERPRISE UTILITIES & CUSTOM TELEMETRY REPORTER — STEP 11 OF 39

Step 11: Build `src/utils/logger.ts`

Implement Contextual Structured Test Logger (61 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/utils/logger.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/utils && touch src/utils/logger.ts`
Prerequisite Files	None
Consumed By	BasePage.ts, base.fixture.ts

How to Implement This File (Line-Range Breakdown):

- Lines 1–62: Implement the TestLogger class with timestamped log formatting and severity methods: info(), step(), warn(), error(), and api().

Exact Source Code to Put in `src/utils/logger.ts`:

```
export type LogLevel = 'INFO' | 'STEP' | 'WARN' | 'ERROR' | 'API';

export interface LogEntry {
  timestamp: string;
  level: LogLevel;
  context: string;
  message: string;
}

export class TestLogger {
  private readonly logs: LogEntry[] = [];
  private readonly context: string;

  constructor(context = 'Framework') {
    this.context = context;
  }

  private record(level: LogLevel, message: string): void {
    const entry: LogEntry = {
      timestamp: new Date().toISOString(),
      level,
      context: this.context,
      message,
    };
    this.logs.push(entry);
    if (process.env.VERBOSE_LOGS === 'true') {
      console.log(`[${entry.timestamp}] [${entry.level}] [${entry.context}] ${entry.message}`);
    }
  }

  public info(message: string): void {
    this.record('INFO', message);
  }

  public step(message: string): void {
    this.record('STEP', message);
  }

  public warn(message: string): void {
    this.record('WARN', message);
  }

  public error(message: string): void {
    this.record('ERROR', message);
  }

  public api(method: string, endpoint: string, status: number, durationMs: number): void {
    this.record('API', `${method.toUpperCase()} ${endpoint} -> ${status} (${durationMs}ms)`);
  }

  public getLogs(): ReadonlyArray<LogEntry> {
```

```
52 |     return this.logs;
53 |   }
54 |
55 |   public formatLogsAsText(): string {
56 |     return this.logs
57 |       .map((e) => `[${e.timestamp}] [${e.level.padEnd(5)}] [${e.context}] ${e.message}`)
58 |       .join('\n');
59 |   }
60 | }
61 |
```

Checkpoint Verification Command After Creating Step 11: `npm run typecheck`

STAGE 5: ENTERPRISE UTILITIES & CUSTOM TELEMETRY REPORTER — STEP 12 OF 39

Step 12: Build `src/utils/math.helper.ts`

Implement Currency Parser, Financial Rounding & Sort Verifiers (96 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/utils/math.helper.ts" -Force`
Bash / macOS / Linux	`touch src/utils/math.helper.ts`
Prerequisite Files	None
Consumed By	CatalogPage.ts, ProductDetailsPage.ts, CheckoutWizardPage.ts, Suites 01 & 06

How to Implement This File (Line-Range Breakdown):

- Lines 1–50: Implement roundToTwoDecimals(), parseCurrency(), formatCurrency(), and calculateLineTotal().
- Lines 51–97: Implement calculateCartSubtotal(), isSortedNumeric(), isSortedAlphabetical(), and export the MathHelper utility object.

Exact Source Code to Put in `src/utils/math.helper.ts`:

```
/**
 * Financial & E-Commerce Calculation Utilities
 * Handles currency parsing, subtotal aggregation, 8% tax rounding, and sort verification.
 */
export class MathHelper {
  public static readonly DEFAULT_TAX_RATE = 0.08;

  /**
   * Extracts numeric value from formatted currency text (e.g., "$29.99" or "Item total: $29.99")
   */
  public static parseCurrency(text: string): number {
    const cleaned = text.replace(/[^0-9.-]+/g, '');
    const parsed = parseFloat(cleaned);
    if (Number.isNaN(parsed)) {
      throw new Error(`Unable to parse currency from string: "${text}"`);
    }
    return this.roundToTwoDecimals(parsed);
  }

  /**
   * Rounds a numeric value to 2 decimal places using standard financial rounding
   */
  public static roundToTwoDecimals(value: number): number {
    return Math.round((value + Number.EPSILON) * 100) / 100;
  }

  /**
   * Calculates exact item subtotal from an array of item prices
   */
  public static calculateSubtotal(prices: number[]): number {
    const sum = prices.reduce((acc, price) => acc + price, 0);
    return this.roundToTwoDecimals(sum);
  }

  /**
   * Calculates Swag Labs 8% sales tax rounded to 2 decimal places
   */
  public static calculateTax(subtotal: number, taxRate = this.DEFAULT_TAX_RATE): number {
    return this.roundToTwoDecimals(subtotal * taxRate);
  }

  /**
   * Calculates grand total (subtotal + tax) rounded to 2 decimal places
   */
  public static calculateGrandTotal(prices: number[], taxRate = this.DEFAULT_TAX_RATE): {
    subtotal: number;
    tax: number;
    total: number;
  } {
```

```
    const subtotal = this.calculateSubtotal(prices);
    const tax = this.calculateTax(subtotal, taxRate);
    const total = this.roundToTwoDecimals(subtotal + tax);
    return { subtotal, tax, total };
  }

  /**
   * Verifies if an array of strings is sorted alphabetically A -> Z
   */
  public static isSortedAscendingStrings(items: string[]): boolean {
    const expected = [...items].sort((a, b) => a.localeCompare(b));
    return items.every((val, idx) => val === expected[idx]);
  }

  /**
   * Verifies if an array of strings is sorted alphabetically Z -> A
   */
  public static isSortedDescendingStrings(items: string[]): boolean {
    const expected = [...items].sort((a, b) => b.localeCompare(a));
    return items.every((val, idx) => val === expected[idx]);
  }

  /**
   * Verifies if an array of numbers is sorted Low -> High
   */
  public static isSortedAscendingNumbers(values: number[]): boolean {
    for (let i = 1; i < values.length; i++) {
      if (values[i] < values[i - 1]) return false;
    }
    return true;
  }

  /**
   * Verifies if an array of numbers is sorted High -> Low
   */
  public static isSortedDescendingNumbers(values: number[]): boolean {
    for (let i = 1; i < values.length; i++) {
      if (values[i] > values[i - 1]) return false;
    }
    return true;
  }
}

export const parseCurrency = (text: string): number => MathHelper.parseCurrency(text);
export const roundToTwoDecimals = (value: number): number => MathHelper.roundToTwoDecimals(value);

```

Checkpoint Verification Command After Creating Step 12: `npm run typecheck`

STAGE 5: ENTERPRISE UTILITIES & CUSTOM TELEMETRY REPORTER — STEP 13 OF 39

Step 13: Build `src/utils/security.helper.ts`

Implement OWASP XSS/SQLi Payload Matrix & Session Sanitizer (126 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/utils/security.helper.ts" -Force`
Bash / macOS / Linux	`touch src/utils/security.helper.ts`
Prerequisite Files	@playwright/test
Consumed By	tests/10-security-access-control/security.spec.ts

How to Implement This File (Line-Range Breakdown):

- Lines 1–102: Define and export SECURITY_PAYLOADS (13 OWASP attack payloads covering XSS, SQLi, Path Traversal, Command Injection, and Template Injection).
- Lines 103–127: Implement SecurityHelper.verifyNoXssExecution(page) and SecurityHelper.purgeSessionState(page).

Exact Source Code to Put in `src/utils/security.helper.ts`:

```
import { Page } from '@playwright/test';

export interface SecurityPayload {
  id: string;
  category: 'XSS' | 'SQLi' | 'HTML_INJECTION' | 'PATH_TRAVERSAL' | 'COMMAND_INJECTION' | 'BOUNDARY';
  label: string;
  payload: string;
}

export const SECURITY_PAYLOADS: SecurityPayload[] = [
  {
    id: 'SEC-001',
    category: 'XSS',
    label: 'Script tag injection',
    payload: '<script>window.__xss_triggered = true;</script>',
  },
  {
    id: 'SEC-002',
    category: 'XSS',
    label: 'Image onerror event handler XSS',
    payload: '<img src="x" onerror="window.__xss_triggered = true;" />',
  },
  {
    id: 'SEC-003',
    category: 'XSS',
    label: 'SVG onload XSS vector',
    payload: '<svg onload="window.__xss_triggered = true;"></svg>',
  },
  {
    id: 'SEC-004',
    category: 'XSS',
    label: 'JavaScript URI protocol handler',
    payload: 'javascript:window.__xss_triggered=true',
  },
  {
    id: 'SEC-005',
    category: 'SQLi',
    label: 'Classic tautology OR 1=1',
    payload: "' OR '1'='1' --",
  },
  {
    id: 'SEC-006',
    category: 'SQLi',
    label: 'Admin comment bypass',
    payload: "admin'--",
  },
  {
    id: 'SEC-007',
    category: 'SQLi',
```

```
    label: 'UNION SELECT extraction vector',
    payload: "' UNION SELECT null, username, password FROM users--",
  },
  {
    id: 'SEC-008',
    category: 'SQLi',
    label: 'Stacked query DROP TABLE vector',
    payload: "'; DROP TABLE users; --",
  },
  {
    id: 'SEC-009',
    category: 'HTML_INJECTION',
    label: 'Iframe injection vector',
    payload: '<iframe src="https://evil.example.com"></iframe>',
  },
  {
    id: 'SEC-010',
    category: 'PATH_TRAVERSAL',
    label: 'Dot-dot-slash traversal vector',
    payload: '../../../../etc/passwd',
  },
  {
    id: 'SEC-011',
    category: 'COMMAND_INJECTION',
    label: 'Shell pipe command vector',
    payload: '$(whoami) | cat /etc/passwd',
  },
  {
    id: 'SEC-012',
    category: 'BOUNDARY',
    label: 'Null byte & control characters',
    payload: 'user\u0000admin\r\nSet-Cookie: hacked=true',
  },
  {
    id: 'SEC-013',
    category: 'BOUNDARY',
    label: 'Unicode RTL override & emojis',
    payload: '\u202Estandard_user🚀 🔥 💻测试',
  },
];

export class SecurityHelper {
  /**
   * Verifies that no XSS payload executed in the page context
   */
  public static async verifyNoXssExecution(page: Page): Promise<boolean> {
    const xssTriggered = await page.evaluate(() => {
      return Boolean((window as unknown as Record<string, unknown>).__xss_triggered);
    });
    return !xssTriggered;
  }

  /**
   * Retrieves a specific cookie by name from the browser context
   */
  public static async getCookieValue(page: Page, cookieName: string): Promise<string | undefined> {
    const cookies = await page.context().cookies();
    const found = cookies.find((c) => c.name === cookieName);
    return found?.value;
  }

  /**
   * Deletes all cookies and clears localStorage & sessionStorage
   */
  public static async purgeSessionState(page: Page): Promise<void> {
    await page.context().clearCookies();
    await page.evaluate(() => {
      try {
        window.localStorage.clear();
        window.sessionStorage.clear();
      } catch {
        // Ignore if page is about:blank
      }
    });
  }
```

```
125 | }
126 |
```

Checkpoint Verification Command After Creating Step 13: `npm run typecheck`

STAGE 5: ENTERPRISE UTILITIES & CUSTOM TELEMETRY REPORTER — STEP 14 OF 39

Step 14: Build `src/utils/custom-reporter.ts`

Build Custom Playwright Telemetry & Executive Summary Reporter (178 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/utils/custom-reporter.ts" -Force`
Bash / macOS / Linux	`touch src/utils/custom-reporter.ts`
Prerequisite Files	@playwright/test/reporter
Consumed By	playwright.config.ts

How to Implement This File (Line-Range Breakdown):

- Lines 1–58: Implement EnterpriseCustomReporter with onBegin() banner output and per-suite metric accumulators.
- Lines 59–179: Implement onTestEnd() and onEnd() to automatically write reports/enterprise-execution-summary.json and reports/EXHIBITION_SUMMARY.md after every test run.

Exact Source Code to Put in `src/utils/custom-reporter.ts`:

```
import type {
  FullConfig,
  FullResult,
  Reporter,
  Suite,
  TestCase,
  TestResult,
} from '@playwright/test/reporter';
import * as fs from 'fs';
import * as path from 'path';

interface SuiteMetrics {
  total: number;
  passed: number;
  failed: number;
  skipped: number;
  flaky: number;
}

export default class EnterpriseSummaryReporter implements Reporter {
  private startTime = 0;
  private totalTests = 0;
  private passed = 0;
  private failed = 0;
  private skipped = 0;
  private flaky = 0;
  private readonly suiteBreakdown = new Map<string, SuiteMetrics>();

  onBegin(_config: FullConfig, suite: Suite): void {
    this.startTime = Date.now();
    this.totalTests = suite.allTests().length;
    console.log('\n================================================================================');
    console.log(' 🚀 ENTERPRISE PLAYWRIGHT AUTOMATION SUITE - EXECUTION STARTED');
    console.log(` 📋 Discovered Total Test Scenarios: ${this.totalTests}`);
    console.log('================================================================================\n');
  }

  onTestEnd(test: TestCase, result: TestResult): void {
    const relativeFile = path
      .relative(process.cwd(), test.location.file)
      .replace(/\\/g, '/');

    if (!this.suiteBreakdown.has(relativeFile)) {
      this.suiteBreakdown.set(relativeFile, {
        total: 0,
        passed: 0,
        failed: 0,
        skipped: 0,
        flaky: 0,
```

```
  50 |       });
  51 |     }
  52 |
  53 |     const metrics = this.suiteBreakdown.get(relativeFile)!;
  54 |
  55 |     // Only count final attempt for a test case
  56 |     const isFinalAttempt =
  57 |       result.status === 'passed' ||
  58 |       result.status === 'skipped' ||
  59 |       result.retry === test.retries;
  60 |
  61 |     if (!isFinalAttempt) {
  62 |       return;
  63 |     }
  64 |
  65 |     metrics.total++;
  66 |
  67 |     if (result.status === 'passed') {
  68 |       if (result.retry > 0) {
  69 |         this.flaky++;
  70 |         metrics.flaky++;
  71 |       }
  72 |       this.passed++;
  73 |       metrics.passed++;
  74 |     } else if (result.status === 'skipped') {
  75 |       this.skipped++;
  76 |       metrics.skipped++;
  77 |     } else {
  78 |       this.failed++;
  79 |       metrics.failed++;
  80 |     }
  81 |   }
  82 |
  83 |   onEnd(result: FullResult): void {
  84 |     const durationSec = ((Date.now() - this.startTime) / 1000).toFixed(2);
  85 |     const executed = this.passed + this.failed;
  86 |     const passRate = executed > 0 ? ((this.passed / executed) * 100).toFixed(2) : '0.00';
  87 |
  88 |     console.log('\n================================================================================');
  89 |     console.log(' 📊 ENTERPRISE PLAYWRIGHT AUTOMATION - EXECUTIVE SUMMARY');
  90 |     console.log('================================================================================');
  91 |     console.log(` 🏁 Overall Status : ${result.status.toUpperCase()}`);
  92 |     console.log(` 🧪 Total Scenarios: ${this.passed + this.failed + this.skipped}`);
  93 |     console.log(` ✅ Passed         : ${this.passed}`);
  94 |     console.log(` ❌ Failed         : ${this.failed}`);
  95 |     console.log(` ⚠️  Flaky (Retried): ${this.flaky}`);
  96 |     console.log(` ⏭️  Skipped        : ${this.skipped}`);
  97 |     console.log(` 📈 Pass Rate      : ${passRate}%`);
  98 |     console.log(` ⏱️  Duration       : ${durationSec}s`);
  99 |     console.log('--------------------------------------------------------------------------------');
 100 |     console.log(' 📁 SUITE BREAKDOWN:');
 101 |
 102 |     const summaryRows: Array<{
 103 |       suite: string;
 104 |       total: number;
 105 |       passed: number;
 106 |       failed: number;
 107 |       skipped: number;
 108 |     }> = [];
 109 |
 110 |     for (const [suiteName, stats] of this.suiteBreakdown.entries()) {
 111 |       console.log(
 112 |         `   • ${suiteName.padEnd(58)} | Total: ${String(stats.total).padStart(3)} | ✅ ${String(stats.passed).padStart
(3)} | ❌ ${stats.failed}`
 113 |       );
 114 |       summaryRows.push({
 115 |         suite: suiteName,
 116 |         total: stats.total,
 117 |         passed: stats.passed,
 118 |         failed: stats.failed,
 119 |         skipped: stats.skipped,
 120 |       });
 121 |     }
 122 |     console.log('================================================================================\n');
 123 |
 124 |     // Persist JSON & Markdown Executive Report for CI/CD and Portfolio Artifacts
 125 |     const reportsDir = path.resolve(process.cwd(), 'reports');
```

```
126 |     if (!fs.existsSync(reportsDir)) {
127 |       fs.mkdirSync(reportsDir, { recursive: true });
128 |     }
129 |
130 |     const summaryPayload = {
131 |       timestamp: new Date().toISOString(),
132 |       status: result.status,
133 |       durationSeconds: Number(durationSec),
134 |       passRate: `${passRate}%`,
135 |       totals: {
136 |         total: this.passed + this.failed + this.skipped,
137 |         passed: this.passed,
138 |         failed: this.failed,
139 |         flaky: this.flaky,
140 |         skipped: this.skipped,
141 |       },
142 |       suites: summaryRows,
143 |     };
144 |
145 |     fs.writeFileSync(
146 |       path.join(reportsDir, 'executive-summary.json'),
147 |       JSON.stringify(summaryPayload, null, 2),
148 |       'utf-8'
149 |     );
150 |
151 |     const mdLines = [
152 |       '# 📊 Enterprise Playwright Automation — Executive Test Summary',
153 |       '',
154 |       `- **Status**: \`${result.status.toUpperCase()}\``,
155 |       `- **Total Test Cases**: **${summaryPayload.totals.total}**`,
156 |       `- **Passed**: **${this.passed}** ✅`,
157 |       `- **Failed**: **${this.failed}** ❌`,
158 |       `- **Pass Rate**: **${passRate}%**`,
159 |       `- **Execution Time**: **${durationSec}s**`,
160 |       '',
161 |       '## Suite-Level Breakdown',
162 |       '',
163 |       '| Test Suite | Total | Passed | Failed | Skipped |',
164 |       '| :--- | :---: | :---: | :---: | :---: |',
165 |       ...summaryRows.map(
166 |         (r) => `| \`${r.suite}\` | ${r.total} | ${r.passed} ✅ | ${r.failed} | ${r.skipped} |`
167 |       ),
168 |       '',
169 |     ];
170 |
171 |     fs.writeFileSync(
172 |       path.join(reportsDir, 'executive-summary.md'),
173 |       mdLines.join('\n'),
174 |       'utf-8'
175 |     );
176 |   }
177 | }
178 |
```

Checkpoint Verification Command After Creating Step 14: `npm run typecheck`

STAGE 6: HYBRID NETWORK VIRTUALIZATION & DI FIXTURES — STEP 15 OF 39

Step 15: Build `src/api/ToolshopApiInterceptor.ts`

Build Stateful In-Memory REST & GraphQL Microservice Engine (1136 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/api/ToolshopApiInterceptor.ts" -Force`
Bash / macOS / Linux	`touch src/api/ToolshopApiInterceptor.ts`
Prerequisite Files	src/data/toolshop.catalog.data.ts, src/data/toolshop.users.data.ts
Consumed By	src/fixtures/base.fixture.ts, tests/12-api-contract-suite/toolshop-api.spec.ts

How to Implement This File (Line-Range Breakdown):

- Lines 1–195: Define stateful interfaces (CartRecord, InvoiceRecord, FavoriteRecord, SupportMessageRecord) and seed default catalog, users, invoices, favorites, and messages.
- Lines 196–550: Implement queryProducts() (category/brand/eco/price/search filtering + 6-way sorting + 9-per-page pagination), cart CRUD, loginUser() (with 4-attempt HTTP 423 lockout), registerUser() (with duplicate email & underage DOB 422 checks), validatePayment(), createInvoice(), and createMessage().
- Lines 551–1137: Implement decodeValidJwt() and attachToPage(page) to intercept https://api.practicesoftwaretesting.com/** for REST and /graphql product comparison queries.

Exact Source Code to Put in `src/api/ToolshopApiInterceptor.ts`:

```
  1 | import { Page, Route } from '@playwright/test';
  2 | import {
  3 |   TOOLSHOP_PRODUCTS,
  4 |   TOOLSHOP_RENTALS,
  5 |   TOOLSHOP_BRANDS,
  6 |   TOOLSHOP_CATEGORIES_TREE,
  7 |   ToolshopProduct,
  8 | } from '@data/toolshop.catalog.data';
  9 | import { TOOLSHOP_USERS, generateToolshopJwt, ToolshopUserAccount } from '@data/toolshop.users.data';
 10 | import { MathHelper } from '@utils/math.helper';
 11 |
 12 | export interface CartItemRecord {
 13 |   id: string;
 14 |   quantity: number;
 15 |   discount_percentage: number | null;
 16 |   cart_id: string;
 17 |   product_id: string;
 18 |   product: ToolshopProduct;
 19 | }
 20 |
 21 | export interface CartRecord {
 22 |   id: string;
 23 |   additional_discount_percentage: number | null;
 24 |   lat: number | null;
 25 |   lng: number | null;
 26 |   cart_items: CartItemRecord[];
 27 | }
 28 |
 29 | export interface FavoriteRecord {
 30 |   id: string;
 31 |   product_id: string;
 32 |   user_id: string;
 33 |   product: ToolshopProduct;
 34 | }
 35 |
 36 | export interface InvoiceRecord {
 37 |   id: string;
 38 |   user_id: string;
 39 |   invoice_date: string;
 40 |   invoice_number: string;
 41 |   billing_street: string;
 42 |   billing_city: string;
 43 |   billing_state: string;
```

```
  billing_country: string;
  billing_postal_code: string;
  subtotal: number;
  total: number;
  payment_method: string;
  payment?: {
    id: string;
    payment_method: string;
    payment_details: Record<string, string>;
  };
  status: string;
  invoicelines: Array<{
    id: string;
    invoice_id: string;
    product_id: string;
    unit_price: number;
    quantity: number;
    product: ToolshopProduct;
  }>;
}

export interface MessageRecord {
  id: string;
  name: string;
  email: string;
  subject: string;
  message: string;
  status: 'NEW' | 'IN_PROGRESS' | 'RESOLVED';
  created_at: string;
  reply: Array<{ id: string; message: string; created_at: string }>;
}

/**
 * Deterministic Toolshop v5.0 REST API Engine & Playwright Route Interceptor.
 * Serves 100% schema-authentic responses to the real Angular 18 frontend at https://practicesoftwaretesting.com
 * and provides direct contract testing methods for Suite 12.
 */
export class ToolshopApiEngine {
  private products: ToolshopProduct[];
  private rentals: ToolshopProduct[];
  private carts = new Map<string, CartRecord>();
  private favorites: FavoriteRecord[] = [];
  private invoices: InvoiceRecord[] = [];
  private messages: MessageRecord[] = [];
  private users: Map<string, ToolshopUserAccount>;

  constructor() {
    this.products = TOOLSHOP_PRODUCTS.map((p) => ({ ...p }));
    this.rentals = TOOLSHOP_RENTALS.map((r) => ({ ...r }));
    this.users = new Map(
      Object.values(TOOLSHOP_USERS).map((u) => [u.email.toLowerCase(), { ...u, address: { ...u.address } }])
    );
    this.seedDefaults();
  }

  private seedDefaults(): void {
    this.favorites = [
      {
        id: '01JFAVDEFAULT0000000000001',
        product_id: this.products[0].id,
        user_id: TOOLSHOP_USERS.customer.id,
        product: this.products[0],
      },
      {
        id: '01JFAVDEFAULT0000000000002',
        product_id: this.products[9].id,
        user_id: TOOLSHOP_USERS.customer.id,
        product: this.products[9],
      },
    ];

    this.invoices = [
      {
        id: '01JINVDEFAULT0000000000001',
        user_id: TOOLSHOP_USERS.customer.id,
        invoice_date: '2026-10-01 14:22:10',
        invoice_number: 'INV-20260001',
```

```
        billing_street: 'Test street 98',
        billing_city: 'Vienna',
        billing_state: 'Vienna',
        billing_country: 'AT',
        billing_postal_code: '1010',
        subtotal: 80.69,
        total: 80.69,
        payment_method: 'credit-card',
        payment: {
          id: '01JPAYDEFAULT0000000000001',
          payment_method: 'credit-card',
          payment_details: {
            credit_card_number: '1111-2222-3333-4444',
            expiration_date: '12/2029',
            cvv: '123',
            card_holder_name: 'Jane Doe',
          },
        },
        status: 'COMPLETED',
        invoicelines: [
          {
            id: '01JINVLINE0000000000000001',
            invoice_id: '01JINVDEFAULT0000000000001',
            product_id: this.products[0].id,
            unit_price: this.products[0].price,
            quantity: 1,
            product: this.products[0],
          },
          {
            id: '01JINVLINE0000000000000002',
            invoice_id: '01JINVDEFAULT0000000000001',
            product_id: this.products[9].id,
            unit_price: this.products[9].price,
            quantity: 1,
            product: this.products[9],
          },
        ],
      },
      {
        id: '01JINVDEFAULT0000000000002',
        user_id: TOOLSHOP_USERS.customer.id,
        invoice_date: '2026-10-05 09:15:00',
        invoice_number: 'INV-20260002',
        billing_street: 'Test street 98',
        billing_city: 'Vienna',
        billing_state: 'Vienna',
        billing_country: 'AT',
        billing_postal_code: '1010',
        subtotal: 48.41,
        total: 48.41,
        payment_method: 'bank-transfer',
        payment: {
          id: '01JPAYDEFAULT0000000000002',
          payment_method: 'bank-transfer',
          payment_details: {
            bank_name: 'Global First Hardware Bank',
            account_name: 'Jane Doe',
            account_number: 'AT893704004405320130000',
          },
        },
        status: 'AWAITING_FULFILLMENT',
        invoicelines: [
          {
            id: '01JINVLINE0000000000000003',
            invoice_id: '01JINVDEFAULT0000000000002',
            product_id: this.products[2].id,
            unit_price: this.products[2].price,
            quantity: 1,
            product: this.products[2],
          },
        ],
      },
    ];

    this.messages = [
      {
        id: '01JMSGDEFAULT0000000000001',
```

```
198 |         name: 'Jane Doe',
199 |         email: 'customer@practicesoftwaretesting.com',
200 |         subject: 'warranty',
201 |         message:
202 |           'Requesting warranty coverage confirmation for our ForgeFlex Combination Pliers purchased under INV-2026000
1.',
203 |         status: 'NEW',
204 |         created_at: '2026-10-08 11:30:00',
205 |         reply: [],
206 |       },
207 |     ];
208 |   }
209 |
210 |   private getCorsHeaders(): Record<string, string> {
211 |     return {
212 |       'access-control-allow-origin': '*',
213 |       'access-control-allow-methods': 'GET, POST, PUT, PATCH, DELETE, QUERY, OPTIONS',
214 |       'access-control-allow-headers': 'Content-Type, Authorization, Accept, X-Requested-With',
215 |       'content-type': 'application/json; charset=utf-8',
216 |       'x-request-id': `toolshop-${Date.now()}-${Math.random().toString(36).slice(2, 7)}`,
217 |       'x-content-type-options': 'nosniff',
218 |     };
219 |   }
220 |
221 |   public queryProducts(params: {
222 |     page?: number | string;
223 |     q?: string;
224 |     sort?: string;
225 |     between?: string;
226 |     by_category?: string;
227 |     by_category_slug?: string;
228 |     by_brand?: string;
229 |     eco_friendly?: string | boolean;
230 |     is_rental?: string | boolean;
231 |   }): {
232 |     current_page: number;
233 |     data: ToolshopProduct[];
234 |     from: number | null;
235 |     last_page: number;
236 |     per_page: number;
237 |     to: number | null;
238 |     total: number;
239 |   } {
240 |     const isRental = params.is_rental === true || params.is_rental === 'true';
241 |     let list = isRental ? [...this.rentals] : [...this.products];
242 |
243 |     if (params.q) {
244 |       const q = params.q.trim().toLowerCase();
245 |       list = list.filter((p) => p.name.toLowerCase().includes(q));
246 |     }
247 |
248 |     if (params.eco_friendly === true || params.eco_friendly === 'true') {
249 |       list = list.filter((p) => p.is_eco_friendly);
250 |     }
251 |
252 |     if (params.by_category) {
253 |       const catIds = params.by_category.split(',').map((s) => s.trim());
254 |       list = list.filter(
255 |         (p) =>
256 |           catIds.includes(p.category.id) ||
257 |           (p.category.parent_id && catIds.includes(p.category.parent_id))
258 |       );
259 |     }
260 |
261 |     if (params.by_category_slug) {
262 |       const slug = params.by_category_slug.trim().toLowerCase();
263 |       const rootCat = TOOLSHOP_CATEGORIES_TREE.find((c) => c.slug === slug);
264 |       if (rootCat) {
265 |         const subIds = new Set((rootCat.sub_categories || []).map((sc) => sc.id));
266 |         list = list.filter(
267 |           (p) => p.category.parent_id === rootCat.id || subIds.has(p.category.id)
268 |         );
269 |       } else {
270 |         list = list.filter((p) => p.category.slug.toLowerCase() === slug);
271 |       }
272 |     }
273 |
```

```
    if (params.by_brand) {
      const brandIds = params.by_brand.split(',').map((s) => s.trim());
      list = list.filter((p) => brandIds.includes(p.brand.id));
    }

    if (params.between) {
      const parts = params.between.split(',');
      if (parts.length === 3 && parts[0] === 'price') {
        const min = parseFloat(parts[1]);
        const max = parseFloat(parts[2]);
        if (!Number.isNaN(min) && !Number.isNaN(max)) {
          list = list.filter((p) => p.price >= min && p.price <= max);
        }
      }
    }

    if (params.sort) {
      const [field, direction] = params.sort.split(',');
      const dir = direction === 'desc' ? -1 : 1;
      list.sort((a, b) => {
        if (field === 'name') {
          return a.name.localeCompare(b.name) * dir;
        }
        if (field === 'price') {
          return (a.price - b.price) * dir;
        }
        if (field === 'co2_rating') {
          return a.co2_rating.localeCompare(b.co2_rating) * dir;
        }
        return 0;
      });
    }

    const page = Math.max(1, Number(params.page || 1));
    const perPage = 9;
    const total = list.length;
    const lastPage = Math.max(1, Math.ceil(total / perPage));
    const startIdx = (page - 1) * perPage;
    const pagedData = list.slice(startIdx, startIdx + perPage);

    return {
      current_page: page,
      data: pagedData,
      from: pagedData.length > 0 ? startIdx + 1 : null,
      last_page: lastPage,
      per_page: perPage,
      to: pagedData.length > 0 ? startIdx + pagedData.length : null,
      total,
    };
  }

  public getProductById(id: string): ToolshopProduct | undefined {
    return (
      this.products.find((p) => p.id === id) ||
      this.rentals.find((r) => r.id === id)
    );
  }

  public getRelatedProducts(id: string): ToolshopProduct[] {
    const current = this.getProductById(id);
    if (!current) return this.products.slice(0, 3);
    const sameCategory = this.products.filter(
      (p) => p.id !== id && p.category.parent_id === current.category.parent_id
    );
    return sameCategory.slice(0, 3);
  }

  public createCart(): CartRecord {
    const id = `01JCART${Date.now()}${Math.random().toString(36).slice(2, 6).toUpperCase()}`;
    const cart: CartRecord = {
      id,
      additional_discount_percentage: null,
      lat: null,
      lng: null,
      cart_items: [],
    };
    this.carts.set(id, cart);
```

```
351 |     return cart;
352 |   }
353 |
354 |   public getCart(cartId: string): CartRecord | undefined {
355 |     return this.carts.get(cartId);
356 |   }
357 |
358 |   public addItemToCart(cartId: string, productId: string, quantity: number): CartRecord {
359 |     let cart = this.carts.get(cartId);
360 |     if (!cart) {
361 |       cart = {
362 |         id: cartId,
363 |         additional_discount_percentage: null,
364 |         lat: null,
365 |         lng: null,
366 |         cart_items: [],
367 |       };
368 |       this.carts.set(cartId, cart);
369 |     }
370 |
371 |     const product = this.getProductById(productId) ?? this.products[0];
372 |     const existing = cart.cart_items.find((ci) => ci.product_id === productId);
373 |     if (existing) {
374 |       existing.quantity += quantity;
375 |     } else {
376 |       cart.cart_items.push({
377 |         id: `01JCARTITEM${Date.now()}${cart.cart_items.length}`,
378 |         quantity,
379 |         discount_percentage: null,
380 |         cart_id: cartId,
381 |         product_id: product.id,
382 |         product,
383 |       });
384 |     }
385 |     return cart;
386 |   }
387 |
388 |   public updateCartItemQuantity(
389 |     cartId: string,
390 |     productId: string,
391 |     quantity: number
392 |   ): CartRecord | undefined {
393 |     const cart = this.carts.get(cartId);
394 |     if (!cart) return undefined;
395 |     const item = cart.cart_items.find((ci) => ci.product_id === productId);
396 |     if (item) {
397 |       item.quantity = Math.max(1, quantity);
398 |     }
399 |     return cart;
400 |   }
401 |
402 |   public removeCartItem(cartId: string, productId: string): boolean {
403 |     const cart = this.carts.get(cartId);
404 |     if (!cart) return false;
405 |     cart.cart_items = cart.cart_items.filter((ci) => ci.product_id !== productId);
406 |     return true;
407 |   }
408 |
409 |   public loginUser(
410 |     email?: string,
411 |     password?: string
412 |   ): { status: number; body: Record<string, unknown> } {
413 |     if (!email || !password) {
414 |       return {
415 |         status: 400,
416 |         body: { error: 'Email and password are required' },
417 |       };
418 |     }
419 |
420 |     const account = this.users.get(email.trim().toLowerCase());
421 |     if (!account) {
422 |       return {
423 |         status: 401,
424 |         body: { error: 'Invalid email or password' },
425 |       };
426 |     }
427 |
```

```
    if (!account.enabled) {
      return {
        status: 423,
        body: { error: 'Account locked, too many failed attempts. Please contact the administrator.' },
      };
    }

    if (account.password !== password) {
      account.failed_login_attempts += 1;
      if (account.failed_login_attempts >= 4) {
        account.enabled = false;
        return {
          status: 423,
          body: { error: 'Account locked, too many failed attempts. Please contact the administrator.' },
        };
      }
      return {
        status: 401,
        body: { error: 'Invalid email or password' },
      };
    }

    account.failed_login_attempts = 0;

    const token = generateToolshopJwt({
      sub: account.id,
      role: account.role,
      expiresInSeconds: 7200,
    });

    return {
      status: 200,
      body: {
        access_token: token,
        token_type: 'bearer',
        expires_in: 7200,
      },
    };
  }

  public registerUser(payload: Record<string, unknown>): {
    status: number;
    body: Record<string, unknown>;
  } {
    const email = String(payload.email || '').trim().toLowerCase();
    if (!email || !payload.first_name || !payload.last_name || !payload.password) {
      return {
        status: 422,
        body: { error: 'Missing required registration fields' },
      };
    }

    if (this.users.has(email)) {
      return {
        status: 422,
        body: {
          email: ['A customer with this email address already exists.'],
        },
      };
    }

    if (payload.dob && String(payload.dob) > '2009-01-01') {
      return {
        status: 422,
        body: {
          dob: ['Customer must be at least 18 years old.'],
        },
      };
    }

    const rawAddress = (payload.address as Record<string, string>) || {};
    const newAccount: ToolshopUserAccount = {
      id: `01JUSERREG${Date.now()}`,
      first_name: String(payload.first_name),
      last_name: String(payload.last_name),
      email,
      password: String(payload.password),
```

```
505 |       role: 'user',
506 |       phone: String(payload.phone || '0123456789'),
507 |       dob: String(payload.dob || '1990-01-01'),
508 |       enabled: true,
509 |       failed_login_attempts: 0,
510 |       address: {
511 |         street: rawAddress.street || String(payload.street || '100 Main St'),
512 |         city: rawAddress.city || String(payload.city || 'San Francisco'),
513 |         state: rawAddress.state || String(payload.state || 'CA'),
514 |         country: rawAddress.country || String(payload.country || 'US'),
515 |         postal_code: rawAddress.postal_code || String(payload.postal_code || '94105'),
516 |       },
517 |     };
518 |
519 |     this.users.set(email, newAccount);
520 |     return {
521 |       status: 201,
522 |       body: {
523 |         id: newAccount.id,
524 |         first_name: newAccount.first_name,
525 |         last_name: newAccount.last_name,
526 |         email: newAccount.email,
527 |         phone: newAccount.phone,
528 |         dob: newAccount.dob,
529 |         address: newAccount.address,
530 |       },
531 |     };
532 |   }
533 |
534 |   public validatePayment(payload: {
535 |     payment_method?: string;
536 |     payment_details?: Record<string, string>;
537 |   }): { status: number; body: { message: string } } {
538 |     if (!payload.payment_method) {
539 |       return {
540 |         status: 400,
541 |         body: { message: 'Payment method is required' },
542 |       };
543 |     }
544 |     return {
545 |       status: 200,
546 |       body: { message: 'Payment was successful' },
547 |     };
548 |   }
549 |
550 |   public createInvoice(payload: Record<string, unknown>): InvoiceRecord {
551 |     const invNum = `INV-2026${String(this.invoices.length + 1001)}`;
552 |     const method = String(payload.payment_method || 'credit-card');
553 |     const newInv: InvoiceRecord = {
554 |       id: `01JINV${Date.now()}`,
555 |       user_id: String(payload.user_id || TOOLSHOP_USERS.customer.id),
556 |       invoice_date: new Date().toISOString().replace('T', ' ').slice(0, 19),
557 |       invoice_number: invNum,
558 |       billing_street: String(payload.billing_street || 'Test street 98'),
559 |       billing_city: String(payload.billing_city || 'Vienna'),
560 |       billing_state: String(payload.billing_state || 'Vienna'),
561 |       billing_country: String(payload.billing_country || 'AT'),
562 |       billing_postal_code: String(payload.billing_postal_code || '1010'),
563 |       subtotal: MathHelper.roundToTwoDecimals(Number(payload.total || 25.0)),
564 |       total: MathHelper.roundToTwoDecimals(Number(payload.total || 25.0)),
565 |       payment_method: method,
566 |       payment: {
567 |         id: `01JPAY${Date.now()}`,
568 |         payment_method: method,
569 |         payment_details: (payload.payment_details as Record<string, string>) || {},
570 |       },
571 |       status: 'COMPLETED',
572 |       invoicelines: [
573 |         {
574 |           id: `01JINVLINE${Date.now()}`,
575 |           invoice_id: invNum,
576 |           product_id: this.products[0].id,
577 |           unit_price: this.products[0].price,
578 |           quantity: 1,
579 |           product: this.products[0],
580 |         },
581 |       ],
```

```
582 |     };
583 |     this.invoices.unshift(newInv);
584 |     return newInv;
585 |   }
586 |
587 |   public createMessage(payload: Record<string, unknown>): {
588 |     status: number;
589 |     body: MessageRecord | { error: string };
590 |   } {
591 |     const subject = String(payload.subject || '');
592 |     const message = String(payload.message || '');
593 |     if (!subject || message.length < 50) {
594 |       return {
595 |         status: 422,
596 |         body: { error: 'Subject is required and message must be at least 50 characters' },
597 |       };
598 |     }
599 |     const record: MessageRecord = {
600 |       id: `01JMSG${Date.now()}`,
601 |       name: `${payload.first_name || 'Customer'} ${payload.last_name || ''}`.trim(),
602 |       email: String(payload.email || 'customer@practicesoftwaretesting.com'),
603 |       subject,
604 |       message,
605 |       status: 'NEW',
606 |       created_at: new Date().toISOString().replace('T', ' ').slice(0, 19),
607 |       reply: [],
608 |     };
609 |     this.messages.unshift(record);
610 |     return {
611 |       status: 200,
612 |       body: record,
613 |     };
614 |   }
615 |
616 |   public getFavorites(): FavoriteRecord[] {
617 |     return this.favorites;
618 |   }
619 |
620 |   public getInvoices(): InvoiceRecord[] {
621 |     return this.invoices;
622 |   }
623 |
624 |   public getMessages(): MessageRecord[] {
625 |     return this.messages;
626 |   }
627 |
628 |   private decodeValidJwt(authHeader?: string): { sub: string; role: string; exp: number } | null {
629 |     if (!authHeader || !authHeader.toLowerCase().startsWith('bearer ')) {
630 |       return null;
631 |     }
632 |     const token = authHeader.slice(7).trim();
633 |     const parts = token.split('.');
634 |     if (parts.length !== 3) {
635 |       return null;
636 |     }
637 |     try {
638 |       const decoded = JSON.parse(
639 |         Buffer.from(parts[1], 'base64url').toString('utf-8')
640 |       ) as { sub?: string; role?: string; exp?: number };
641 |       if (!decoded || !decoded.sub) {
642 |         return null;
643 |       }
644 |       const nowSec = Math.floor(Date.now() / 1000);
645 |       if (typeof decoded.exp === 'number' && decoded.exp < nowSec) {
646 |         return null;
647 |       }
648 |       return {
649 |         sub: decoded.sub,
650 |         role: decoded.role || 'user',
651 |         exp: decoded.exp || nowSec + 7200,
652 |       };
653 |     } catch {
654 |       return null;
655 |     }
656 |   }
657 |
658 |   /**
```

```
659 |    * Attaches Playwright route interceptor for all https://api.practicesoftwaretesting.com/** requests
660 |    */
661 |   public async attachToPage(page: Page): Promise<void> {
662 |     await page.route('https://api.practicesoftwaretesting.com/**', async (route: Route) => {
663 |       const req = route.request();
664 |       const method = req.method().toUpperCase();
665 |       const url = new URL(req.url());
666 |       const pathname = url.pathname.replace(/\/+$/, '') || '/';
667 |       const headers = this.getCorsHeaders();
668 |
669 |       if (method === 'OPTIONS') {
670 |         await route.fulfill({ status: 204, headers });
671 |         return;
672 |       }
673 |
674 |       let bodyJson: Record<string, unknown> = {};
675 |       try {
676 |         const rawPost = req.postData();
677 |         if (rawPost && rawPost.trim().startsWith('{')) {
678 |           bodyJson = JSON.parse(rawPost) as Record<string, unknown>;
679 |         }
680 |       } catch {
681 |         // Ignore non-JSON body (e.g. multipart FormData)
682 |       }
683 |
684 |       const queryParams: Record<string, string> = {};
685 |       url.searchParams.forEach((v, k) => {
686 |         queryParams[k] = v;
687 |       });
688 |       const mergedParams = { ...queryParams, ...bodyJson };
689 |
690 |       // 0. GraphQL endpoint (used by /comparison table)
691 |       if (pathname === '/graphql' && method === 'POST') {
692 |         const queryStr = String(bodyJson.query || '');
693 |         const data: Record<string, unknown> = {};
694 |         const regex = /(\w+):\s*product\(\s*id:\s*"([^"]+)"\s*\)/g;
695 |         let match = regex.exec(queryStr);
696 |         while (match !== null) {
697 |           const alias = match[1];
698 |           const prodId = match[2];
699 |           const prod = this.getProductById(prodId);
700 |           if (prod) {
701 |             data[alias] = prod;
702 |           }
703 |           match = regex.exec(queryStr);
704 |         }
705 |         await route.fulfill({
706 |           status: 200,
707 |           headers,
708 |           body: JSON.stringify({ data }),
709 |         });
710 |         return;
711 |       }
712 |
713 |       // 1. Brands
714 |       if (pathname === '/brands' && method === 'GET') {
715 |         await route.fulfill({
716 |           status: 200,
717 |           headers,
718 |           body: JSON.stringify(TOOLSHOP_BRANDS),
719 |         });
720 |         return;
721 |       }
722 |
723 |       // 2. Categories
724 |       if ((pathname === '/categories/tree' || pathname === '/categories') && method === 'GET') {
725 |         await route.fulfill({
726 |           status: 200,
727 |           headers,
728 |           body: JSON.stringify(TOOLSHOP_CATEGORIES_TREE),
729 |         });
730 |         return;
731 |       }
732 |
733 |       // 3. Product Specs Names
734 |       if (pathname === '/product-specs/names' && method === 'GET') {
735 |         await route.fulfill({
```

```
            status: 200,
            headers,
            body: JSON.stringify([
              { name: 'Handle Material', values: ['Bi-component', 'Recycled TPR', 'FSC Certified Beechwood'] },
              { name: 'Material', values: ['Chrome Vanadium Steel', 'Carbon Steel', 'Drop-Forged Steel'] },
            ]),
          });
          return;
        }

        // 4. Products List & Search (QUERY or GET)
        if (
          (pathname === '/products' || pathname === '/products/search') &&
          (method === 'QUERY' || method === 'GET')
        ) {
          const result = this.queryProducts(mergedParams);
          await route.fulfill({
            status: 200,
            headers,
            body: JSON.stringify(result),
          });
          return;
        }

        // 5. Related Products
        const relatedMatch = pathname.match(/^\/products\/([^/]+)\/related$/);
        if (relatedMatch && method === 'GET') {
          const related = this.getRelatedProducts(relatedMatch[1]);
          await route.fulfill({
            status: 200,
            headers,
            body: JSON.stringify(related),
          });
          return;
        }

        // 6. Single Product by ID
        const productMatch = pathname.match(/^\/products\/([^/]+)$/);
        if (productMatch && method === 'GET') {
          const prod = this.getProductById(productMatch[1]);
          if (!prod) {
            await route.fulfill({
              status: 404,
              headers,
              body: JSON.stringify({ message: 'Requested item not found' }),
            });
            return;
          }
          await route.fulfill({
            status: 200,
            headers,
            body: JSON.stringify(prod),
          });
          return;
        }

        // 7. Carts endpoints
        if (pathname === '/carts' && method === 'POST') {
          const cart = this.createCart();
          await route.fulfill({
            status: 201,
            headers,
            body: JSON.stringify({ id: cart.id }),
          });
          return;
        }

        const cartQtyMatch = pathname.match(/^\/carts\/([^/]+)\/product\/quantity$/);
        if (cartQtyMatch && method === 'PUT') {
          this.updateCartItemQuantity(
            cartQtyMatch[1],
            String(bodyJson.product_id),
            Number(bodyJson.quantity || 1)
          );
          await route.fulfill({
            status: 200,
            headers,
```

```
            body: JSON.stringify({ result: 'item added or updated' }),
          });
          return;
        }

        const cartDeleteProductMatch = pathname.match(/^\/carts\/([^/]+)\/product\/([^/]+)$/);
        if (cartDeleteProductMatch && method === 'DELETE') {
          this.removeCartItem(cartDeleteProductMatch[1], cartDeleteProductMatch[2]);
          await route.fulfill({ status: 204, headers });
          return;
        }

        const cartIdMatch = pathname.match(/^\/carts\/([^/]+)$/);
        if (cartIdMatch) {
          const cartId = cartIdMatch[1];
          if (method === 'POST') {
            this.addItemToCart(
              cartId,
              String(bodyJson.product_id),
              Number(bodyJson.quantity || 1)
            );
            await route.fulfill({
              status: 200,
              headers,
              body: JSON.stringify({ result: 'item added or updated' }),
            });
            return;
          }
          if (method === 'GET') {
            const cart = this.getCart(cartId) ?? {
              id: cartId,
              additional_discount_percentage: null,
              lat: null,
              lng: null,
              cart_items: [],
            };
            await route.fulfill({
              status: 200,
              headers,
              body: JSON.stringify(cart),
            });
            return;
          }
          if (method === 'DELETE') {
            this.carts.delete(cartId);
            await route.fulfill({ status: 204, headers });
            return;
          }
        }

        // 8. Users & Authentication
        if (pathname === '/users/login' && method === 'POST') {
          const res = this.loginUser(
            String(bodyJson.email || ''),
            String(bodyJson.password || '')
          );
          await route.fulfill({
            status: res.status,
            headers,
            body: JSON.stringify(res.body),
          });
          return;
        }

        if (pathname === '/users/register' && method === 'POST') {
          const res = this.registerUser(bodyJson);
          await route.fulfill({
            status: res.status,
            headers,
            body: JSON.stringify(res.body),
          });
          return;
        }

        if (pathname === '/users/forgot-password' && method === 'POST') {
          await route.fulfill({
            status: 200,
```

```
890 |           headers,
891 |           body: JSON.stringify({ result: 'success' }),
892 |         });
893 |         return;
894 |       }
895 |
896 |       if (pathname === '/users/refresh' && method === 'GET') {
897 |         const validClaims = this.decodeValidJwt(req.headers()['authorization']);
898 |         if (!validClaims) {
899 |           await route.fulfill({
900 |             status: 401,
901 |             headers,
902 |             body: JSON.stringify({ message: 'Unauthenticated.' }),
903 |           });
904 |           return;
905 |         }
906 |         const token = generateToolshopJwt({
907 |           sub: validClaims.sub,
908 |           role: validClaims.role === 'admin' ? 'admin' : 'user',
909 |           expiresInSeconds: 7200,
910 |         });
911 |         await route.fulfill({
912 |           status: 200,
913 |           headers,
914 |           body: JSON.stringify({ access_token: token }),
915 |         });
916 |         return;
917 |       }
918 |
919 |       if (pathname === '/users/me' && method === 'GET') {
920 |         const validClaims = this.decodeValidJwt(req.headers()['authorization']);
921 |         if (!validClaims) {
922 |           await route.fulfill({
923 |             status: 401,
924 |             headers,
925 |             body: JSON.stringify({ message: 'Unauthenticated.' }),
926 |           });
927 |           return;
928 |         }
929 |         const foundById = Array.from(this.users.values()).find((u) => u.id === validClaims.sub);
930 |         const targetUser = foundById ?? this.users.get(TOOLSHOP_USERS.customer.email)!;
931 |         await route.fulfill({
932 |           status: 200,
933 |           headers,
934 |           body: JSON.stringify({
935 |             id: targetUser.id,
936 |             first_name: targetUser.first_name,
937 |             last_name: targetUser.last_name,
938 |             email: targetUser.email,
939 |             phone: targetUser.phone,
940 |             dob: targetUser.dob,
941 |             address: {
942 |               ...targetUser.address,
943 |               house_number: '98',
944 |             },
945 |           }),
946 |         });
947 |         return;
948 |       }
949 |
950 |       if (pathname.startsWith('/users/') && method === 'PUT') {
951 |         const customer = this.users.get(TOOLSHOP_USERS.customer.email)!;
952 |         if (bodyJson.first_name) customer.first_name = String(bodyJson.first_name);
953 |         if (bodyJson.last_name) customer.last_name = String(bodyJson.last_name);
954 |         if (bodyJson.phone) customer.phone = String(bodyJson.phone);
955 |         await route.fulfill({
956 |           status: 200,
957 |           headers,
958 |           body: JSON.stringify({ success: true }),
959 |         });
960 |         return;
961 |       }
962 |
963 |       if (pathname === '/users/change-password' && method === 'POST') {
964 |         await route.fulfill({
965 |           status: 200,
966 |           headers,
```

```
            body: JSON.stringify({ success: true }),
          });
          return;
        }

        // 9. Payment Check & Invoices
        if (pathname === '/payment/check' && method === 'POST') {
          const res = this.validatePayment(bodyJson);
          if (res.status === 200) {
            this.createInvoice(bodyJson);
          }
          await route.fulfill({
            status: res.status,
            headers,
            body: JSON.stringify(res.body),
          });
          return;
        }

        if ((pathname === '/invoices' || pathname === '/invoices/guest') && method === 'POST') {
          const inv = this.createInvoice(bodyJson);
          await route.fulfill({
            status: 201,
            headers,
            body: JSON.stringify(inv),
          });
          return;
        }

        if (pathname === '/invoices' && method === 'GET') {
          await route.fulfill({
            status: 200,
            headers,
            body: JSON.stringify({
              current_page: 1,
              data: this.invoices,
              from: 1,
              last_page: 1,
              per_page: 15,
              to: this.invoices.length,
              total: this.invoices.length,
            }),
          });
          return;
        }

        const invoiceIdMatch = pathname.match(/^\/invoices\/([^/]+)$/);
        if (invoiceIdMatch && method === 'GET') {
          const inv = this.invoices.find((i) => i.id === invoiceIdMatch[1]) ?? this.invoices[0];
          await route.fulfill({
            status: 200,
            headers,
            body: JSON.stringify(inv),
          });
          return;
        }

        // 10. Favorites
        if (pathname === '/favorites' && method === 'GET') {
          await route.fulfill({
            status: 200,
            headers,
            body: JSON.stringify(this.favorites),
          });
          return;
        }

        if (pathname === '/favorites' && method === 'POST') {
          const prodId = String(bodyJson.product_id || this.products[0].id);
          const prod = this.getProductById(prodId) ?? this.products[0];
          const exists = this.favorites.find((f) => f.product_id === prodId);
          if (exists) {
            await route.fulfill({
              status: 422,
              headers,
              body: JSON.stringify({ message: 'Duplicate Entry' }),
            });
```

```
1044 |           return;
1045 |         }
1046 |         const fav: FavoriteRecord = {
1047 |           id: `01JFAV${Date.now()}`,
1048 |           product_id: prod.id,
1049 |           user_id: TOOLSHOP_USERS.customer.id,
1050 |           product: prod,
1051 |         };
1052 |         this.favorites.push(fav);
1053 |         await route.fulfill({
1054 |           status: 201,
1055 |           headers,
1056 |           body: JSON.stringify(fav),
1057 |         });
1058 |         return;
1059 |       }
1060 |
1061 |       const favDeleteMatch = pathname.match(/^\/favorites\/([^/]+)$/);
1062 |       if (favDeleteMatch && method === 'DELETE') {
1063 |         this.favorites = this.favorites.filter((f) => f.id !== favDeleteMatch[1]);
1064 |         await route.fulfill({ status: 204, headers });
1065 |         return;
1066 |       }
1067 |
1068 |       // 11. Contact Support Messages
1069 |       if (pathname === '/messages' && method === 'POST') {
1070 |         const res = this.createMessage(bodyJson);
1071 |         await route.fulfill({
1072 |           status: res.status,
1073 |           headers,
1074 |           body: JSON.stringify(res.body),
1075 |         });
1076 |         return;
1077 |       }
1078 |
1079 |       if (pathname.match(/^\/messages\/[^/]+\/attach-file$/) && method === 'POST') {
1080 |         await route.fulfill({
1081 |           status: 200,
1082 |           headers,
1083 |           body: JSON.stringify({ success: true }),
1084 |         });
1085 |         return;
1086 |       }
1087 |
1088 |       if (pathname === '/messages' && method === 'GET') {
1089 |         await route.fulfill({
1090 |           status: 200,
1091 |           headers,
1092 |           body: JSON.stringify({
1093 |             current_page: 1,
1094 |             data: this.messages,
1095 |             from: 1,
1096 |             last_page: 1,
1097 |             per_page: 15,
1098 |             to: this.messages.length,
1099 |             total: this.messages.length,
1100 |           }),
1101 |         });
1102 |         return;
1103 |       }
1104 |
1105 |       const msgIdMatch = pathname.match(/^\/messages\/([^/]+)$/);
1106 |       if (msgIdMatch && method === 'GET') {
1107 |         const msg = this.messages.find((m) => m.id === msgIdMatch[1]) ?? this.messages[0];
1108 |         await route.fulfill({
1109 |           status: 200,
1110 |           headers,
1111 |           body: JSON.stringify(msg),
1112 |         });
1113 |         return;
1114 |       }
1115 |
1116 |       if (pathname.includes('postcode')) {
1117 |         await route.fulfill({
1118 |           status: 404,
1119 |           headers,
1120 |           body: JSON.stringify({ message: 'Postcode lookup disabled in deterministic test mode' }),
```

```
1121 |         });
1122 |         return;
1123 |       }
1124 |
1125 |       // Default fallback for any other API endpoint (e.g. TOTP setup, live sales-stream)
1126 |       await route.fulfill({
1127 |         status: 200,
1128 |         headers,
1129 |         body: JSON.stringify({
1130 |           status: 'ok',
1131 |           data: [],
1132 |         }),
1133 |       });
1134 |     });
1135 |   }
1136 | }
```

Checkpoint Verification Command After Creating Step 15: `npm run typecheck`

STAGE 6: HYBRID NETWORK VIRTUALIZATION & DI FIXTURES — STEP 16 OF 39

Step 16: Build `src/fixtures/base.fixture.ts`

Build Playwright Dependency Injection Container & 2-Tier Asset Cache (262 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/fixtures/base.fixture.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/fixtures && touch src/fixtures/base.fixture.ts`
Prerequisite Files	All Page Objects (src/pages/*) and ToolshopApiInterceptor.ts
Consumed By	All 12 test spec files (tests/01..12)

How to Implement This File (Line-Range Breakdown):

- Lines 1–88: Import all Page Objects, declare EnterpriseFixtures interface, and implement SHA-1 disk cache helpers (playwright/.cache/toolshop-v5-assets).
- Lines 89–215: Override the page fixture to block external ad/analytics domains, serve cached Angular 20 JS/CSS/HTML bundles, enforce AdminAuthGuard on /chunk-SAZHGXZ6.js, and attach ToolshopApiInterceptor.
- Lines 216–263: Register fixtures for navbar, catalogPage, productDetailsPage, rentalsPage, authPage, checkoutWizardPage, contactPage, accountPortalPage, and apiEngine.

Exact Source Code to Put in `src/fixtures/base.fixture.ts`:

```
  1 | import { test as base, expect, Page } from '@playwright/test';
  2 | import * as fs from 'fs';
  3 | import * as path from 'path';
  4 | import * as crypto from 'crypto';
  5 | import { CatalogPage } from '@pages/CatalogPage';
  6 | import { ProductDetailsPage } from '@pages/ProductDetailsPage';
  7 | import { RentalsPage } from '@pages/RentalsPage';
  8 | import { AuthPage } from '@pages/AuthPage';
  9 | import { CheckoutWizardPage } from '@pages/CheckoutWizardPage';
 10 | import { ContactPage } from '@pages/ContactPage';
 11 | import { AccountPortalPage } from '@pages/AccountPortalPage';
 12 | import { NavbarComponent } from '@pages/components/NavbarComponent';
 13 | import { ToolshopApiEngine } from '@api/ToolshopApiInterceptor';
 14 | import { TestLogger } from '@utils/logger';
 15 |
 16 | interface CachedStaticAsset {
 17 |   status: number;
 18 |   contentType: string;
 19 |   bodyBase64: string;
 20 | }
 21 |
 22 | const memoryAssetCache = new Map<string, CachedStaticAsset>();
 23 | const diskCacheDir = path.resolve(process.cwd(), 'playwright', '.cache', 'toolshop-v5-assets');
 24 |
 25 | function getDiskCachePath(urlKey: string): string {
 26 |   const hash = crypto.createHash('sha1').update(urlKey).digest('hex');
 27 |   return path.join(diskCacheDir, `${hash}.json`);
 28 | }
 29 |
 30 | function readFromDiskCache(urlKey: string): CachedStaticAsset | null {
 31 |   try {
 32 |     const filePath = getDiskCachePath(urlKey);
 33 |     if (!fs.existsSync(filePath)) return null;
 34 |     const raw = fs.readFileSync(filePath, 'utf-8');
 35 |     return JSON.parse(raw) as CachedStaticAsset;
 36 |   } catch {
 37 |     return null;
 38 |   }
 39 | }
 40 |
 41 | function writeToDiskCache(urlKey: string, asset: CachedStaticAsset): void {
 42 |   try {
 43 |     fs.mkdirSync(diskCacheDir, { recursive: true });
 44 |     const filePath = getDiskCachePath(urlKey);
 45 |     const tempPath = `${filePath}.${process.pid}.tmp`;
```

```
  46 |     fs.writeFileSync(tempPath, JSON.stringify(asset), 'utf-8');
  47 |     fs.renameSync(tempPath, filePath);
  48 |   } catch {
  49 |     // Ignore concurrent worker cache write races
  50 |   }
  51 | }
  52 |
  53 | /**
  54 |  * 1x1 transparent PNG for fast product card image fulfillment if local asset is missing
  55 |  */
  56 | const TRANSPARENT_PNG_BUFFER = Buffer.from(
  57 |   'iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAQAAAC1HAwCAAAAC0lEQVR42mNkYAAAAAYAAjCB0C8AAAAASUVORK5CYII=',
  58 |   'base64'
  59 | );
  60 |
  61 | export interface EnterpriseToolshopFixtures {
  62 |   catalogPage: CatalogPage;
  63 |   productDetailsPage: ProductDetailsPage;
  64 |   rentalsPage: RentalsPage;
  65 |   authPage: AuthPage;
  66 |   checkoutWizardPage: CheckoutWizardPage;
  67 |   contactPage: ContactPage;
  68 |   accountPortalPage: AccountPortalPage;
  69 |   navbar: NavbarComponent;
  70 |   apiEngine: ToolshopApiEngine;
  71 |   testLogger: TestLogger;
  72 | }
  73 |
  74 | export const test = base.extend<EnterpriseToolshopFixtures>({
  75 |   apiEngine: async ({}, use) => {
  76 |     const engine = new ToolshopApiEngine();
  77 |     await use(engine);
  78 |   },
  79 |
  80 |   page: async ({ page, apiEngine }, use) => {
  81 |     // 1. Neutralize 3rd-party telemetry, Cloudflare beacon, & external font CDNs to prevent external latency
  82 |     await page.route(
  83 |       (url) =>
  84 |         url.hostname.includes('googletagmanager.com') ||
  85 |         url.hostname.includes('google-analytics.com') ||
  86 |         url.hostname.includes('cloudflareinsights.com') ||
  87 |         url.hostname.includes('fonts.googleapis.com') ||
  88 |         url.hostname.includes('fonts.gstatic.com') ||
  89 |         url.pathname.startsWith('/cdn-cgi/'),
  90 |       async (route) => {
  91 |         await route.fulfill({
  92 |           status: 204,
  93 |           body: '',
  94 |         });
  95 |       }
  96 |     );
  97 |
  98 |     // 2. Two-tier memory + disk cache for Toolshop v5 Angular static assets
  99 |     await page.route(
 100 |       (url) => url.hostname === 'practicesoftwaretesting.com' && !url.pathname.startsWith('/cdn-cgi/'),
 101 |       async (route, request) => {
 102 |         const reqUrl = new URL(request.url());
 103 |         const pathname = reqUrl.pathname;
 104 |
 105 |         // Instant local fulfillment for product/rental images so cards render immediately
 106 |         if (pathname.startsWith('/assets/img/') || pathname.endsWith('.avif') || pathname.endsWith('.jpg') || pathname.e
ndsWith('.png')) {
 107 |           await route.fulfill({
 108 |             status: 200,
 109 |             contentType: 'image/png',
 110 |             body: TRANSPARENT_PNG_BUFFER,
 111 |           });
 112 |           return;
 113 |         }
 114 |
 115 |         if (pathname === '/chunk-SAZHGXZ6.js') {
 116 |           const adminGuardModule = `
 117 |             const token = window.localStorage.getItem('auth-token');
 118 |             let isAdmin = false;
 119 |             if (token) {
 120 |               try {
 121 |                 const payload = JSON.parse(atob(token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/')));
```

```
                isAdmin = payload.role === 'admin' && (!payload.exp || payload.exp * 1000 > Date.now());
              } catch {}
            }
            if (!isAdmin) {
              window.location.href = '/auth/login';
            }
            export class AdminModule {}
          `;
          await route.fulfill({
            status: 200,
            contentType: 'application/javascript; charset=utf-8',
            body: adminGuardModule,
          });
          return;
        }

        // SPA routes (e.g. /auth/login, /product/123, /checkout, /rentals, /contact, /account) all use index.html
        const isStaticAsset =
          pathname.endsWith('.js') ||
          pathname.endsWith('.css') ||
          pathname.endsWith('.json') ||
          pathname.endsWith('.ico') ||
          pathname.endsWith('.woff') ||
          pathname.endsWith('.woff2') ||
          pathname.endsWith('.ttf') ||
          pathname.endsWith('.svg');

        const cacheKey = isStaticAsset
          ? `${reqUrl.origin}${pathname}`
          : `${reqUrl.origin}/index.html`;

        let cached = memoryAssetCache.get(cacheKey);
        if (!cached) {
          const fromDisk = readFromDiskCache(cacheKey);
          if (fromDisk) {
            cached = fromDisk;
            memoryAssetCache.set(cacheKey, cached);
          }
        }

        try {
          if (cached) {
            let bodyBuf = Buffer.from(cached.bodyBase64, 'base64');
            if (cached.contentType.includes('text/html')) {
              const cleanHtml = bodyBuf
                .toString('utf-8')
                .replace(/<script>\(function\(\)\{function c\(\)[\s\S]*?<\/script>/g, '')
                .replace(/<script[^>]*cloudflareinsights\.com[\s\S]*?<\/script>/g, '');
              bodyBuf = Buffer.from(cleanHtml, 'utf-8');
            }
            await route.fulfill({
              status: cached.status,
              contentType: cached.contentType,
              body: bodyBuf,
            });
            return;
          }

          // Fetch from upstream once and populate both memory and disk caches
          const fetchUrl = isStaticAsset ? request.url() : `${reqUrl.origin}/`;
          const response = await route.fetch({ url: fetchUrl });
          let bodyBuffer = await response.body();
          const contentType = response.headers()['content-type'] ?? 'text/html; charset=utf-8';
          const status = response.status();

          if (contentType.includes('text/html')) {
            const cleanHtml = bodyBuffer
              .toString('utf-8')
              .replace(/<script>\(function\(\)\{function c\(\)[\s\S]*?<\/script>/g, '')
              .replace(/<script[^>]*cloudflareinsights\.com[\s\S]*?<\/script>/g, '');
            bodyBuffer = Buffer.from(cleanHtml, 'utf-8');
          }

          if (status === 200 && bodyBuffer.length > 0) {
            const newCached: CachedStaticAsset = {
              status,
              contentType,
```

```
199 |               bodyBase64: bodyBuffer.toString('base64'),
200 |             };
201 |             memoryAssetCache.set(cacheKey, newCached);
202 |             writeToDiskCache(cacheKey, newCached);
203 |           }
204 |
205 |           await route.fulfill({
206 |             status,
207 |             contentType,
208 |             body: bodyBuffer,
209 |           });
210 |         } catch {
211 |           // Ignore in-flight route errors if the test ended during lazy chunk fetch
212 |         }
213 |       }
214 |     );
215 |
216 |     // 3. Attach deterministic Toolshop v5 REST API Engine for https://api.practicesoftwaretesting.com/**
217 |     await apiEngine.attachToPage(page);
218 |
219 |     await use(page);
220 |     await page.unrouteAll({ behavior: 'ignoreErrors' });
221 |   },
222 |
223 |   testLogger: async ({}, use, testInfo) => {
224 |     const logger = new TestLogger(testInfo.title);
225 |     await use(logger);
226 |   },
227 |
228 |   navbar: async ({ page }, use) => {
229 |     await use(new NavbarComponent(page));
230 |   },
231 |
232 |   catalogPage: async ({ page }, use) => {
233 |     await use(new CatalogPage(page));
234 |   },
235 |
236 |   productDetailsPage: async ({ page }, use) => {
237 |     await use(new ProductDetailsPage(page));
238 |   },
239 |
240 |   rentalsPage: async ({ page }, use) => {
241 |     await use(new RentalsPage(page));
242 |   },
243 |
244 |   authPage: async ({ page }, use) => {
245 |     await use(new AuthPage(page));
246 |   },
247 |
248 |   checkoutWizardPage: async ({ page }, use) => {
249 |     await use(new CheckoutWizardPage(page));
250 |   },
251 |
252 |   contactPage: async ({ page }, use) => {
253 |     await use(new ContactPage(page));
254 |   },
255 |
256 |   accountPortalPage: async ({ page }, use) => {
257 |     await use(new AccountPortalPage(page));
258 |   },
259 | });
260 |
261 | export { expect };
262 | export type { Page };
```

Checkpoint Verification Command After Creating Step 16: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 17 OF 39

Step 17: Build `src/pages/BasePage.ts`

Create Abstract BasePage Class (59 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/BasePage.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/pages && touch src/pages/BasePage.ts`
Prerequisite Files	@playwright/test, src/config/env.config.ts, src/utils/logger.ts
Consumed By	All Page Objects and NavbarComponent

How to Implement This File (Line-Range Breakdown):

- Lines 1–60: Implement shared navigation (navigateTo, reloadPage), DOM text extraction (getTrimmedText), and Web Storage helpers (setLocalStorageItem, getLocalStorageItem, getSessionStorageItem, clearBrowserStorage).

Exact Source Code to Put in `src/pages/BasePage.ts` :

```
 1 | import { Page, Locator, expect } from '@playwright/test';
 2 | import { EnvConfig } from '@config/env.config';
 3 | import { TestLogger } from '@utils/logger';
 4 |
 5 | export abstract class BasePage {
 6 |   protected readonly page: Page;
 7 |   protected readonly logger: TestLogger;
 8 |
 9 |   constructor(page: Page, pageName = 'BasePage') {
10 |     this.page = page;
11 |     this.logger = new TestLogger(pageName);
12 |   }
13 |
14 |   public async navigateTo(pathOrUrl = '/'): Promise<void> {
15 |     const targetUrl = pathOrUrl.startsWith('http')
16 |       ? pathOrUrl
17 |       : `${EnvConfig.baseUrl}${pathOrUrl.startsWith('/') ? '' : '/'}${pathOrUrl}`;
18 |     this.logger.step(`Navigating to ${targetUrl}`);
19 |     await this.page.goto(targetUrl, { waitUntil: 'domcontentloaded' });
20 |   }
21 |
22 |   public getCurrentUrl(): string {
23 |     return this.page.url();
24 |   }
25 |
26 |   public async getPageTitle(): Promise<string> {
27 |     return await this.page.title();
28 |   }
29 |
30 |   public async verifyUrlContains(fragment: string): Promise<void> {
31 |     await expect(this.page).toHaveURL(new RegExp(fragment.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')));
32 |   }
33 |
34 |   public async reloadPage(): Promise<void> {
35 |     await this.page.reload({ waitUntil: 'domcontentloaded' });
36 |   }
37 |
38 |   public async getLocalStorageItem(key: string): Promise<string | null> {
39 |     return await this.page.evaluate((k) => window.localStorage.getItem(k), key);
40 |   }
41 |
42 |   public async setLocalStorageItem(key: string, value: string): Promise<void> {
43 |     await this.page.evaluate(({ k, v }) => window.localStorage.setItem(k, v), {
44 |       k: key,
45 |       v: value,
46 |     });
47 |   }
48 |
49 |   public async getSessionStorageItem(key: string): Promise<string | null> {
50 |     return await this.page.evaluate((k) => window.sessionStorage.getItem(k), key);
51 |   }
```

```
52 |
53 |   protected async getTrimmedText(locator: Locator): Promise<string> {
54 |     await locator.first().waitFor({ state: 'visible' });
55 |     const text = await locator.first().textContent();
56 |     return (text ?? '').trim();
57 |   }
58 | }
59 |
```

Checkpoint Verification Command After Creating Step 17: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 18 OF 39

Step 18: Build `src/pages/components/NavbarComponent.ts`

Create Global Navbar, i18n Switcher & Floating Widgets Component (109 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/components/NavbarComponent.ts" -Force`
Bash / macOS / Linux	`mkdir -p src/pages/components && touch src/pages/components/NavbarComponent.ts`
Prerequisite Files	src/pages/BasePage.ts
Consumed By	src/fixtures/base.fixture.ts, Suites 02, 03, 04, 05, 09

How to Implement This File (Line-Range Breakdown):

- Lines 1–73: Bind locators for navbar links, category dropdown items, language selector, cart quantity badge, AI chat widget, and live activity feed.
- Lines 74–110: Implement selectCategoryFromDropdown(), switchLanguage(), getCartBadgeCount(), and signOut().

Exact Source Code to Put in `src/pages/components/NavbarComponent.ts`:

```
  1 | import { Page, Locator, expect } from '@playwright/test';
  2 | import { BasePage } from '../BasePage';
  3 |
  4 | export class NavbarComponent extends BasePage {
  5 |   public readonly notificationBar: Locator;
  6 |   public readonly navHome: Locator;
  7 |   public readonly navCategories: Locator;
  8 |   public readonly navHandTools: Locator;
  9 |   public readonly navPowerTools: Locator;
 10 |   public readonly navOther: Locator;
 11 |   public readonly navSpecialTools: Locator;
 12 |   public readonly navRentals: Locator;
 13 |   public readonly navContact: Locator;
 14 |   public readonly navSignIn: Locator;
 15 |   public readonly navMenu: Locator;
 16 |   public readonly navMyAccount: Locator;
 17 |   public readonly navMyProfile: Locator;
 18 |   public readonly navMyInvoices: Locator;
 19 |   public readonly navMyFavorites: Locator;
 20 |   public readonly navMyMessages: Locator;
 21 |   public readonly navSignOut: Locator;
 22 |   public readonly navCart: Locator;
 23 |   public readonly cartQuantityBadge: Locator;
 24 |   public readonly languageSelect: Locator;
 25 |   public readonly chatToggle: Locator;
 26 |   public readonly chatWindow: Locator;
 27 |   public readonly chatInput: Locator;
 28 |   public readonly chatSend: Locator;
 29 |   public readonly chatClose: Locator;
 30 |   public readonly liveActivityToggle: Locator;
 31 |   public readonly liveActivityWindow: Locator;
 32 |   public readonly liveActivityClose: Locator;
 33 |   public readonly footerExplore: Locator;
 34 |   public readonly footerCourses: Locator;
 35 |   public readonly footerSpector: Locator;
 36 |   public readonly footerGithub: Locator;
 37 |
 38 |   constructor(page: Page) {
 39 |     super(page, 'NavbarComponent');
 40 |     this.notificationBar = page.locator('[data-test="notification-bar"]');
 41 |     this.navHome = page.locator('[data-test="nav-home"]');
 42 |     this.navCategories = page.locator('[data-test="nav-categories"]');
 43 |     this.navHandTools = page.locator('[data-test="nav-hand-tools"]');
 44 |     this.navPowerTools = page.locator('[data-test="nav-power-tools"]');
 45 |     this.navOther = page.locator('[data-test="nav-other"]');
 46 |     this.navSpecialTools = page.locator('[data-test="nav-special-tools"]');
 47 |     this.navRentals = page.locator('[data-test="nav-rentals"]');
 48 |     this.navContact = page.locator('[data-test="nav-contact"]');
 49 |     this.navSignIn = page.locator('[data-test="nav-sign-in"]');
```

```
    this.navMenu = page.locator('[data-test="nav-menu"]');
    this.navMyAccount = page.locator('[data-test="nav-my-account"]');
    this.navMyProfile = page.locator('[data-test="nav-my-profile"]');
    this.navMyInvoices = page.locator('[data-test="nav-my-invoices"]');
    this.navMyFavorites = page.locator('[data-test="nav-my-favorites"]');
    this.navMyMessages = page.locator('[data-test="nav-my-messages"]');
    this.navSignOut = page.locator('[data-test="nav-sign-out"]');
    this.navCart = page.locator('[data-test="nav-cart"]');
    this.cartQuantityBadge = page.locator('[data-test="cart-quantity"]');
    this.languageSelect = page.locator('[data-test="language-select"]');
    this.chatToggle = page.locator('[data-test="chat-toggle"]');
    this.chatWindow = page.locator('[data-test="chat-window"]');
    this.chatInput = page.locator('[data-test="chat-input"]');
    this.chatSend = page.locator('[data-test="chat-send"]');
    this.chatClose = page.locator('[data-test="chat-close"]');
    this.liveActivityToggle = page.locator('[data-test="live-activity-toggle"]');
    this.liveActivityWindow = page.locator('[data-test="live-activity-window"]');
    this.liveActivityClose = page.locator('[data-test="live-activity-close"]');
    this.footerExplore = page.locator('[data-test="footer-explore"]');
    this.footerCourses = page.locator('[data-test="footer-learn-courses"]');
    this.footerSpector = page.locator('[data-test="footer-learn-spector"]');
    this.footerGithub = page.locator('[data-test="footer-learn-github"]');
  }

  public async selectCategoryFromDropdown(
    target: 'hand-tools' | 'power-tools' | 'other' | 'special-tools' | 'rentals'
  ): Promise<void> {
    await this.navCategories.click();
    const map: Record<string, Locator> = {
      'hand-tools': this.navHandTools,
      'power-tools': this.navPowerTools,
      other: this.navOther,
      'special-tools': this.navSpecialTools,
      rentals: this.navRentals,
    };
    await map[target].click();
  }

  public async switchLanguage(
    langTestId: 'lang-en' | 'lang-de' | 'lang-es' | 'lang-fr' | 'lang-nl' | 'lang-tr' | 'lang-el'
  ): Promise<void> {
    await this.languageSelect.click();
    await this.page.locator(`[data-test="${langTestId}"]`).click();
  }

  public async getCartQuantityCount(): Promise<number> {
    if (!(await this.cartQuantityBadge.isVisible())) {
      return 0;
    }
    const raw = await this.cartQuantityBadge.textContent();
    return Number(raw?.trim() || 0);
  }

  public async signOut(): Promise<void> {
    await this.navMenu.click();
    await expect(this.navSignOut).toBeVisible();
    await this.navSignOut.click();
  }
}

```

Checkpoint Verification Command After Creating Step 18: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 19 OF 39

Step 19: Build `src/pages/CatalogPage.ts`

Create Home Product Catalog, Filter, Sort & Search Page Object (137 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/CatalogPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/CatalogPage.ts`
Prerequisite Files	src/pages/BasePage.ts, src/utils/math.helper.ts
Consumed By	src/fixtures/base.fixture.ts, Suites 01, 03, 09, 10, 11

How to Implement This File (Line-Range Breakdown):

- Lines 1–66: Bind locators for product cards, sort dropdown, search input/submit/reset, eco-friendly filter, and pagination.
- Lines 67–138: Implement open(), selectSortOrder(), searchForProduct(), resetSearch(), toggleCategoryByName(), toggleBrandByName(), goToPage(), and card name/price readers.

Exact Source Code to Put in `src/pages/CatalogPage.ts`:

```
import { Page, Locator, expect } from '@playwright/test';
import { BasePage } from './BasePage';
import { MathHelper } from '@utils/math.helper';

export type ToolshopSortOption =
  | 'name,asc'
  | 'name,desc'
  | 'price,desc'
  | 'price,asc'
  | 'co2_rating,asc'
  | 'co2_rating,desc';

export class CatalogPage extends BasePage {
  public readonly searchQueryInput: Locator;
  public readonly searchSubmitButton: Locator;
  public readonly searchResetButton: Locator;
  public readonly searchCaption: Locator;
  public readonly noResultsContainer: Locator;
  public readonly noResultsMessage: Locator;
  public readonly sortDropdown: Locator;
  public readonly ecoFriendlyFilter: Locator;
  public readonly productCards: Locator;
  public readonly productNames: Locator;
  public readonly productPrices: Locator;
  public readonly co2Badges: Locator;
  public readonly ecoBadges: Locator;
  public readonly outOfStockBadges: Locator;
  public readonly paginationPrev: Locator;
  public readonly paginationNext: Locator;
  public readonly pageTitle: Locator;

  constructor(page: Page) {
    super(page, 'CatalogPage');
    this.searchQueryInput = page.locator('[data-test="search-query"]');
    this.searchSubmitButton = page.locator('[data-test="search-submit"]');
    this.searchResetButton = page.locator('[data-test="search-reset"]');
    this.searchCaption = page.locator('[data-test="search-caption"]');
    this.noResultsContainer = page.locator('[data-test="no-results"]');
    this.noResultsMessage = page.locator('[data-test="no-results"]');
    this.sortDropdown = page.locator('[data-test="sort"]');
    this.ecoFriendlyFilter = page.locator('[data-test="eco-friendly-filter"]');
    this.productCards = page.locator('a.card[data-test^="product-"]');
    this.productNames = page.locator('[data-test="product-name"]');
    this.productPrices = page.locator('[data-test="product-price"]');
    this.co2Badges = page.locator('[data-test="co2-rating-badge"]');
    this.ecoBadges = page.locator('[data-test="eco-badge"]');
    this.outOfStockBadges = page.locator('[data-test="out-of-stock"]');
    this.paginationPrev = page.locator('a[aria-label="Previous"], [data-test="pagination-prev"]');
    this.paginationNext = page.locator('a[aria-label="Next"], [data-test="pagination-next"]');
```

```
    this.pageTitle = page.locator('[data-test="page-title"]');
  }

  public async open(): Promise<void> {
    await this.navigateTo('/');
    await expect(this.productCards.first()).toBeVisible();
  }

  public async searchFor(keyword: string): Promise<void> {
    await this.searchQueryInput.fill(keyword);
    await this.searchSubmitButton.click();
  }

  public async searchForProduct(keyword: string): Promise<void> {
    await this.searchFor(keyword);
  }

  public async resetSearch(): Promise<void> {
    await this.searchResetButton.click();
  }

  public async selectSort(option: ToolshopSortOption): Promise<void> {
    await this.sortDropdown.selectOption(option);
  }

  public async selectSortOption(option: ToolshopSortOption): Promise<void> {
    await this.selectSort(option);
  }

  public async toggleEcoFriendlyFilter(): Promise<void> {
    await this.ecoFriendlyFilter.click();
  }

  public getCategoryCheckbox(categoryId: string): Locator {
    return this.page.locator(`[data-test="category-${categoryId}"]`);
  }

  public async toggleCategoryFilter(categoryId: string): Promise<void> {
    const cb = this.getCategoryCheckbox(categoryId);
    await expect(cb).toBeVisible();
    await cb.click();
  }

  public getBrandCheckbox(brandId: string): Locator {
    return this.page.locator(`[data-test="brand-${brandId}"]`);
  }

  public async toggleBrandFilter(brandId: string): Promise<void> {
    const cb = this.getBrandCheckbox(brandId);
    await expect(cb).toBeVisible();
    await cb.click();
  }

  public getProductCardById(productId: string): Locator {
    return this.page.locator(`[data-test="product-${productId}"]`);
  }

  public getProductCardByName(name: string): Locator {
    return this.productCards.filter({
      has: this.page.locator('[data-test="product-name"]', { hasText: name }),
    });
  }

  public async getAllVisibleProductNames(): Promise<string[]> {
    await expect(this.productNames.first()).toBeVisible();
    const raw = await this.productNames.allTextContents();
    return raw.map((n) => n.trim());
  }

  public async getVisibleProductNames(): Promise<string[]> {
    return await this.getAllVisibleProductNames();
  }

  public async getAllVisibleProductPrices(): Promise<number[]> {
    await expect(this.productPrices.first()).toBeVisible();
    const raw = await this.productPrices.allTextContents();
    return raw.map((p) => MathHelper.parseCurrency(p));
```

```
  }

  public async getVisibleProductPrices(): Promise<number[]> {
    return await this.getAllVisibleProductPrices();
  }

  public async openProductByName(name: string): Promise<void> {
    await this.productNames.filter({ hasText: name }).first().click();
  }
}

```

Checkpoint Verification Command After Creating Step 19: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 20 OF 39

Step 20: Build `src/pages/ProductDetailsPage.ts`

Create Product Details Page (PDP), Specs & Cart Actions Page Object (81 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/ProductDetailsPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/ProductDetailsPage.ts`
Prerequisite Files	src/pages/BasePage.ts, src/utils/math.helper.ts
Consumed By	src/fixtures/base.fixture.ts, Suites 02, 06, 08, 10, 11

How to Implement This File (Line-Range Breakdown):

- Lines 1–43: Bind locators for productName, unitPrice, co2Badge, outOfStockBadge, quantity stepper, add-to-cart, add-to-favorites, and specs table.
- Lines 44–82: Implement openProductById(), getDetails(), addToCart() (awaiting [data-test="cart-quantity"] visibility), and getSpecificationsMap().

Exact Source Code to Put in `src/pages/ProductDetailsPage.ts` :

```
  1 | import { Page, Locator, expect } from '@playwright/test';
  2 | import { BasePage } from './BasePage';
  3 | import { MathHelper } from '@utils/math.helper';
  4 |
  5 | export class ProductDetailsPage extends BasePage {
  6 |   public readonly productName: Locator;
  7 |   public readonly unitPrice: Locator;
  8 |   public readonly productDescription: Locator;
  9 |   public readonly co2Badge: Locator;
 10 |   public readonly outOfStockBadge: Locator;
 11 |   public readonly decreaseQuantityBtn: Locator;
 12 |   public readonly quantityInput: Locator;
 13 |   public readonly increaseQuantityBtn: Locator;
 14 |   public readonly addToCartBtn: Locator;
 15 |   public readonly addToFavoritesBtn: Locator;
 16 |   public readonly addToCompareBtn: Locator;
 17 |   public readonly specsTable: Locator;
 18 |   public readonly specRows: Locator;
 19 |   public readonly toastAlert: Locator;
 20 |
 21 |   constructor(page: Page) {
 22 |     super(page, 'ProductDetailsPage');
 23 |     this.productName = page.locator('[data-test="product-name"]');
 24 |     this.unitPrice = page.locator('[data-test="unit-price"]');
 25 |     this.productDescription = page.locator('[data-test="product-description"]');
 26 |     this.co2Badge = page.locator('[data-test="co2-rating-badge"]');
 27 |     this.outOfStockBadge = page.locator('[data-test="out-of-stock"]');
 28 |     this.decreaseQuantityBtn = page.locator('[data-test="decrease-quantity"]');
 29 |     this.quantityInput = page.locator('[data-test="quantity"]');
 30 |     this.increaseQuantityBtn = page.locator('[data-test="increase-quantity"]');
 31 |     this.addToCartBtn = page.locator('[data-test="add-to-cart"]');
 32 |     this.addToFavoritesBtn = page.locator('[data-test="add-to-favorites"]');
 33 |     this.addToCompareBtn = page.locator('[data-test="add-to-compare"]');
 34 |     this.specsTable = page.locator('[data-test="product-specs"]');
 35 |     this.specRows = page.locator('[data-test="spec-row"]');
 36 |     this.toastAlert = page.locator('#toast-container, .ngx-toastr, .toast-message, div[role="alert"]').first();
 37 |   }
 38 |
 39 |   public async openProductById(productId: string): Promise<void> {
 40 |     await this.navigateTo(`/product/${productId}`);
 41 |     await expect(this.productName).toBeVisible();
 42 |   }
 43 |
 44 |   public async getDetails(): Promise<{
 45 |     name: string;
 46 |     price: number;
 47 |     description: string;
 48 |     co2Rating: string;
```

```
  }> {
    const name = await this.getTrimmedText(this.productName);
    const priceStr = await this.getTrimmedText(this.unitPrice);
    const description = await this.getTrimmedText(this.productDescription);
    const co2Rating = (await this.co2Badge.locator('.badge, span.active').first().textContent().catch(() => '')) || '';
    return {
      name,
      price: MathHelper.parseCurrency(priceStr),
      description,
      co2Rating: co2Rating.trim(),
    };
  }

  public async addToCart(quantity = 1): Promise<void> {
    if (quantity > 1) {
      await this.quantityInput.fill(String(quantity));
    }
    await this.addToCartBtn.click();
    await expect(this.page.locator('[data-test="cart-quantity"]')).toBeVisible();
  }

  public async getSpecificationsMap(): Promise<Record<string, string>> {
    const count = await this.specRows.count();
    const specs: Record<string, string> = {};
    for (let i = 0; i < count; i++) {
      const row = this.specRows.nth(i);
      const name = ((await row.locator('[data-test="spec-name"]').textContent()) ?? '').trim();
      const val = ((await row.locator('[data-test="spec-value"]').textContent()) ?? '').trim();
      specs[name] = val;
    }
    return specs;
  }
}
```

Checkpoint Verification Command After Creating Step 20: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 21 OF 39

Step 21: Build `src/pages/RentalsPage.ts`

Create Heavy Equipment Rentals Page Object (24 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/RentalsPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/RentalsPage.ts`
Prerequisite Files	src/pages/BasePage.ts
Consumed By	src/fixtures/base.fixture.ts, Suite 03

How to Implement This File (Line-Range Breakdown):

- Lines 1–25: Bind pageTitle and rentalCards locators and implement open() and getRentalCardById().

Exact Source Code to Put in `src/pages/RentalsPage.ts`:

```
 1 | import { Page, Locator, expect } from '@playwright/test';
 2 | import { BasePage } from './BasePage';
 3 |
 4 | export class RentalsPage extends BasePage {
 5 |   public readonly pageTitle: Locator;
 6 |   public readonly rentalCards: Locator;
 7 |
 8 |   constructor(page: Page) {
 9 |     super(page, 'RentalsPage');
10 |     this.pageTitle = page.locator('[data-test="page-title"]');
11 |     this.rentalCards = page.locator('[data-test^="product-"]');
12 |   }
13 |
14 |   public async open(): Promise<void> {
15 |     await this.navigateTo('/rentals');
16 |     await expect(this.pageTitle).toHaveText('Rentals');
17 |     await expect(this.rentalCards.first()).toBeVisible();
18 |   }
19 |
20 |   public getRentalCardById(rentalId: string): Locator {
21 |     return this.page.locator(`[data-test="product-${rentalId}"]`);
22 |   }
23 | }
24 |
```

Checkpoint Verification Command After Creating Step 21: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 22 OF 39

Step 22: Build `src/pages/AuthPage.ts`

Create Authentication, Registration & Fast JWT Injection Page Object (159 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/AuthPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/AuthPage.ts`
Prerequisite Files	src/pages/BasePage.ts, src/data/toolshop.users.data.ts
Consumed By	src/fixtures/base.fixture.ts, Suites 02, 04, 05, 06, 07, 08, 10, 11

How to Implement This File (Line-Range Breakdown):

- Lines 1–86: Bind all login, registration (including house_number), field error alerts, and forgot-password locators.
- Lines 87–160: Implement openLogin(), openRegister(), login(), fillRegistrationForm() (with blur trigger for Angular updateOn: "blur"), submitRegistration(), authenticateAsCustomer(), and authenticateAsAdmin().

Exact Source Code to Put in `src/pages/AuthPage.ts`:

```
  1 | import { Page, Locator, expect } from '@playwright/test';
  2 | import { BasePage } from './BasePage';
  3 | import {
  4 |   RegistrationCustomerProfile,
  5 |   TOOLSHOP_USERS,
  6 |   generateToolshopJwt,
  7 | } from '@data/toolshop.users.data';
  8 |
  9 | export class AuthPage extends BasePage {
 10 |   // Login form locators
 11 |   public readonly emailInput: Locator;
 12 |   public readonly passwordInput: Locator;
 13 |   public readonly loginSubmitButton: Locator;
 14 |   public readonly emailError: Locator;
 15 |   public readonly passwordError: Locator;
 16 |   public readonly loginError: Locator;
 17 |   public readonly registerLink: Locator;
 18 |   public readonly forgotPasswordLink: Locator;
 19 |
 20 |   // Registration form locators
 21 |   public readonly firstNameInput: Locator;
 22 |   public readonly lastNameInput: Locator;
 23 |   public readonly dobInput: Locator;
 24 |   public readonly streetInput: Locator;
 25 |   public readonly postalCodeInput: Locator;
 26 |   public readonly houseNumberInput: Locator;
 27 |   public readonly cityInput: Locator;
 28 |   public readonly stateInput: Locator;
 29 |   public readonly countrySelect: Locator;
 30 |   public readonly phoneInput: Locator;
 31 |   public readonly registerSubmitButton: Locator;
 32 |
 33 |   // Registration validation locators
 34 |   public readonly firstNameError: Locator;
 35 |   public readonly lastNameError: Locator;
 36 |   public readonly dobError: Locator;
 37 |   public readonly streetError: Locator;
 38 |   public readonly postalCodeError: Locator;
 39 |   public readonly cityError: Locator;
 40 |   public readonly stateError: Locator;
 41 |   public readonly countryError: Locator;
 42 |   public readonly phoneError: Locator;
 43 |   public readonly registerError: Locator;
 44 |
 45 |   // Forgot Password locators
 46 |   public readonly forgotPasswordSubmitButton: Locator;
 47 |   public readonly forgotPasswordSuccessAlert: Locator;
 48 |
 49 |   constructor(page: Page) {
```

```
    super(page, 'AuthPage');
    this.emailInput = page.locator('[data-test="email"]');
    this.passwordInput = page.locator('[data-test="password"]');
    this.loginSubmitButton = page.locator('[data-test="login-submit"]');
    this.emailError = page.locator('[data-test="email-error"]');
    this.passwordError = page.locator('[data-test="password-error"]');
    this.loginError = page.locator('[data-test="login-error"]');
    this.registerLink = page.locator('[data-test="register-link"]');
    this.forgotPasswordLink = page.locator('[data-test="forgot-password-link"]');

    this.firstNameInput = page.locator('[data-test="first-name"]');
    this.lastNameInput = page.locator('[data-test="last-name"]');
    this.dobInput = page.locator('[data-test="dob"]');
    this.streetInput = page.locator('[data-test="street"]');
    this.postalCodeInput = page.locator('[data-test="postal_code"]');
    this.houseNumberInput = page.locator('[data-test="house_number"]');
    this.cityInput = page.locator('[data-test="city"]');
    this.stateInput = page.locator('[data-test="state"]');
    this.countrySelect = page.locator('[data-test="country"]');
    this.phoneInput = page.locator('[data-test="phone"]');
    this.registerSubmitButton = page.locator('[data-test="register-submit"]');

    this.firstNameError = page.locator('[data-test="first-name-error"]');
    this.lastNameError = page.locator('[data-test="last-name-error"]');
    this.dobError = page.locator('[data-test="dob-error"]');
    this.streetError = page.locator('[data-test="street-error"]');
    this.postalCodeError = page.locator('[data-test="postal_code-error"]');
    this.cityError = page.locator('[data-test="city-error"]');
    this.stateError = page.locator('[data-test="state-error"]');
    this.countryError = page.locator('[data-test="country-error"]');
    this.phoneError = page.locator('[data-test="phone-error"]');
    this.registerError = page.locator('[data-test="register-error"]');

    this.forgotPasswordSubmitButton = page.locator('[data-test="forgot-password-submit"]');
    this.forgotPasswordSuccessAlert = page.locator('.alert-success');
  }

  public async openLogin(): Promise<void> {
    await this.navigateTo('/auth/login');
    await expect(this.emailInput).toBeVisible();
  }

  public async openRegister(): Promise<void> {
    await this.navigateTo('/auth/register');
    await expect(this.firstNameInput).toBeVisible();
  }

  public async openForgotPassword(): Promise<void> {
    await this.navigateTo('/auth/forgot-password');
    await expect(this.emailInput).toBeVisible();
  }

  public async login(email: string, password: string): Promise<void> {
    this.logger.step(`Submitting login for email: ${email}`);
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginSubmitButton.click();
  }

  public async fillRegistrationForm(profile: RegistrationCustomerProfile): Promise<void> {
    await this.firstNameInput.fill(profile.firstName);
    await this.lastNameInput.fill(profile.lastName);
    await this.dobInput.fill(profile.dob);
    if (profile.country) {
      await this.countrySelect.selectOption(profile.country);
    }
    await this.postalCodeInput.fill(profile.postalCode);
    if ((await this.houseNumberInput.count()) > 0) {
      await this.houseNumberInput.fill('98');
    }
    await this.streetInput.fill(profile.street);
    await this.cityInput.fill(profile.city);
    await this.stateInput.fill(profile.state);
    await this.phoneInput.fill(profile.phone);
    await this.emailInput.fill(profile.email);
    await this.passwordInput.fill(profile.password);
    await this.passwordInput.blur();
```

```
  }

  public async submitRegistration(profile: RegistrationCustomerProfile): Promise<void> {
    await this.fillRegistrationForm(profile);
    await this.registerSubmitButton.click();
  }

  public async submitForgotPassword(email: string): Promise<void> {
    await this.emailInput.fill(email);
    await this.forgotPasswordSubmitButton.click();
  }

  /**
   * Fast JWT injection into localStorage for authenticated workflow suites
   */
  public async authenticateAsCustomer(
    userKey: 'customer' | 'customer2' = 'customer'
  ): Promise<string> {
    const user = TOOLSHOP_USERS[userKey];
    const token = generateToolshopJwt({ sub: user.id, role: user.role });
    await this.navigateTo('/');
    await this.setLocalStorageItem('auth-token', token);
    return token;
  }

  public async authenticateAsAdmin(): Promise<string> {
    const admin = TOOLSHOP_USERS.admin;
    const token = generateToolshopJwt({ sub: admin.id, role: 'admin' });
    await this.navigateTo('/');
    await this.setLocalStorageItem('auth-token', token);
    return token;
  }
}
```

Checkpoint Verification Command After Creating Step 22: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 23 OF 39

Step 23: Build `src/pages/CheckoutWizardPage.ts`

Create 4-Step Shopping Cart & Multi-Payment Checkout Wizard Page Object (196 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/CheckoutWizardPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/CheckoutWizardPage.ts`
Prerequisite Files	src/pages/BasePage.ts, src/utils/math.helper.ts, src/data/toolshop.checkout.data.ts
Consumed By	src/fixtures/base.fixture.ts, Suites 06, 08, 11

How to Implement This File (Line-Range Breakdown):

- Lines 1–91: Define CartTableLineItem interface and bind locators for Step 1 (Cart Table), Step 2 (Sign-In), Step 3 (Billing Address including country select and house_number), and Step 4 (Payment Methods).
- Lines 92–197: Implement getCartLineItems(), getCartTotalPrice(), updateLineItemQuantity(), deleteLineItem(), fillBillingAddress(), selectAndFillPaymentMethod(), and confirmPayment().

Exact Source Code to Put in `src/pages/CheckoutWizardPage.ts` :

```
  1 | import { Page, Locator, expect } from '@playwright/test';
  2 | import { BasePage } from './BasePage';
  3 | import { parseCurrency } from '@utils/math.helper';
  4 | import { PaymentMethodScenario } from '@data/toolshop.checkout.data';
  5 |
  6 | export interface CartTableLineItem {
  7 |   title: string;
  8 |   quantity: number;
  9 |   unitPrice: number;
 10 |   lineTotal: number;
 11 | }
 12 |
 13 | export class CheckoutWizardPage extends BasePage {
 14 |   // Step 1: Cart Review Table
 15 |   public readonly productTitles: Locator;
 16 |   public readonly productQuantities: Locator;
 17 |   public readonly productPrices: Locator;
 18 |   public readonly linePrices: Locator;
 19 |   public readonly cartTotalCell: Locator;
 20 |   public readonly proceedStep1Button: Locator;
 21 |   public readonly continueShoppingButton: Locator;
 22 |
 23 |   // Step 2: Sign-In Check
 24 |   public readonly proceedStep2Button: Locator;
 25 |   public readonly emailInput: Locator;
 26 |   public readonly passwordInput: Locator;
 27 |   public readonly loginSubmitButton: Locator;
 28 |
 29 |   // Step 3: Billing Address
 30 |   public readonly streetInput: Locator;
 31 |   public readonly cityInput: Locator;
 32 |   public readonly stateInput: Locator;
 33 |   public readonly countryInput: Locator;
 34 |   public readonly postalCodeInput: Locator;
 35 |   public readonly houseNumberInput: Locator;
 36 |   public readonly proceedStep3Button: Locator;
 37 |
 38 |   // Step 4: Payment Methods & Finish
 39 |   public readonly paymentMethodSelect: Locator;
 40 |   public readonly bankNameInput: Locator;
 41 |   public readonly accountNameInput: Locator;
 42 |   public readonly accountNumberInput: Locator;
 43 |   public readonly creditCardNumberInput: Locator;
 44 |   public readonly expirationDateInput: Locator;
 45 |   public readonly cvvInput: Locator;
 46 |   public readonly cardHolderNameInput: Locator;
 47 |   public readonly monthlyInstallmentsSelect: Locator;
 48 |   public readonly giftCardNumberInput: Locator;
```

```
  public readonly validationCodeInput: Locator;
  public readonly finishButton: Locator;
  public readonly paymentSuccessMessage: Locator;
  public readonly orderConfirmationNumber: Locator;

  constructor(page: Page) {
    super(page, 'CheckoutWizardPage');
    this.productTitles = page.locator('[data-test="product-title"]');
    this.productQuantities = page.locator('[data-test="product-quantity"]');
    this.productPrices = page.locator('[data-test="product-price"]');
    this.linePrices = page.locator('[data-test="line-price"]');
    this.cartTotalCell = page.locator('[data-test="cart-total"]');
    this.proceedStep1Button = page.locator('[data-test="proceed-1"]');
    this.continueShoppingButton = page.locator('[data-test="continue-shopping"]');

    this.proceedStep2Button = page.locator('[data-test="proceed-2"]');
    this.emailInput = page.locator('[data-test="email"]');
    this.passwordInput = page.locator('[data-test="password"]');
    this.loginSubmitButton = page.locator('[data-test="login-submit"]');

    this.streetInput = page.locator('[data-test="street"]');
    this.cityInput = page.locator('[data-test="city"]');
    this.stateInput = page.locator('[data-test="state"]');
    this.countryInput = page.locator('[data-test="country"]');
    this.postalCodeInput = page.locator('[data-test="postal_code"]');
    this.houseNumberInput = page.locator('[data-test="house_number"]');
    this.proceedStep3Button = page.locator('[data-test="proceed-3"]');

    this.paymentMethodSelect = page.locator('[data-test="payment-method"]');
    this.bankNameInput = page.locator('[data-test="bank_name"]');
    this.accountNameInput = page.locator('[data-test="account_name"]');
    this.accountNumberInput = page.locator('[data-test="account_number"]');
    this.creditCardNumberInput = page.locator('[data-test="credit_card_number"]');
    this.expirationDateInput = page.locator('[data-test="expiration_date"]');
    this.cvvInput = page.locator('[data-test="cvv"]');
    this.cardHolderNameInput = page.locator('[data-test="card_holder_name"]');
    this.monthlyInstallmentsSelect = page.locator('[data-test="monthly_installments"]');
    this.giftCardNumberInput = page.locator('[data-test="gift_card_number"]');
    this.validationCodeInput = page.locator('[data-test="validation_code"]');
    this.finishButton = page.locator('[data-test="finish"]');
    this.paymentSuccessMessage = page.locator('[data-test="payment-success-message"]');
    this.orderConfirmationNumber = page.locator('#order-confirmation');
  }

  public async openCheckout(): Promise<void> {
    await this.navigateTo('/checkout');
  }

  public async getCartLineItems(): Promise<CartTableLineItem[]> {
    const count = await this.productTitles.count();
    const items: CartTableLineItem[] = [];
    for (let i = 0; i < count; i++) {
      const title = ((await this.productTitles.nth(i).textContent()) ?? '').trim();
      const qtyStr = await this.productQuantities.nth(i).inputValue();
      const unitStr = ((await this.productPrices.nth(i).textContent()) ?? '').trim();
      const totalStr = ((await this.linePrices.nth(i).textContent()) ?? '').trim();
      items.push({
        title,
        quantity: parseInt(qtyStr, 10) || 0,
        unitPrice: parseCurrency(unitStr),
        lineTotal: parseCurrency(totalStr),
      });
    }
    return items;
  }

  public async getCartTotalPrice(): Promise<number> {
    const text = await this.getTrimmedText(this.cartTotalCell);
    return parseCurrency(text);
  }

  public async updateLineItemQuantity(rowIndex: number, newQuantity: number): Promise<void> {
    const input = this.productQuantities.nth(rowIndex);
    await input.fill(String(newQuantity));
    await input.press('Enter');
  }

```

```
126 |   public async deleteLineItem(rowIndex: number): Promise<void> {
127 |     const deleteButtons = this.page.locator('a.btn-danger');
128 |     await deleteButtons.nth(rowIndex).click();
129 |   }
130 |
131 |   public async proceedFromCartStep(): Promise<void> {
132 |     await expect(this.proceedStep1Button).toBeVisible();
133 |     await this.proceedStep1Button.click();
134 |   }
135 |
136 |   public async proceedFromSignInStep(): Promise<void> {
137 |     await expect(this.proceedStep2Button).toBeVisible();
138 |     await this.proceedStep2Button.click();
139 |   }
140 |
141 |   public async fillBillingAddress(address: {
142 |     street: string;
143 |     city: string;
144 |     state: string;
145 |     country: string;
146 |     postal_code: string;
147 |   }): Promise<void> {
148 |     await expect(this.streetInput).toBeVisible();
149 |     await this.countryInput.selectOption(address.country);
150 |     await this.postalCodeInput.fill(address.postal_code);
151 |     if ((await this.houseNumberInput.count()) > 0) {
152 |       await this.houseNumberInput.fill('98');
153 |     }
154 |     await this.streetInput.fill(address.street);
155 |     await this.cityInput.fill(address.city);
156 |     await this.stateInput.fill(address.state);
157 |   }
158 |
159 |   public async proceedFromAddressStep(): Promise<void> {
160 |     await expect(this.proceedStep3Button).toBeEnabled();
161 |     await this.proceedStep3Button.click();
162 |   }
163 |
164 |   public async selectAndFillPaymentMethod(scenario: PaymentMethodScenario): Promise<void> {
165 |     await expect(this.paymentMethodSelect).toBeVisible();
166 |     await this.paymentMethodSelect.selectOption(scenario.methodValue);
167 |
168 |     switch (scenario.methodValue) {
169 |       case 'bank-transfer':
170 |         await this.bankNameInput.fill(scenario.fields.bank_name);
171 |         await this.accountNameInput.fill(scenario.fields.account_name);
172 |         await this.accountNumberInput.fill(scenario.fields.account_number);
173 |         break;
174 |       case 'cash-on-delivery':
175 |         break;
176 |       case 'credit-card':
177 |         await this.creditCardNumberInput.fill(scenario.fields.credit_card_number);
178 |         await this.expirationDateInput.fill(scenario.fields.expiration_date);
179 |         await this.cvvInput.fill(scenario.fields.cvv);
180 |         await this.cardHolderNameInput.fill(scenario.fields.card_holder_name);
181 |         break;
182 |       case 'buy-now-pay-later':
183 |         await this.monthlyInstallmentsSelect.selectOption(scenario.fields.monthly_installments);
184 |         break;
185 |       case 'gift-card':
186 |         await this.giftCardNumberInput.fill(scenario.fields.gift_card_number);
187 |         await this.validationCodeInput.fill(scenario.fields.validation_code);
188 |         break;
189 |     }
190 |   }
191 |
192 |   public async confirmPayment(): Promise<void> {
193 |     await expect(this.finishButton).toBeEnabled();
194 |     await this.finishButton.click();
195 |   }
196 | }
```

Checkpoint Verification Command After Creating Step 23: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 24 OF 39

Step 24: Build `src/pages/ContactPage.ts`

Create Customer Support Ticket & File Upload Page Object (62 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/ContactPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/ContactPage.ts`
Prerequisite Files	src/pages/BasePage.ts
Consumed By	src/fixtures/base.fixture.ts, Suites 07, 10, 11

How to Implement This File (Line-Range Breakdown):

- Lines 1–63: Bind contact form inputs and validation error locators, and implement open() and fillContactForm().

Exact Source Code to Put in `src/pages/ContactPage.ts`:

```
import { Page, Locator, expect } from '@playwright/test';
import { BasePage } from './BasePage';
import { ContactTicketScenario } from '@data/toolshop.checkout.data';

export class ContactPage extends BasePage {
  public readonly firstNameInput: Locator;
  public readonly lastNameInput: Locator;
  public readonly emailInput: Locator;
  public readonly subjectSelect: Locator;
  public readonly messageInput: Locator;
  public readonly attachmentInput: Locator;
  public readonly contactSubmitButton: Locator;

  // Validation error locators
  public readonly firstNameError: Locator;
  public readonly lastNameError: Locator;
  public readonly emailError: Locator;
  public readonly subjectError: Locator;
  public readonly messageError: Locator;
  public readonly attachmentError: Locator;
  public readonly successAlert: Locator;

  constructor(page: Page) {
    super(page, 'ContactPage');
    this.firstNameInput = page.locator('[data-test="first-name"]');
    this.lastNameInput = page.locator('[data-test="last-name"]');
    this.emailInput = page.locator('[data-test="email"]');
    this.subjectSelect = page.locator('[data-test="subject"]');
    this.messageInput = page.locator('[data-test="message"]');
    this.attachmentInput = page.locator('[data-test="attachment"]');
    this.contactSubmitButton = page.locator('[data-test="contact-submit"]');

    this.firstNameError = page.locator('[data-test="first-name-error"]');
    this.lastNameError = page.locator('[data-test="last-name-error"]');
    this.emailError = page.locator('[data-test="email-error"]');
    this.subjectError = page.locator('[data-test="subject-error"]');
    this.messageError = page.locator('[data-test="message-error"]');
    this.attachmentError = page.locator('[data-test="attachment-error"]');
    this.successAlert = page.locator('.alert-success');
  }

  public async open(): Promise<void> {
    await this.navigateTo('/contact');
    await expect(this.contactSubmitButton).toBeVisible();
  }

  public async fillContactForm(ticket: ContactTicketScenario): Promise<void> {
    if (await this.firstNameInput.isVisible()) {
      await this.firstNameInput.fill(ticket.firstName);
      await this.lastNameInput.fill(ticket.lastName);
      await this.emailInput.fill(ticket.email);
    }
```

```
53 |     await this.subjectSelect.selectOption(ticket.subjectValue);
54 |     await this.messageInput.fill(ticket.message);
55 |   }
56 |
57 |   public async submitContactTicket(ticket: ContactTicketScenario): Promise<void> {
58 |     await this.fillContactForm(ticket);
59 |     await this.contactSubmitButton.click();
60 |   }
61 | }
62 |
```

Checkpoint Verification Command After Creating Step 24: `npm run typecheck`

STAGE 7: PAGE OBJECT MODEL (POM) & COMPONENT LAYER — STEP 25 OF 39

Step 25: Build `src/pages/AccountPortalPage.ts`

Create Customer Account Portal, Favorites & Invoices Page Object (92 Lines)

PowerShell Command	`New-Item -ItemType File -Path "src/pages/AccountPortalPage.ts" -Force`
Bash / macOS / Linux	`touch src/pages/AccountPortalPage.ts`
Prerequisite Files	src/pages/BasePage.ts
Consumed By	src/fixtures/base.fixture.ts, Suite 08

How to Implement This File (Line-Range Breakdown):

- Lines 1–93: Bind account navigation links, profile fields, password change inputs, favorite cards, and invoice table cells, and implement openDashboard(), openProfile(), openFavorites(), openInvoices(), and openMessages().

Exact Source Code to Put in `src/pages/AccountPortalPage.ts`:

```
  1 | import { Page, Locator, expect } from '@playwright/test';
  2 | import { BasePage } from './BasePage';
  3 |
  4 | export class AccountPortalPage extends BasePage {
  5 |   public readonly pageTitle: Locator;
  6 |   public readonly navFavoritesLink: Locator;
  7 |   public readonly navProfileLink: Locator;
  8 |   public readonly navInvoicesLink: Locator;
  9 |   public readonly navMessagesLink: Locator;
 10 |
 11 |   // Profile fields
 12 |   public readonly firstNameInput: Locator;
 13 |   public readonly lastNameInput: Locator;
 14 |   public readonly emailInput: Locator;
 15 |   public readonly phoneInput: Locator;
 16 |   public readonly streetInput: Locator;
 17 |   public readonly postalCodeInput: Locator;
 18 |   public readonly cityInput: Locator;
 19 |   public readonly stateInput: Locator;
 20 |   public readonly countryInput: Locator;
 21 |   public readonly updateProfileSubmit: Locator;
 22 |   public readonly currentPasswordInput: Locator;
 23 |   public readonly newPasswordInput: Locator;
 24 |   public readonly newPasswordConfirmInput: Locator;
 25 |   public readonly changePasswordSubmit: Locator;
 26 |
 27 |   // Favorites cards
 28 |   public readonly favoriteCards: Locator;
 29 |   public readonly deleteFavoriteButtons: Locator;
 30 |
 31 |   // Invoices table
 32 |   public readonly invoiceRows: Locator;
 33 |   public readonly invoiceNumberCells: Locator;
 34 |   public readonly invoiceTotalCells: Locator;
 35 |
 36 |   constructor(page: Page) {
 37 |     super(page, 'AccountPortalPage');
 38 |     this.pageTitle = page.locator('[data-test="page-title"]');
 39 |     this.navFavoritesLink = page.locator('[data-test="nav-favorites"]');
 40 |     this.navProfileLink = page.locator('[data-test="nav-profile"]');
 41 |     this.navInvoicesLink = page.locator('[data-test="nav-invoices"]');
 42 |     this.navMessagesLink = page.locator('[data-test="nav-messages"]');
 43 |
 44 |     this.firstNameInput = page.locator('[data-test="first-name"]');
 45 |     this.lastNameInput = page.locator('[data-test="last-name"]');
 46 |     this.emailInput = page.locator('[data-test="email"]');
 47 |     this.phoneInput = page.locator('[data-test="phone"]');
 48 |     this.streetInput = page.locator('[data-test="street"]');
 49 |     this.postalCodeInput = page.locator('[data-test="postal_code"]');
 50 |     this.cityInput = page.locator('[data-test="city"]');
 51 |     this.stateInput = page.locator('[data-test="state"]');
```

```
    this.countryInput = page.locator('[data-test="country"]');
    this.updateProfileSubmit = page.locator('[data-test="update-profile-submit"]');
    this.currentPasswordInput = page.locator('[data-test="current-password"]');
    this.newPasswordInput = page.locator('[data-test="new-password"]');
    this.newPasswordConfirmInput = page.locator('[data-test="new-password-confirm"]');
    this.changePasswordSubmit = page.locator('[data-test="change-password-submit"]');

    this.favoriteCards = page.locator('[data-test^="favorite-"]');
    this.deleteFavoriteButtons = page.locator('[data-test="delete"]');

    this.invoiceRows = page.locator('table tbody tr');
    this.invoiceNumberCells = page.locator('table tbody tr td:nth-child(1)');
    this.invoiceTotalCells = page.locator('table tbody tr td:nth-child(4)');
  }

  public async openDashboard(): Promise<void> {
    await this.navigateTo('/account');
    await expect(this.pageTitle).toBeVisible();
  }

  public async openProfile(): Promise<void> {
    await this.navigateTo('/account/profile');
    await expect(this.firstNameInput).toBeVisible();
  }

  public async openFavorites(): Promise<void> {
    await this.navigateTo('/account/favorites');
    await expect(this.pageTitle).toBeVisible();
  }

  public async openInvoices(): Promise<void> {
    await this.navigateTo('/account/invoices');
    await expect(this.pageTitle).toBeVisible();
    await expect(this.invoiceRows.first()).toBeVisible();
  }

  public async openMessages(): Promise<void> {
    await this.navigateTo('/account/messages');
    await expect(this.pageTitle).toBeVisible();
  }
}
```

Checkpoint Verification Command After Creating Step 25: `npm run typecheck`

STAGE 8: MASTER PLAYWRIGHT RUNNER CONFIGURATION — STEP 26 OF 39

Step 26: Build `playwright.config.ts`

Create Master Playwright Test Runner & Multi-Reporter Configuration (42 Lines)

PowerShell Command	`New-Item -ItemType File -Path "playwright.config.ts" -Force`
Bash / macOS / Linux	`touch playwright.config.ts`
Prerequisite Files	@playwright/test, src/config/env.config.ts, src/utils/custom-reporter.ts
Consumed By	Playwright CLI (npx playwright test)

How to Implement This File (Line-Range Breakdown):

- Lines 1–24: Configure testDir ("./tests"), fullyParallel: true, workers: 6, timeouts, and the 5-reporter pipeline (list, html, json, junit, and custom-reporter.ts).
- Lines 25–43: Configure baseURL, trace/screenshot/video policies, 1920x1080 viewport, and the chromium-enterprise browser project.

Exact Source Code to Put in `playwright.config.ts` :

```
import { defineConfig, devices } from '@playwright/test';
import { EnvConfig } from './src/config/env.config';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : EnvConfig.execution.retries,
  workers: process.env.CI ? 4 : EnvConfig.execution.workers,
  timeout: 45000,
  expect: {
    timeout: 15000,
  },
  reporter: [
    ['list'],
    ['html', { outputFolder: 'playwright-report', open: 'never' }],
    ['json', { outputFile: 'reports/playwright-results.json' }],
    ['junit', { outputFile: 'reports/junit-results.xml' }],
    ['./src/utils/custom-reporter.ts'],
  ],
  use: {
    baseURL: EnvConfig.baseUrl,
    headless: EnvConfig.execution.headless,
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'on-first-retry',
    actionTimeout: 15000,
    navigationTimeout: 30000,
    viewport: { width: 1280, height: 720 },
    ignoreHTTPSErrors: true,
  },
  projects: [
    {
      name: 'chromium-enterprise',
      use: {
        ...devices['Desktop Chrome'],
        viewport: { width: 1280, height: 720 },
      },
    },
  ],
});

```

Checkpoint Verification Command After Creating Step 26: `npm run typecheck`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 27 OF 39

Step 27: Build `tests/01-catalog-filtering-sorting/catalog.spec.ts`

Create Suite 01: Product Catalog, Filtering, Sorting & Search (38 Tests) (256 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/01-catalog-filtering-sorting/catalog.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/01-catalog-filtering-sorting && touch tests/01-catalog-filtering-sorting/catalog.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.catalog.data.ts
Consumed By	npm run test:catalog

How to Implement This File (Line-Range Breakdown):

- Lines 1–66: Implement CAT-001 to CAT-005 verifying Page 1 (9 items), Page 2 (3 items), out-of-stock badge, eco badges, and CO2 ratings.
- Lines 67–178: Implement CAT-006 to CAT-029 verifying all 6 sort modes, all 14 category checkboxes, and all 4 brand filters.
- Lines 179–257: Implement CAT-030 to CAT-038 verifying keyword search, reset search, no-results state, eco filter, and combined brand+category filters.

Exact Source Code to Put in `tests/01-catalog-filtering-sorting/catalog.spec.ts`:

```
import { test, expect } from '@fixtures/base.fixture';
import {
  TOOLSHOP_PRODUCTS,
  TOOLSHOP_BRANDS,
  TOOLSHOP_CATEGORIES_TREE,
} from '@data/toolshop.catalog.data';

test.describe('Suite 01: Product Catalog, Multi-Faceted Filtering, Sorting & Search Engine', () => {
  test.beforeEach(async ({ catalogPage }) => {
    await catalogPage.open();
  });

  test.describe('Catalog Grid & Pagination Verification', () => {
    test('CAT-001: Home catalog page renders initial paginated grid of 9 hardware products', async ({
      catalogPage,
    }) => {
      await expect(catalogPage.productCards).toHaveCount(9);
      const names = await catalogPage.getVisibleProductNames();
      expect(names.length).toBe(9);
      expect(names[0]).toBe(TOOLSHOP_PRODUCTS[0].name);
    });

    test('CAT-002: Navigating to Page 2 displays the remaining 3 catalog products', async ({
      catalogPage,
      page,
    }) => {
      const page2Button = page.getByRole('button', { name: 'Page-2' });
      await expect(page2Button).toBeVisible();
      await page2Button.click();

      await expect(catalogPage.productCards).toHaveCount(3);
      const names = await catalogPage.getVisibleProductNames();
      expect(names).toEqual(TOOLSHOP_PRODUCTS.slice(9, 12).map((p) => p.name));
    });

    test('CAT-003: Out-of-stock product displays [data-test="out-of-stock"] badge on catalog card', async ({
      catalogPage,
    }) => {
      const outOfStockProduct = TOOLSHOP_PRODUCTS.find((p) => !p.in_stock)!;
      const card = catalogPage.getProductCardById(outOfStockProduct.id);
      await expect(card).toBeVisible();
      await expect(card.locator('[data-test="out-of-stock"]')).toBeVisible();
      await expect(card.locator('[data-test="out-of-stock"]')).toContainText('Out of stock');
```

```
  });

  test('CAT-004: Eco-friendly products display [data-test="eco-badge"] on catalog cards', async ({
    catalogPage,
  }) => {
    const ecoProductsPage1 = TOOLSHOP_PRODUCTS.slice(0, 9).filter((p) => p.is_eco_friendly);
    for (const product of ecoProductsPage1) {
      const card = catalogPage.getProductCardById(product.id);
      await expect(card.locator('[data-test="eco-badge"]')).toBeVisible();
    }
  });

  test('CAT-005: Every catalog card renders a valid CO2 rating badge (A-E) with active highlight', async ({
    catalogPage,
  }) => {
    const page1Products = TOOLSHOP_PRODUCTS.slice(0, 9);
    for (const product of page1Products) {
      const card = catalogPage.getProductCardById(product.id);
      const co2Badge = card.locator('[data-test="co2-rating-badge"]');
      await expect(co2Badge).toBeVisible();
      const activeLetter = co2Badge.locator('.co2-letter.active');
      await expect(activeLetter).toHaveText(product.co2_rating);
    }
  });
});

test.describe('Multi-Criteria Sorting Engine', () => {
  test('CAT-006: Sort by Name (A - Z) orders catalog alphabetically ascending', async ({
    catalogPage,
  }) => {
    await catalogPage.selectSortOption('name,asc');
    const expectedFirst = [...TOOLSHOP_PRODUCTS].sort((a, b) => a.name.localeCompare(b.name))[0];
    await expect(catalogPage.productNames.first()).toHaveText(expectedFirst.name);

    const visibleNames = await catalogPage.getVisibleProductNames();
    const sortedCopy = [...visibleNames].sort((a, b) => a.localeCompare(b));
    expect(visibleNames).toEqual(sortedCopy);
  });

  test('CAT-007: Sort by Name (Z - A) orders catalog alphabetically descending', async ({
    catalogPage,
  }) => {
    await catalogPage.selectSortOption('name,desc');
    const expectedFirst = [...TOOLSHOP_PRODUCTS].sort((a, b) => b.name.localeCompare(a.name))[0];
    await expect(catalogPage.productNames.first()).toHaveText(expectedFirst.name);

    const visibleNames = await catalogPage.getVisibleProductNames();
    const sortedCopy = [...visibleNames].sort((a, b) => b.localeCompare(a));
    expect(visibleNames).toEqual(sortedCopy);
  });

  test('CAT-008: Sort by Price (Low - High) orders catalog by ascending price', async ({
    catalogPage,
  }) => {
    await catalogPage.selectSortOption('price,asc');
    const expectedFirst = [...TOOLSHOP_PRODUCTS].sort((a, b) => a.price - b.price)[0];
    await expect(catalogPage.productNames.first()).toHaveText(expectedFirst.name);

    const prices = await catalogPage.getVisibleProductPrices();
    for (let i = 1; i < prices.length; i++) {
      expect(prices[i]).toBeGreaterThanOrEqual(prices[i - 1]);
    }
  });

  test('CAT-009: Sort by Price (High - Low) orders catalog by descending price', async ({
    catalogPage,
  }) => {
    await catalogPage.selectSortOption('price,desc');
    const expectedFirst = [...TOOLSHOP_PRODUCTS].sort((a, b) => b.price - a.price)[0];
    await expect(catalogPage.productNames.first()).toHaveText(expectedFirst.name);

    const prices = await catalogPage.getVisibleProductPrices();
    for (let i = 1; i < prices.length; i++) {
      expect(prices[i]).toBeLessThanOrEqual(prices[i - 1]);
    }
  });

```

```
  121 |     test('CAT-010: Sort by CO2 Rating (A - E) places eco-optimal tools first', async ({
  122 |       catalogPage,
  123 |     }) => {
  124 |       await catalogPage.selectSortOption('co2_rating,asc');
  125 |       const firstActiveLetter = catalogPage.productCards
  126 |         .first()
  127 |         .locator('[data-test="co2-rating-badge"] .co2-letter.active');
  128 |       await expect(firstActiveLetter).toHaveText('A');
  129 |     });
  130 |
  131 |     test('CAT-011: Sort by CO2 Rating (E - A) places highest CO2 rating tools first', async ({
  132 |       catalogPage,
  133 |     }) => {
  134 |       await catalogPage.selectSortOption('co2_rating,desc');
  135 |       const firstActiveLetter = catalogPage.productCards
  136 |         .first()
  137 |         .locator('[data-test="co2-rating-badge"] .co2-letter.active');
  138 |       await expect(firstActiveLetter).toHaveText('E');
  139 |     });
  140 |   });
  141 |
  142 |   test.describe('Parameterized Product Search Verification (All 12 Catalog Tools)', () => {
  143 |     for (const [index, product] of TOOLSHOP_PRODUCTS.entries()) {
  144 |       const testId = `CAT-${String(index + 12).padStart(3, '0')}`;
  145 |       test(`${testId}: Searching for "${product.name}" returns matching product card with $${product.price.toFixed(2)}`,
async ({
  146 |         catalogPage,
  147 |       }) => {
  148 |         await catalogPage.searchForProduct(product.name);
  149 |         await expect(catalogPage.searchCaption).toContainText(product.name);
  150 |         const matchingCard = catalogPage.getProductCardById(product.id);
  151 |         await expect(matchingCard).toBeVisible();
  152 |         await expect(matchingCard.locator('[data-test="product-name"]')).toHaveText(product.name);
  153 |         await expect(matchingCard.locator('[data-test="product-price"]')).toContainText(
  154 |           product.price.toFixed(2)
  155 |         );
  156 |       });
  157 |     }
  158 |
  159 |     test('CAT-024: Searching with case-insensitive partial keyword "pliers" returns all 2 pliers tools', async ({
  160 |       catalogPage,
  161 |     }) => {
  162 |       const expectedPliersCount = TOOLSHOP_PRODUCTS.filter((p) =>
  163 |         p.name.toLowerCase().includes('pliers')
  164 |       ).length;
  165 |       await catalogPage.searchForProduct('pliers');
  166 |       await expect(catalogPage.productCards).toHaveCount(expectedPliersCount);
  167 |       const names = await catalogPage.getVisibleProductNames();
  168 |       for (const name of names) {
  169 |         expect(name.toLowerCase()).toContain('pliers');
  170 |       }
  171 |     });
  172 |
  173 |     test('CAT-025: Searching for non-existent tool displays [data-test="no-results"] empty state', async ({
  174 |       catalogPage,
  175 |     }) => {
  176 |       await catalogPage.searchForProduct('QuantumPlasmaCutter9999');
  177 |       await expect(catalogPage.noResultsMessage).toBeVisible();
  178 |       await expect(catalogPage.productCards).toHaveCount(0);
  179 |     });
  180 |
  181 |     test('CAT-026: Clicking [data-test="search-reset"] clears search filter and restores full catalog', async ({
  182 |       catalogPage,
  183 |     }) => {
  184 |       await catalogPage.searchForProduct('Thor Hammer');
  185 |       await expect(catalogPage.productCards).toHaveCount(1);
  186 |
  187 |       await catalogPage.resetSearch();
  188 |       await expect(catalogPage.productCards).toHaveCount(9);
  189 |     });
  190 |   });
  191 |
  192 |   test.describe('Faceted Brand, Category & Sustainability Filtering', () => {
  193 |     for (const [bIdx, brand] of TOOLSHOP_BRANDS.entries()) {
  194 |       const testNum = 27 + bIdx;
  195 |       test(`CAT-0${testNum}: Filtering by brand "${brand.name}" displays only ${brand.name} products`, async ({
  196 |         catalogPage,
```

```
197 |       }) => {
198 |         const expectedCount = TOOLSHOP_PRODUCTS.filter((p) => p.brand.id === brand.id).length;
199 |         await catalogPage.toggleBrandFilter(brand.id);
200 |         await expect(catalogPage.productCards).toHaveCount(Math.min(expectedCount, 9));
201 |       });
202 |     }
203 |
204 |     test('CAT-029: Filtering by Eco-Friendly checkbox displays exclusively eco-certified products', async ({
205 |       catalogPage,
206 |     }) => {
207 |       const expectedEcoProducts = TOOLSHOP_PRODUCTS.filter((p) => p.is_eco_friendly);
208 |       await catalogPage.toggleEcoFriendlyFilter();
209 |       await expect(catalogPage.productCards).toHaveCount(expectedEcoProducts.length);
210 |
211 |       const badgeCount = await catalogPage.ecoBadges.count();
212 |       expect(badgeCount).toBe(expectedEcoProducts.length);
213 |     });
214 |
215 |     const populatedSubCategories = TOOLSHOP_CATEGORIES_TREE.flatMap(
216 |       (root) => root.sub_categories ?? []
217 |     ).filter((sub) => TOOLSHOP_PRODUCTS.some((p) => p.category.id === sub.id));
218 |
219 |     for (const [sIdx, subCat] of populatedSubCategories.entries()) {
220 |       const testNum = 30 + sIdx;
221 |       test(`CAT-0${testNum}: Filtering by subcategory "${subCat.name}" filters grid accurately`, async ({
222 |         catalogPage,
223 |       }) => {
224 |         const expectedProducts = TOOLSHOP_PRODUCTS.filter((p) => p.category.id === subCat.id);
225 |         await catalogPage.toggleCategoryFilter(subCat.id);
226 |         await expect(catalogPage.productCards).toHaveCount(expectedProducts.length);
227 |         const visibleNames = await catalogPage.getVisibleProductNames();
228 |         expect(visibleNames.sort()).toEqual(expectedProducts.map((p) => p.name).sort());
229 |       });
230 |     }
231 |
232 |     test('CAT-039: Filtering by Root Category "Power Tools" aggregates all child subcategory products', async ({
233 |       catalogPage,
234 |     }) => {
235 |       const powerToolsRoot = TOOLSHOP_CATEGORIES_TREE.find((c) => c.slug === 'power-tools')!;
236 |       const childIds = new Set((powerToolsRoot.sub_categories ?? []).map((s) => s.id));
237 |       const expectedProducts = TOOLSHOP_PRODUCTS.filter((p) => childIds.has(p.category.id));
238 |
239 |       await catalogPage.toggleCategoryFilter(powerToolsRoot.id);
240 |       await expect(catalogPage.productCards).toHaveCount(expectedProducts.length);
241 |     });
242 |
243 |     test('CAT-040: Combining Brand filter + Sort by Price (High - Low) applies compound query parameters', async ({
244 |       catalogPage,
245 |     }) => {
246 |       const forgeFlex = TOOLSHOP_BRANDS[0];
247 |       await catalogPage.toggleBrandFilter(forgeFlex.id);
248 |       await catalogPage.selectSortOption('price,desc');
249 |
250 |       const expectedSorted = TOOLSHOP_PRODUCTS.filter((p) => p.brand.id === forgeFlex.id).sort(
251 |         (a, b) => b.price - a.price
252 |       );
253 |       await expect(catalogPage.productNames.first()).toHaveText(expectedSorted[0].name);
254 |     });
255 |   });
256 | });
```

Checkpoint Verification Command After Creating Step 27: `npm run test:catalog`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 28 OF 39

Step 28: Build `tests/02-product-details-specs/product-details.spec.ts`

Create Suite 02: Product Details Page (PDP), Specs & Cart Actions (40 Tests) (125 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/02-product-details-specs/product-details.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/02-product-details-specs && touch tests/02-product-details-specs/product-details.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.catalog.data.ts
Consumed By	npm run test:pdp

How to Implement This File (Line-Range Breakdown):

- Lines 1–55: Implement PDP-001 to PDP-035 verifying metadata, 4-row technical specification tables, and add-to-cart badge updates across all 12 tools.
- Lines 56–126: Implement PDP-036 to PDP-040 verifying out-of-stock disabled buttons, quantity increment/decrement bounds, related products, and add-to-favorites toast.

Exact Source Code to Put in `tests/02-product-details-specs/product-details.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import { TOOLSHOP_PRODUCTS } from '@data/toolshop.catalog.data';
  3 |
  4 | test.describe('Suite 02: Product Details Page (PDP), Technical Specs, CO2 Badges & Cart Actions', () => {
  5 |   test.describe('Parameterized Product Details Metadata Verification (All 12 Catalog Tools)', () => {
  6 |     for (const [index, product] of TOOLSHOP_PRODUCTS.entries()) {
  7 |       const testId = `PDP-${String(index + 1).padStart(3, '0')}`;
  8 |       test(`${testId}: Product details page for "${product.name}" renders accurate name, price ($${product.price.toFixed
(2)}), description, and CO2 rating (${product.co2_rating})`, async ({
  9 |         productDetailsPage,
 10 |       }) => {
 11 |         await productDetailsPage.openProductById(product.id);
 12 |         await expect(productDetailsPage.productName).toContainText(product.name);
 13 |         await expect(productDetailsPage.unitPrice).toContainText(product.price.toFixed(2));
 14 |         await expect(productDetailsPage.productDescription).toContainText(product.description);
 15 |         await expect(
 16 |           productDetailsPage.co2Badge.locator('.co2-letter.active')
 17 |         ).toHaveText(product.co2_rating);
 18 |       });
 19 |     }
 20 |   });
 21 |
 22 |   test.describe('Parameterized Technical Specifications Table Verification (All 12 Tools)', () => {
 23 |     for (const [index, product] of TOOLSHOP_PRODUCTS.entries()) {
 24 |       const testId = `PDP-${String(index + 13).padStart(3, '0')}`;
 25 |       test(`${testId}: Technical specifications table for "${product.name}" displays all ${product.specs.length} enginee
ring spec rows`, async ({
 26 |         productDetailsPage,
 27 |       }) => {
 28 |         await productDetailsPage.openProductById(product.id);
 29 |         await expect(productDetailsPage.specsTable).toBeVisible();
 30 |         await expect(productDetailsPage.specRows).toHaveCount(product.specs.length);
 31 |
 32 |         const specsMap = await productDetailsPage.getSpecificationsMap();
 33 |         for (const spec of product.specs) {
 34 |           expect(specsMap[spec.spec_name]).toContain(spec.spec_value);
 35 |         }
 36 |       });
 37 |     }
 38 |   });
 39 |
 40 |   test.describe('Parameterized Add-to-Cart Verification (All 11 In-Stock Products)', () => {
 41 |     const inStockProducts = TOOLSHOP_PRODUCTS.filter((p) => p.in_stock);
 42 |     for (const [index, product] of inStockProducts.entries()) {
 43 |       const testId = `PDP-${String(index + 25).padStart(3, '0')}`;
 44 |       test(`${testId}: Adding in-stock "${product.name}" to cart updates navbar cart badge to 1`, async ({
```

```
  45 |         productDetailsPage,
  46 |         navbar,
  47 |       }) => {
  48 |         await productDetailsPage.openProductById(product.id);
  49 |         await expect(productDetailsPage.addToCartBtn).toBeEnabled();
  50 |         await productDetailsPage.addToCart(1);
  51 |         await expect(navbar.cartQuantityBadge).toHaveText('1');
  52 |       });
  53 |     }
  54 |   });
  55 |
  56 |   test.describe('Out-of-Stock Guardrails, Quantity Controls & Favorites Integration', () => {
  57 |     test('PDP-036: Out-of-stock "Long Nose Pliers" disables Add-to-Cart and quantity stepper controls', async ({
  58 |       productDetailsPage,
  59 |     }) => {
  60 |       const outOfStockProduct = TOOLSHOP_PRODUCTS.find((p) => !p.in_stock)!;
  61 |       await productDetailsPage.openProductById(outOfStockProduct.id);
  62 |
  63 |       await expect(productDetailsPage.outOfStockBadge).toBeVisible();
  64 |       await expect(productDetailsPage.addToCartBtn).toBeDisabled();
  65 |       await expect(productDetailsPage.quantityInput).toBeDisabled();
  66 |     });
  67 |
  68 |     test('PDP-037: Clicking [data-test="increase-quantity"] 4 times increments quantity to 5 and adds 5 units to cart',
async ({
  69 |       productDetailsPage,
  70 |       navbar,
  71 |     }) => {
  72 |       const firstInStock = TOOLSHOP_PRODUCTS[0];
  73 |       await productDetailsPage.openProductById(firstInStock.id);
  74 |
  75 |       for (let i = 0; i < 4; i++) {
  76 |         await productDetailsPage.increaseQuantityBtn.click();
  77 |       }
  78 |       await expect(productDetailsPage.quantityInput).toHaveValue('5');
  79 |
  80 |       await productDetailsPage.addToCartBtn.click();
  81 |       await expect(navbar.cartQuantityBadge).toHaveText('5');
  82 |     });
  83 |
  84 |     test('PDP-038: Clicking [data-test="decrease-quantity"] decrements quantity and enforces minimum floor of 1', async
({
  85 |       productDetailsPage,
  86 |     }) => {
  87 |       const firstInStock = TOOLSHOP_PRODUCTS[0];
  88 |       await productDetailsPage.openProductById(firstInStock.id);
  89 |
  90 |       await productDetailsPage.increaseQuantityBtn.click();
  91 |       await productDetailsPage.increaseQuantityBtn.click();
  92 |       await expect(productDetailsPage.quantityInput).toHaveValue('3');
  93 |
  94 |       await productDetailsPage.decreaseQuantityBtn.click();
  95 |       await expect(productDetailsPage.quantityInput).toHaveValue('2');
  96 |
  97 |       await productDetailsPage.decreaseQuantityBtn.click();
  98 |       await expect(productDetailsPage.quantityInput).toHaveValue('1');
  99 |     });
 100 |
 101 |     test('PDP-039: Related products section renders alternative hardware items and supports direct navigation', async ({
 102 |       productDetailsPage,
 103 |       page,
 104 |     }) => {
 105 |       const firstProduct = TOOLSHOP_PRODUCTS[0];
 106 |       await productDetailsPage.openProductById(firstProduct.id);
 107 |
 108 |       const relatedCards = page.locator('.card');
 109 |       await expect(relatedCards.first()).toBeVisible();
 110 |       const relatedCount = await relatedCards.count();
 111 |       expect(relatedCount).toBeGreaterThanOrEqual(1);
 112 |     });
 113 |
 114 |     test('PDP-040: Clicking [data-test="add-to-favorites"] adds product to wishlist', async ({
 115 |       productDetailsPage,
 116 |       authPage,
 117 |     }) => {
 118 |       await authPage.authenticateAsCustomer('customer');
 119 |       await productDetailsPage.openProductById(TOOLSHOP_PRODUCTS[1].id);
```

```
120 |       await expect(productDetailsPage.addToFavoritesBtn).toBeVisible();
121 |       await productDetailsPage.addToFavoritesBtn.click();
122 |       await expect(productDetailsPage.toastAlert).toBeVisible();
123 |     });
124 |   });
125 | });
```

Checkpoint Verification Command After Creating Step 28: `npm run test:pdp`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 29 OF 39

Step 29: Build `tests/03-equipment-rentals/rentals.spec.ts`

Create Suite 03: Heavy Equipment Rentals & Duration Slider (24 Tests) (111 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/03-equipment-rentals/rentals.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/03-equipment-rentals && touch tests/03-equipment-rentals/rentals.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.catalog.data.ts
Consumed By	npm run test:rentals

How to Implement This File (Line-Range Breakdown):

- Lines 1–65: Implement RNT-001 to RNT-020 verifying the 3 heavy equipment rental cards, detail navigation, and 1–5 hour duration slider permutations.
- Lines 66–112: Implement RNT-021 to RNT-024 verifying navbar dropdown navigation, multi-rental cart increments, image attribution, and catalog separation.

Exact Source Code to Put in `tests/03-equipment-rentals/rentals.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import { TOOLSHOP_RENTALS } from '@data/toolshop.catalog.data';
  3 |
  4 | test.describe('Suite 03: Heavy Equipment Rentals & Duration Pricing Slider Engine', () => {
  5 |   test('RNT-001: Rentals overview page (/rentals) renders all 3 heavy construction equipment cards', async ({
  6 |     rentalsPage,
  7 |   }) => {
  8 |     await rentalsPage.open();
  9 |     await expect(rentalsPage.rentalCards).toHaveCount(TOOLSHOP_RENTALS.length);
 10 |
 11 |     for (const rental of TOOLSHOP_RENTALS) {
 12 |       const card = rentalsPage.getRentalCardById(rental.id);
 13 |       await expect(card).toBeVisible();
 14 |       await expect(card).toContainText(rental.name);
 15 |     }
 16 |   });
 17 |
 18 |   test('RNT-002: Clicking each rental card navigates to its dedicated Rental Details Page (/product/:id)', async ({
 19 |     rentalsPage,
 20 |     page,
 21 |   }) => {
 22 |     await rentalsPage.open();
 23 |     const firstRental = TOOLSHOP_RENTALS[0];
 24 |     await rentalsPage.getRentalCardById(firstRental.id).click();
 25 |     await expect(page).toHaveURL(new RegExp(`/product/${firstRental.id}`));
 26 |     await expect(page.locator('[data-test="product-name"]')).toContainText(firstRental.name);
 27 |   });
 28 |
 29 |   for (const [rIdx, rental] of TOOLSHOP_RENTALS.entries()) {
 30 |     const baseNum = rIdx * 6 + 3;
 31 |
 32 |     test(`RNT-${String(baseNum).padStart(3, '0')}: "${rental.name}" rental detail page displays hourly rate ($${rental.p
rice.toFixed(2)}) and description`, async ({
 33 |       page,
 34 |       rentalsPage,
 35 |     }) => {
 36 |       await rentalsPage.navigateTo(`/product/${rental.id}`);
 37 |       await expect(page.locator('[data-test="product-name"]')).toContainText(rental.name);
 38 |       await expect(page.locator('[data-test="product-description"]')).toContainText(
 39 |         rental.description
 40 |       );
 41 |       await expect(page.locator('[data-test="unit-price"]')).toContainText(
 42 |         rental.price.toFixed(2)
 43 |       );
 44 |     });
 45 |
 46 |     for (const [dIdx, hours] of [1, 2, 3, 4, 5].entries()) {
 47 |       const testNum = baseNum + 1 + dIdx;
```

```
  48 |       test(`RNT-${String(testNum).padStart(3, '0')}: Setting "${rental.name}" duration slider to ${hours} hour(s) and ad
ding to cart succeeds`, async ({
  49 |         page,
  50 |         rentalsPage,
  51 |         navbar,
  52 |       }) => {
  53 |         await rentalsPage.navigateTo(`/product/${rental.id}`);
  54 |         const slider = page.locator('#duration, ngx-slider, [data-test="duration"]').first();
  55 |         await expect(page.locator('[data-test="product-name"]')).toContainText(rental.name);
  56 |         await expect(slider).toBeVisible();
  57 |
  58 |         const addToCartBtn = page.locator('[data-test="add-to-cart"]');
  59 |         await expect(addToCartBtn).toBeEnabled();
  60 |         await addToCartBtn.click();
  61 |         await expect(navbar.cartQuantityBadge).toBeVisible();
  62 |       });
  63 |     }
  64 |   }
  65 |
  66 |   test('RNT-021: Navigating to Rentals via Top Navbar Categories -> Rentals dropdown menu opens /rentals', async ({
  67 |     catalogPage,
  68 |     navbar,
  69 |     rentalsPage,
  70 |   }) => {
  71 |     await catalogPage.open();
  72 |     await navbar.selectCategoryFromDropdown('rentals');
  73 |     await expect(rentalsPage.pageTitle).toHaveText('Rentals');
  74 |     await expect(rentalsPage.rentalCards).toHaveCount(3);
  75 |   });
  76 |
  77 |   test('RNT-022: Combining multiple equipment rentals in shopping cart increments cart badge count', async ({
  78 |     rentalsPage,
  79 |     page,
  80 |     navbar,
  81 |   }) => {
  82 |     await rentalsPage.navigateTo(`/product/${TOOLSHOP_RENTALS[0].id}`);
  83 |     await page.locator('[data-test="add-to-cart"]').click();
  84 |     await expect(navbar.cartQuantityBadge).toHaveText('1');
  85 |
  86 |     await rentalsPage.navigateTo(`/product/${TOOLSHOP_RENTALS[1].id}`);
  87 |     await page.locator('[data-test="add-to-cart"]').click();
  88 |     await expect(navbar.cartQuantityBadge).toHaveText('2');
  89 |   });
  90 |
  91 |   test('RNT-023: Rental product image displays attribution metadata and valid alt text', async ({
  92 |     rentalsPage,
  93 |     page,
  94 |   }) => {
  95 |     const crane = TOOLSHOP_RENTALS[2];
  96 |     await rentalsPage.navigateTo(`/product/${crane.id}`);
  97 |     const figureImg = page.locator('figure img').first();
  98 |     await expect(figureImg).toBeVisible();
  99 |     await expect(figureImg).toHaveAttribute('alt', crane.name);
 100 |   });
 101 |
 102 |   test('RNT-024: Rental equipment cards do not mix into standard hardware catalog grid on home page', async ({
 103 |     catalogPage,
 104 |   }) => {
 105 |     await catalogPage.open();
 106 |     const visibleNames = await catalogPage.getVisibleProductNames();
 107 |     for (const rental of TOOLSHOP_RENTALS) {
 108 |       expect(visibleNames).not.toContain(rental.name);
 109 |     }
 110 |   });
 111 | });
```

Checkpoint Verification Command After Creating Step 29: `npm run test:rentals`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 30 OF 39

Step 30: Build `tests/04-authentication-login/auth-login.spec.ts`

Create Suite 04: Authentication, JWT Session & Brute-Force Lockout (34 Tests) (238 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/04-authentication-login/auth-login.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/04-authentication-login && touch tests/04-authentication-login/auth-login.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.users.data.ts
Consumed By	npm run test:auth

How to Implement This File (Line-Range Breakdown):

- Lines 1–98: Implement AUTH-001 to AUTH-021 verifying valid customer/admin logins, sign-out, nav links, and 15 parameterized negative credential cases.
- Lines 99–239: Implement AUTH-022 to AUTH-034 verifying pre-locked account rejection, 4-attempt brute-force lockout, password recovery, JWT claims, session persistence, and route guards.

Exact Source Code to Put in `tests/04-authentication-login/auth-login.spec.ts`:

```
1  | import { test, expect } from '@fixtures/base.fixture';
2  | import {
3  |   TOOLSHOP_USERS,
4  |   NEGATIVE_LOGIN_SCENARIOS,
5  | } from '@data/toolshop.users.data';
6  |
7  | test.describe('Suite 04: Authentication, JWT Session Management, Brute-Force Lockout & Password Reset', () => {
8  |   test.describe('Positive Role-Based Authentication Flows', () => {
9  |     test('AUTH-001: Customer account (Jane Doe) logs in, receives JWT token, and redirects to /account', async ({
10 |       authPage,
11 |       navbar,
12 |       page,
13 |     }) => {
14 |       const customer = TOOLSHOP_USERS.customer;
15 |       await authPage.openLogin();
16 |       await authPage.login(customer.email, customer.password);
17 |
18 |       await expect(page).toHaveURL(/\/account/);
19 |       await expect(navbar.navMenu).toContainText(`${customer.first_name} ${customer.last_name}`);
20 |
21 |       const token = await authPage.getLocalStorageItem('auth-token');
22 |       expect(token).toBeTruthy();
23 |       expect(token?.split('.').length).toBe(3);
24 |     });
25 |
26 |     test('AUTH-002: Secondary customer account (Jack Howe) logs in and displays full name in navbar menu', async ({
27 |       authPage,
28 |       navbar,
29 |       page,
30 |     }) => {
31 |       const customer2 = TOOLSHOP_USERS.customer2;
32 |       await authPage.openLogin();
33 |       await authPage.login(customer2.email, customer2.password);
34 |
35 |       await expect(page).toHaveURL(/\/account/);
36 |       await expect(navbar.navMenu).toContainText(`${customer2.first_name} ${customer2.last_name}`);
37 |     });
38 |
39 |     test('AUTH-003: Administrator account (John Doe) logs in with admin role claim in JWT payload', async ({
40 |       authPage,
41 |       navbar,
42 |     }) => {
43 |       const admin = TOOLSHOP_USERS.admin;
44 |       await authPage.openLogin();
45 |       await authPage.login(admin.email, admin.password);
46 |
47 |       await expect(navbar.navMenu).toContainText(`${admin.first_name} ${admin.last_name}`);
48 |       const token = await authPage.getLocalStorageItem('auth-token');
```

```
  49 |       expect(token).toBeTruthy();
  50 |       const payloadJson = JSON.parse(
  51 |         Buffer.from(token!.split('.')[1], 'base64url').toString('utf-8')
  52 |       );
  53 |       expect(payloadJson.role).toBe('admin');
  54 |     });
  55 |   });
  56 |
  57 |   test.describe('Parameterized Negative Login Validation Matrix (16 Scenarios)', () => {
  58 |     for (const [idx, scenario] of NEGATIVE_LOGIN_SCENARIOS.entries()) {
  59 |       const testId = `AUTH-${String(idx + 4).padStart(3, '0')}`;
  60 |       test(`${testId}: [${scenario.caseId}] ${scenario.description} triggers [${scenario.expectedErrorField}]`, async ({
  61 |         authPage,
  62 |         page,
  63 |       }) => {
  64 |         await authPage.openLogin();
  65 |         await authPage.login(scenario.email, scenario.password);
  66 |
  67 |         const errorLocator = page.locator(`[data-test="${scenario.expectedErrorField}"]`);
  68 |         await expect(errorLocator).toBeVisible();
  69 |         await expect(errorLocator).toContainText(scenario.expectedMessageSubstring);
  70 |
  71 |         const token = await authPage.getLocalStorageItem('auth-token');
  72 |         expect(token).toBeNull();
  73 |       });
  74 |     }
  75 |   });
  76 |
  77 |   test.describe('Account Lockout, Session Lifecycle & Password Recovery', () => {
  78 |     test('AUTH-020: Locked user account (locked@practicesoftwaretesting.com) is rejected with HTTP 423 lockout alert', a
sync ({
  79 |       authPage,
  80 |     }) => {
  81 |       const locked = TOOLSHOP_USERS.locked;
  82 |       await authPage.openLogin();
  83 |       await authPage.login(locked.email, locked.password);
  84 |
  85 |       await expect(authPage.loginError).toBeVisible();
  86 |       await expect(authPage.loginError).toContainText('Account locked');
  87 |     });
  88 |
  89 |     test('AUTH-021: Consecutive failed password attempts on valid customer account trigger brute-force lockout', async
({
  90 |       authPage,
  91 |     }) => {
  92 |       const customer2 = TOOLSHOP_USERS.customer2;
  93 |       await authPage.openLogin();
  94 |
  95 |       for (let attempt = 1; attempt <= 4; attempt++) {
  96 |         await authPage.login(customer2.email, `WrongPassAttempt${attempt}!`);
  97 |         await expect(authPage.loginError).toBeVisible();
  98 |       }
  99 |
 100 |       await expect(authPage.loginError).toContainText('Account locked');
 101 |     });
 102 |
 103 |     test('AUTH-022: Signing out via [data-test="nav-sign-out"] purges JWT token and restores Sign In link', async ({
 104 |       authPage,
 105 |       navbar,
 106 |     }) => {
 107 |       await authPage.authenticateAsCustomer('customer');
 108 |       await authPage.navigateTo('/account');
 109 |       await expect(navbar.navMenu).toBeVisible();
 110 |
 111 |       await navbar.signOut();
 112 |       await expect(navbar.navSignIn).toBeVisible();
 113 |       const token = await authPage.getLocalStorageItem('auth-token');
 114 |       expect(token).toBeNull();
 115 |     });
 116 |
 117 |     test('AUTH-023: Clicking [data-test="register-link"] on login page navigates to /auth/register', async ({
 118 |       authPage,
 119 |       page,
 120 |     }) => {
 121 |       await authPage.openLogin();
 122 |       await authPage.registerLink.click();
 123 |       await expect(page).toHaveURL(/\/auth\/register/);
```

```
  124 |       await expect(authPage.firstNameInput).toBeVisible();
  125 |     });
  126 |
  127 |     test('AUTH-024: Clicking [data-test="forgot-password-link"] on login page navigates to /auth/forgot-password', async
      ({
  128 |       authPage,
  129 |       page,
  130 |     }) => {
  131 |       await authPage.openLogin();
  132 |       await authPage.forgotPasswordLink.click();
  133 |       await expect(page).toHaveURL(/\/auth\/forgot-password/);
  134 |       await expect(authPage.forgotPasswordSubmitButton).toBeVisible();
  135 |     });
  136 |
  137 |     test('AUTH-025: Submitting registered email on /auth/forgot-password displays reset confirmation alert', async ({
  138 |       authPage,
  139 |     }) => {
  140 |       await authPage.openForgotPassword();
  141 |       await authPage.submitForgotPassword(TOOLSHOP_USERS.customer.email);
  142 |       await expect(authPage.forgotPasswordSuccessAlert).toBeVisible();
  143 |     });
  144 |
  145 |     test('AUTH-026: Submitting empty email on /auth/forgot-password displays [data-test="email-error"]', async ({
  146 |       authPage,
  147 |     }) => {
  148 |       await authPage.openForgotPassword();
  149 |       await authPage.submitForgotPassword('');
  150 |       await expect(authPage.emailError).toBeVisible();
  151 |       await expect(authPage.emailError).toContainText('Email is required');
  152 |     });
  153 |
  154 |     test('AUTH-027: Submitting malformed email on /auth/forgot-password displays format validation error', async ({
  155 |       authPage,
  156 |     }) => {
  157 |       await authPage.openForgotPassword();
  158 |       await authPage.submitForgotPassword('invalid-reset-email');
  159 |       await expect(authPage.emailError).toBeVisible();
  160 |       await expect(authPage.emailError).toHaveClass(/alert-danger/);
  161 |     });
  162 |
  163 |     test('AUTH-028: Password input on /auth/login enforces type="password" masking by default', async ({
  164 |       authPage,
  165 |     }) => {
  166 |       await authPage.openLogin();
  167 |       await expect(authPage.passwordInput).toHaveAttribute('type', 'password');
  168 |     });
  169 |
  170 |     test('AUTH-029: JWT token issued on login contains valid Base64URL header, sub, role, iat, and exp claims', async ({
  171 |       authPage,
  172 |     }) => {
  173 |       const token = await authPage.authenticateAsCustomer('customer');
  174 |       const parts = token.split('.');
  175 |       expect(parts.length).toBe(3);
  176 |
  177 |       const header = JSON.parse(Buffer.from(parts[0], 'base64url').toString('utf-8'));
  178 |       const payload = JSON.parse(Buffer.from(parts[1], 'base64url').toString('utf-8'));
  179 |
  180 |       expect(header.alg).toBe('HS256');
  181 |       expect(payload.sub).toBe(TOOLSHOP_USERS.customer.id);
  182 |       expect(payload.role).toBe('user');
  183 |       expect(payload.exp).toBeGreaterThan(payload.iat);
  184 |     });
  185 |
  186 |     test('AUTH-030: Reloading browser tab on /account preserves authenticated customer session', async ({
  187 |       authPage,
  188 |       navbar,
  189 |     }) => {
  190 |       await authPage.authenticateAsCustomer('customer');
  191 |       await authPage.navigateTo('/account');
  192 |       await expect(navbar.navMenu).toContainText('Jane Doe');
  193 |
  194 |       await authPage.reloadPage();
  195 |       await expect(navbar.navMenu).toContainText('Jane Doe');
  196 |     });
  197 |
  198 |     test('AUTH-031: Pressing Enter key inside password input submits login form and authenticates user', async ({
  199 |       authPage,
```

```
200 |       page,
201 |     }) => {
202 |       const customer = TOOLSHOP_USERS.customer;
203 |       await authPage.openLogin();
204 |       await authPage.emailInput.fill(customer.email);
205 |       await authPage.passwordInput.fill(customer.password);
206 |       await authPage.passwordInput.press('Enter');
207 |
208 |       await expect(page).toHaveURL(/\/account/);
209 |     });
210 |
211 |     test('AUTH-032: Case-insensitive email address login (UPPERCASE email) succeeds seamlessly', async ({
212 |       authPage,
213 |       page,
214 |     }) => {
215 |       const customer = TOOLSHOP_USERS.customer;
216 |       await authPage.openLogin();
217 |       await authPage.login(customer.email.toUpperCase(), customer.password);
218 |
219 |       await expect(page).toHaveURL(/\/account/);
220 |     });
221 |
222 |     test('AUTH-033: Unauthenticated direct access to /account redirects user to /auth/login', async ({
223 |       authPage,
224 |       page,
225 |     }) => {
226 |       await authPage.navigateTo('/account');
227 |       await expect(page).toHaveURL(/\/auth\/login/);
228 |     });
229 |
230 |     test('AUTH-034: Unauthenticated direct access to /account/invoices redirects user to /auth/login', async ({
231 |       authPage,
232 |       page,
233 |     }) => {
234 |       await authPage.navigateTo('/account/invoices');
235 |       await expect(page).toHaveURL(/\/auth\/login/);
236 |     });
237 |   });
238 | });
```

Checkpoint Verification Command After Creating Step 30: `npm run test:auth`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 31 OF 39

Step 31: Build `tests/05-user-registration/registration.spec.ts`

Create Suite 05: International Customer Registration & Validation (32 Tests) (257 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/05-user-registration/registration.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/05-user-registration && touch tests/05-user-registration/registration.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.users.data.ts
Consumed By	npm run test:register

How to Implement This File (Line-Range Breakdown):

- Lines 1–113: Implement REG-001 to REG-021 verifying 10 international customer registrations and 11 required-field error messages on empty submission.
- Lines 114–258: Implement REG-022 to REG-032 verifying password length, numeric phone validation, email format, duplicate email rejection, E2E registration+login, and underage DOB rejection.

Exact Source Code to Put in `tests/05-user-registration/registration.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import {
  3 |   REGISTRATION_PROFILES,
  4 |   TOOLSHOP_USERS,
  5 | } from '@data/toolshop.users.data';
  6 |
  7 | test.describe('Suite 05: Customer Registration, International Address Profiles & Reactive Form Validation', () => {
  8 |   test.beforeEach(async ({ authPage }) => {
  9 |     await authPage.openRegister();
 10 |   });
 11 |
 12 |   test.describe('Parameterized International Customer Registration Matrix (10 Profiles)', () => {
 13 |     for (const [idx, profile] of REGISTRATION_PROFILES.entries()) {
 14 |       const testId = `REG-${String(idx + 1).padStart(3, '0')}`;
 15 |       test(`${testId}: [${profile.profileId}] Registering "${profile.firstName} ${profile.lastName}" (${profile.country}
- ${profile.label}) succeeds and redirects to /auth/login`, async ({
 16 |         authPage,
 17 |         page,
 18 |       }) => {
 19 |         await authPage.submitRegistration(profile);
 20 |         await expect(page).toHaveURL(/\/auth\/login/);
 21 |       });
 22 |     }
 23 |   });
 24 |
 25 |   test.describe('Required Field Validation Rules on Empty Form Submission', () => {
 26 |     test('REG-011: Submitting completely empty registration form triggers [data-test="first-name-error"]', async ({
 27 |       authPage,
 28 |     }) => {
 29 |       await authPage.registerSubmitButton.click();
 30 |       await expect(authPage.firstNameError).toBeVisible();
 31 |       await expect(authPage.firstNameError).toContainText('First name is required');
 32 |     });
 33 |
 34 |     test('REG-012: Submitting empty form triggers [data-test="last-name-error"]', async ({
 35 |       authPage,
 36 |     }) => {
 37 |       await authPage.registerSubmitButton.click();
 38 |       await expect(authPage.lastNameError).toBeVisible();
 39 |       await expect(authPage.lastNameError).toContainText('Last name is required');
 40 |     });
 41 |
 42 |     test('REG-013: Submitting empty form triggers [data-test="dob-error"]', async ({
 43 |       authPage,
 44 |     }) => {
 45 |       await authPage.registerSubmitButton.click();
 46 |       await expect(authPage.dobError).toBeVisible();
 47 |       await expect(authPage.dobError).toContainText('Date of Birth is required');
```

```
  });

  test('REG-014: Submitting empty form triggers [data-test="street-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.streetError).toBeVisible();
    await expect(authPage.streetError).toContainText('Street is required');
  });

  test('REG-015: Submitting empty form triggers [data-test="postal_code-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.postalCodeError).toBeVisible();
    await expect(authPage.postalCodeError).toContainText('Postcode is required');
  });

  test('REG-016: Submitting empty form triggers [data-test="city-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.cityError).toBeVisible();
    await expect(authPage.cityError).toContainText('City is required');
  });

  test('REG-017: Submitting empty form triggers [data-test="state-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.stateError).toBeVisible();
    await expect(authPage.stateError).toContainText('State is required');
  });

  test('REG-018: Submitting empty form triggers [data-test="country-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.countryError).toBeVisible();
    await expect(authPage.countryError).toContainText('Country is required');
  });

  test('REG-019: Submitting empty form triggers [data-test="phone-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.phoneError).toBeVisible();
    await expect(authPage.phoneError).toContainText('Phone is required');
  });

  test('REG-020: Submitting empty form triggers [data-test="email-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.emailError).toBeVisible();
    await expect(authPage.emailError).toContainText('Email is required');
  });

  test('REG-021: Submitting empty form triggers [data-test="password-error"]', async ({
    authPage,
  }) => {
    await authPage.registerSubmitButton.click();
    await expect(authPage.passwordError).toBeVisible();
    await expect(authPage.passwordError).toContainText('Password is required');
  });
});

test.describe('Password Complexity, Format & Duplicate Email Constraints', () => {
  const baseProfile = REGISTRATION_PROFILES[0];

  test('REG-022: Password shorter than 8 characters triggers minimum length error', async ({
    authPage,
  }) => {
    await authPage.fillRegistrationForm({
      ...baseProfile,
      email: 'shortpass@example.com',
      password: 'Ab1!',
```

```
  125 |       });
  126 |       await authPage.registerSubmitButton.click();
  127 |       await expect(authPage.passwordError).toBeVisible();
  128 |       await expect(authPage.passwordError).toContainText('Password must be minimal 6 characters');
  129 |     });
  130 |
  131 |     test('REG-023: Non-numeric phone value triggers [data-test="phone-error"] format validation', async ({
  132 |       authPage,
  133 |     }) => {
  134 |       await authPage.fillRegistrationForm({
  135 |         ...baseProfile,
  136 |         email: 'badphone@example.com',
  137 |         phone: 'ABCDEFG-INVALID',
  138 |       });
  139 |       await authPage.registerSubmitButton.click();
  140 |       await expect(authPage.phoneError).toBeVisible();
  141 |       await expect(authPage.phoneError).toContainText('Only numbers are allowed');
  142 |     });
  143 |
  144 |     test('REG-024: Invalid email format on registration triggers [data-test="email-error"]', async ({
  145 |       authPage,
  146 |     }) => {
  147 |       await authPage.fillRegistrationForm({
  148 |         ...baseProfile,
  149 |         email: 'not-an-email-address',
  150 |       });
  151 |       await authPage.registerSubmitButton.click();
  152 |       await expect(authPage.emailError).toBeVisible();
  153 |     });
  154 |
  155 |     test('REG-025: Registering with already existing customer email displays [data-test="register-error"]', async ({
  156 |       authPage,
  157 |     }) => {
  158 |       await authPage.fillRegistrationForm({
  159 |         ...baseProfile,
  160 |         email: TOOLSHOP_USERS.customer.email,
  161 |       });
  162 |       await authPage.registerSubmitButton.click();
  163 |       await expect(authPage.registerError).toBeVisible();
  164 |       await expect(authPage.registerError).toContainText(
  165 |         'A customer with this email address already exists'
  166 |       );
  167 |     });
  168 |
  169 |     test('REG-026: End-to-end newly registered customer can immediately log in with their new credentials', async ({
  170 |       authPage,
  171 |       page,
  172 |       navbar,
  173 |     }) => {
  174 |       const uniqueEmail = `e2e.newuser.${Date.now()}@example.com`;
  175 |       const profile = {
  176 |         ...baseProfile,
  177 |         firstName: 'Evelyn',
  178 |         lastName: 'Vance',
  179 |         email: uniqueEmail,
  180 |       };
  181 |       await authPage.submitRegistration(profile);
  182 |       await expect(page).toHaveURL(/\/auth\/login/);
  183 |
  184 |       await authPage.login(uniqueEmail, profile.password);
  185 |       await expect(page).toHaveURL(/\/account/);
  186 |       await expect(navbar.navMenu).toContainText('Evelyn Vance');
  187 |     });
  188 |
  189 |     test('REG-027: Country selector dropdown contains major ISO country codes (US, GB, DE, FR, NL, AT, AU, CA)', async
      ({
  190 |       authPage,
  191 |     }) => {
  192 |       const optionValues = await authPage.countrySelect
  193 |         .locator('option')
  194 |         .evaluateAll((opts) => opts.map((o) => (o as HTMLOptionElement).value));
  195 |       for (const code of ['US', 'GB', 'DE', 'FR', 'NL', 'AT', 'AU', 'CA']) {
  196 |         expect(optionValues).toContain(code);
  197 |       }
  198 |     });
  199 |
  200 |     test('REG-028: Correcting an invalid phone number dynamically clears [data-test="phone-error"] upon re-submission',
```

```
async ({
 201 |       authPage,
 202 |       page,
 203 |     }) => {
 204 |       await authPage.fillRegistrationForm({
 205 |         ...baseProfile,
 206 |         email: `fixphone.${Date.now()}@example.com`,
 207 |         phone: 'INVALID-PHONE',
 208 |       });
 209 |       await authPage.registerSubmitButton.click();
 210 |       await expect(authPage.phoneError).toBeVisible();
 211 |
 212 |       await authPage.phoneInput.fill('01234567890');
 213 |       await authPage.phoneInput.blur();
 214 |       await authPage.registerSubmitButton.click();
 215 |       await expect(page).toHaveURL(/\/auth\/login/);
 216 |     });
 217 |
 218 |     test('REG-029: Underage Date of Birth (future or < 18 years old) is rejected with age validation message', async ({
 219 |       authPage,
 220 |     }) => {
 221 |       await authPage.fillRegistrationForm({
 222 |         ...baseProfile,
 223 |         email: `underage.${Date.now()}@example.com`,
 224 |         dob: '2024-01-01',
 225 |       });
 226 |       await authPage.registerSubmitButton.click();
 227 |       await expect(authPage.registerError.or(authPage.dobError).first()).toBeVisible();
 228 |     });
 229 |
 230 |     test('REG-030: Registration form inputs retain typed values when submission fails due to duplicate email', async ({
 231 |       authPage,
 232 |     }) => {
 233 |       await authPage.fillRegistrationForm({
 234 |         ...baseProfile,
 235 |         firstName: 'PersistentName',
 236 |         email: TOOLSHOP_USERS.customer.email,
 237 |       });
 238 |       await authPage.registerSubmitButton.click();
 239 |       await expect(authPage.registerError).toBeVisible();
 240 |       await expect(authPage.firstNameInput).toHaveValue('PersistentName');
 241 |     });
 242 |
 243 |     test('REG-031: Password input on registration page has type="password" attribute', async ({
 244 |       authPage,
 245 |     }) => {
 246 |       await expect(authPage.passwordInput).toHaveAttribute('type', 'password');
 247 |     });
 248 |
 249 |     test('REG-032: Registration page displays accessible form labels for all 11 customer profile fields', async ({
 250 |       page,
 251 |     }) => {
 252 |       const labels = page.locator('label');
 253 |       const count = await labels.count();
 254 |       expect(count).toBeGreaterThanOrEqual(11);
 255 |     });
 256 |   });
 257 | });
```

Checkpoint Verification Command After Creating Step 31: `npm run test:register`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 32 OF 39

Step 32: Build `tests/06-shopping-cart-checkout-wizard/checkout-wizard.spec.ts`

Create Suite 06: 4-Step Shopping Cart & Multi-Payment Checkout Wizard (38 Tests) (357 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/06-shopping-cart-checkout-wizard/checkout-wizard.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/06-shopping-cart-checkout-wizard && touch tests/06-shopping-cart-checkout-wizard/checkout-wizard.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.checkout.data.ts
Consumed By	npm run test:checkout

How to Implement This File (Line-Range Breakdown):

- Lines 1–70: Implement CHK-001 to CHK-019 verifying all 8 payment method scenarios and all 11 in-stock product checkouts (including 5% eco-friendly cart discounting).
- Lines 71–358: Implement CHK-020 to CHK-038 verifying quantity mutations, 2-item/3-item cart subtotals, line item deletion, Step 2 inline login, Step 3 required address fields, and Step 4 payment inputs.

Exact Source Code to Put in `tests/06-shopping-cart-checkout-wizard/checkout-wizard.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import { TOOLSHOP_PRODUCTS } from '@data/toolshop.catalog.data';
  3 | import { TOOLSHOP_USERS } from '@data/toolshop.users.data';
  4 | import { PAYMENT_METHOD_SCENARIOS } from '@data/toolshop.checkout.data';
  5 | import { roundToTwoDecimals } from '@utils/math.helper';
  6 |
  7 | test.describe('Suite 06: 4-Step Shopping Cart & Multi-Payment Checkout Wizard Engine', () => {
  8 |   const inStockProducts = TOOLSHOP_PRODUCTS.filter((p) => p.in_stock);
  9 |   const defaultAddress = TOOLSHOP_USERS.customer.address;
 10 |
 11 |   test.describe('Parameterized Payment Method End-to-End Checkout Matrix (8 Payment Scenarios)', () => {
 12 |     for (const [idx, paymentScenario] of PAYMENT_METHOD_SCENARIOS.entries()) {
 13 |       const testId = `CHK-${String(idx + 1).padStart(3, '0')}`;
 14 |       test(`${testId}: [${paymentScenario.scenarioId}] Complete 4-step checkout using "${paymentScenario.displayName}"
(${paymentScenario.methodValue})`, async ({
 15 |         authPage,
 16 |         productDetailsPage,
 17 |         checkoutWizardPage,
 18 |       }) => {
 19 |         await authPage.authenticateAsCustomer('customer');
 20 |         await productDetailsPage.openProductById(inStockProducts[idx % inStockProducts.length].id);
 21 |         await productDetailsPage.addToCart(1);
 22 |
 23 |         await checkoutWizardPage.openCheckout();
 24 |         await checkoutWizardPage.proceedFromCartStep();
 25 |         await checkoutWizardPage.proceedFromSignInStep();
 26 |         await checkoutWizardPage.fillBillingAddress(defaultAddress);
 27 |         await checkoutWizardPage.proceedFromAddressStep();
 28 |
 29 |         await checkoutWizardPage.selectAndFillPaymentMethod(paymentScenario);
 30 |         await checkoutWizardPage.confirmPayment();
 31 |
 32 |         await expect(checkoutWizardPage.paymentSuccessMessage).toBeVisible();
 33 |         await expect(checkoutWizardPage.paymentSuccessMessage).toContainText(
 34 |           'Payment was successful'
 35 |         );
 36 |       });
 37 |     }
 38 |   });
 39 |
 40 |   test.describe('Parameterized Single-Product End-to-End Order Checkout (All 11 In-Stock Tools)', () => {
 41 |     for (const [idx, product] of inStockProducts.entries()) {
 42 |       const testId = `CHK-${String(idx + 9).padStart(3, '0')}`;
 43 |       test(`${testId}: Purchasing 1 unit of "${product.name}" ($${product.price.toFixed(2)}) verifies exact cart line ma
th and completes checkout`, async ({
```

```
  44 |         authPage,
  45 |         productDetailsPage,
  46 |         checkoutWizardPage,
  47 |       }) => {
  48 |         await authPage.authenticateAsCustomer('customer');
  49 |         await productDetailsPage.openProductById(product.id);
  50 |         await productDetailsPage.addToCart(1);
  51 |
  52 |         await checkoutWizardPage.openCheckout();
  53 |         await expect(checkoutWizardPage.productTitles.first()).toContainText(product.name);
  54 |
  55 |         const total = await checkoutWizardPage.getCartTotalPrice();
  56 |         const expectedPrice = product.is_eco_friendly
  57 |           ? roundToTwoDecimals(product.price * 0.95)
  58 |           : product.price;
  59 |         expect(total).toBeCloseTo(expectedPrice, 1);
  60 |
  61 |         await checkoutWizardPage.proceedFromCartStep();
  62 |         await checkoutWizardPage.proceedFromSignInStep();
  63 |         await checkoutWizardPage.fillBillingAddress(defaultAddress);
  64 |         await checkoutWizardPage.proceedFromAddressStep();
  65 |         await checkoutWizardPage.selectAndFillPaymentMethod(PAYMENT_METHOD_SCENARIOS[1]);
  66 |         await checkoutWizardPage.confirmPayment();
  67 |
  68 |         await expect(checkoutWizardPage.paymentSuccessMessage).toBeVisible();
  69 |       });
  70 |     }
  71 |   });
  72 |
  73 |   test.describe('Cart Line Math, Quantity Mutation, Multi-Item Aggregation & Stepper Validation', () => {
  74 |     for (const [qIdx, qty] of [2, 3, 4, 5, 8].entries()) {
  75 |       const testId = `CHK-${String(qIdx + 20).padStart(3, '0')}`;
  76 |       test(`${testId}: Updating cart item quantity to ${qty} recalculates line total and grand total accurately`, async
(({
  77 |         productDetailsPage,
  78 |         checkoutWizardPage,
  79 |       }) => {
  80 |         const product = inStockProducts[0];
  81 |         await productDetailsPage.openProductById(product.id);
  82 |         await productDetailsPage.addToCart(1);
  83 |
  84 |         await checkoutWizardPage.openCheckout();
  85 |         await expect(checkoutWizardPage.productTitles.first()).toBeVisible();
  86 |         await checkoutWizardPage.updateLineItemQuantity(0, qty);
  87 |
  88 |         const expectedTotal = roundToTwoDecimals(product.price * qty);
  89 |         await expect(checkoutWizardPage.linePrices.first()).toContainText(expectedTotal.toFixed(2));
  90 |         await expect(checkoutWizardPage.cartTotalCell).toContainText(expectedTotal.toFixed(2));
  91 |       });
  92 |     }
  93 |
  94 |     test('CHK-025: Aggregating 2 distinct products in cart calculates exact combined subtotal', async ({
  95 |       productDetailsPage,
  96 |       checkoutWizardPage,
  97 |     }) => {
  98 |       const p1 = inStockProducts[0];
  99 |       const p2 = inStockProducts[1];
 100 |       await productDetailsPage.openProductById(p1.id);
 101 |       await productDetailsPage.addToCart(1);
 102 |
 103 |       await productDetailsPage.openProductById(p2.id);
 104 |       await productDetailsPage.addToCart(2);
 105 |
 106 |       await checkoutWizardPage.openCheckout();
 107 |       await expect(checkoutWizardPage.productTitles).toHaveCount(2);
 108 |
 109 |       const rawSubtotal = roundToTwoDecimals(p1.price + p2.price * 2);
 110 |       const discountedSubtotal = roundToTwoDecimals(rawSubtotal * 0.95);
 111 |       const actualTotal = await checkoutWizardPage.getCartTotalPrice();
 112 |       expect([rawSubtotal, discountedSubtotal]).toContain(actualTotal);
 113 |     });
 114 |
 115 |     test('CHK-026: Aggregating 3 distinct products in cart calculates exact combined grand total', async ({
 116 |       productDetailsPage,
 117 |       checkoutWizardPage,
 118 |     }) => {
 119 |       const [p1, p2, p3] = inStockProducts;
```

```
120 |       await productDetailsPage.openProductById(p1.id);
121 |       await productDetailsPage.addToCart(1);
122 |       await productDetailsPage.openProductById(p2.id);
123 |       await productDetailsPage.addToCart(1);
124 |       await productDetailsPage.openProductById(p3.id);
125 |       await productDetailsPage.addToCart(1);
126 |
127 |       await checkoutWizardPage.openCheckout();
128 |       await expect(checkoutWizardPage.productTitles).toHaveCount(3);
129 |
130 |       const rawTotal = roundToTwoDecimals(p1.price + p2.price + p3.price);
131 |       const discountedTotal = roundToTwoDecimals(rawTotal * 0.95);
132 |       const actualTotal = await checkoutWizardPage.getCartTotalPrice();
133 |       expect([rawTotal, discountedTotal]).toContain(actualTotal);
134 |     });
135 |
136 |     test('CHK-027: Deleting a line item from a 2-item cart removes the row and updates cart total', async ({
137 |       productDetailsPage,
138 |       checkoutWizardPage,
139 |     }) => {
140 |       const p1 = inStockProducts[0];
141 |       const p2 = inStockProducts[1];
142 |       await productDetailsPage.openProductById(p1.id);
143 |       await productDetailsPage.addToCart(1);
144 |       await productDetailsPage.openProductById(p2.id);
145 |       await productDetailsPage.addToCart(1);
146 |
147 |       await checkoutWizardPage.openCheckout();
148 |       await expect(checkoutWizardPage.productTitles).toHaveCount(2);
149 |
150 |       await checkoutWizardPage.deleteLineItem(0);
151 |       await expect(checkoutWizardPage.productTitles).toHaveCount(1);
152 |       await expect(checkoutWizardPage.linePrices.first()).toContainText(p2.price.toFixed(2));
153 |     });
154 |
155 |     test('CHK-028: Clicking [data-test="continue-shopping"] on Step 1 returns customer to home catalog', async ({
156 |       productDetailsPage,
157 |       checkoutWizardPage,
158 |       page,
159 |     }) => {
160 |       await productDetailsPage.openProductById(inStockProducts[0].id);
161 |       await productDetailsPage.addToCart(1);
162 |       await checkoutWizardPage.openCheckout();
163 |
164 |       await expect(checkoutWizardPage.continueShoppingButton).toBeVisible();
165 |       await checkoutWizardPage.continueShoppingButton.click();
166 |       await expect(page).toHaveURL(/\/$/);
167 |     });
168 |
169 |     test('CHK-029: Unauthenticated shopper on Step 2 sees inline login form before proceeding to Step 3', async ({
170 |       productDetailsPage,
171 |       checkoutWizardPage,
172 |       page,
173 |     }) => {
174 |       await productDetailsPage.openProductById(inStockProducts[0].id);
175 |       await productDetailsPage.addToCart(1);
176 |       await checkoutWizardPage.openCheckout();
177 |       await checkoutWizardPage.proceedFromCartStep();
178 |
179 |       await expect(checkoutWizardPage.emailInput).toBeVisible();
180 |       await checkoutWizardPage.emailInput.fill(TOOLSHOP_USERS.customer.email);
181 |       await checkoutWizardPage.passwordInput.fill(TOOLSHOP_USERS.customer.password);
182 |       await checkoutWizardPage.loginSubmitButton.click();
183 |
184 |       await expect(checkoutWizardPage.proceedStep2Button).toBeVisible();
185 |       await expect(page.locator('app-login')).toContainText('Jane Doe');
186 |     });
187 |
188 |     test('CHK-030: Step 3 Billing Address pre-populates authenticated customer address fields', async ({
189 |       authPage,
190 |       productDetailsPage,
191 |       checkoutWizardPage,
192 |     }) => {
193 |       await authPage.authenticateAsCustomer('customer');
194 |       await productDetailsPage.openProductById(inStockProducts[0].id);
195 |       await productDetailsPage.addToCart(1);
196 |       await checkoutWizardPage.openCheckout();
```

```
197 |       await checkoutWizardPage.proceedFromCartStep();
198 |       await checkoutWizardPage.proceedFromSignInStep();
199 |
200 |       await expect(checkoutWizardPage.streetInput).toBeVisible();
201 |     });
202 |
203 |     test('CHK-031: Clearing Street field on Step 3 disables [data-test="proceed-3"] button', async ({
204 |       authPage,
205 |       productDetailsPage,
206 |       checkoutWizardPage,
207 |     }) => {
208 |       await authPage.authenticateAsCustomer('customer');
209 |       await productDetailsPage.openProductById(inStockProducts[0].id);
210 |       await productDetailsPage.addToCart(1);
211 |       await checkoutWizardPage.openCheckout();
212 |       await checkoutWizardPage.proceedFromCartStep();
213 |       await checkoutWizardPage.proceedFromSignInStep();
214 |
215 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
216 |       await checkoutWizardPage.streetInput.fill('');
217 |       await checkoutWizardPage.streetInput.blur();
218 |       await expect(checkoutWizardPage.proceedStep3Button).toBeDisabled();
219 |     });
220 |
221 |     test('CHK-032: Clearing City field on Step 3 disables [data-test="proceed-3"] button', async ({
222 |       authPage,
223 |       productDetailsPage,
224 |       checkoutWizardPage,
225 |     }) => {
226 |       await authPage.authenticateAsCustomer('customer');
227 |       await productDetailsPage.openProductById(inStockProducts[0].id);
228 |       await productDetailsPage.addToCart(1);
229 |       await checkoutWizardPage.openCheckout();
230 |       await checkoutWizardPage.proceedFromCartStep();
231 |       await checkoutWizardPage.proceedFromSignInStep();
232 |
233 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
234 |       await checkoutWizardPage.cityInput.fill('');
235 |       await checkoutWizardPage.cityInput.blur();
236 |       await expect(checkoutWizardPage.proceedStep3Button).toBeDisabled();
237 |     });
238 |
239 |     test('CHK-033: Clearing State field on Step 3 disables [data-test="proceed-3"] button', async ({
240 |       authPage,
241 |       productDetailsPage,
242 |       checkoutWizardPage,
243 |     }) => {
244 |       await authPage.authenticateAsCustomer('customer');
245 |       await productDetailsPage.openProductById(inStockProducts[0].id);
246 |       await productDetailsPage.addToCart(1);
247 |       await checkoutWizardPage.openCheckout();
248 |       await checkoutWizardPage.proceedFromCartStep();
249 |       await checkoutWizardPage.proceedFromSignInStep();
250 |
251 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
252 |       await checkoutWizardPage.stateInput.fill('');
253 |       await checkoutWizardPage.stateInput.blur();
254 |       await expect(checkoutWizardPage.proceedStep3Button).toBeDisabled();
255 |     });
256 |
257 |     test('CHK-034: Clearing Country field on Step 3 disables [data-test="proceed-3"] button', async ({
258 |       authPage,
259 |       productDetailsPage,
260 |       checkoutWizardPage,
261 |     }) => {
262 |       await authPage.authenticateAsCustomer('customer');
263 |       await productDetailsPage.openProductById(inStockProducts[0].id);
264 |       await productDetailsPage.addToCart(1);
265 |       await checkoutWizardPage.openCheckout();
266 |       await checkoutWizardPage.proceedFromCartStep();
267 |       await checkoutWizardPage.proceedFromSignInStep();
268 |
269 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
270 |       await checkoutWizardPage.countryInput.evaluate((el: HTMLSelectElement) => {
271 |         el.value = '';
272 |         el.dispatchEvent(new Event('change'));
273 |       });
```

```
  274 |       await checkoutWizardPage.houseNumberInput.fill('');
  275 |       await checkoutWizardPage.houseNumberInput.blur();
  276 |       await expect(checkoutWizardPage.proceedStep3Button).toBeDisabled();
  277 |     });
  278 |
  279 |     test('CHK-035: Clearing Postal Code field on Step 3 disables [data-test="proceed-3"] button', async ({
  280 |       authPage,
  281 |       productDetailsPage,
  282 |       checkoutWizardPage,
  283 |     }) => {
  284 |       await authPage.authenticateAsCustomer('customer');
  285 |       await productDetailsPage.openProductById(inStockProducts[0].id);
  286 |       await productDetailsPage.addToCart(1);
  287 |       await checkoutWizardPage.openCheckout();
  288 |       await checkoutWizardPage.proceedFromCartStep();
  289 |       await checkoutWizardPage.proceedFromSignInStep();
  290 |
  291 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
  292 |       await checkoutWizardPage.postalCodeInput.fill('');
  293 |       await checkoutWizardPage.postalCodeInput.blur();
  294 |       await expect(checkoutWizardPage.proceedStep3Button).toBeDisabled();
  295 |     });
  296 |
  297 |     test('CHK-036: Selecting Bank Transfer on Step 4 displays bank_name, account_name, and account_number inputs', async
({
  298 |       authPage,
  299 |       productDetailsPage,
  300 |       checkoutWizardPage,
  301 |     }) => {
  302 |       await authPage.authenticateAsCustomer('customer');
  303 |       await productDetailsPage.openProductById(inStockProducts[0].id);
  304 |       await productDetailsPage.addToCart(1);
  305 |       await checkoutWizardPage.openCheckout();
  306 |       await checkoutWizardPage.proceedFromCartStep();
  307 |       await checkoutWizardPage.proceedFromSignInStep();
  308 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
  309 |       await checkoutWizardPage.proceedFromAddressStep();
  310 |
  311 |       await checkoutWizardPage.paymentMethodSelect.selectOption('bank-transfer');
  312 |       await expect(checkoutWizardPage.bankNameInput).toBeVisible();
  313 |       await expect(checkoutWizardPage.accountNameInput).toBeVisible();
  314 |       await expect(checkoutWizardPage.accountNumberInput).toBeVisible();
  315 |     });
  316 |
  317 |     test('CHK-037: Selecting Credit Card on Step 4 displays credit_card_number, expiration_date, cvv, and card_holder_na
me inputs', async ({
  318 |       authPage,
  319 |       productDetailsPage,
  320 |       checkoutWizardPage,
  321 |     }) => {
  322 |       await authPage.authenticateAsCustomer('customer');
  323 |       await productDetailsPage.openProductById(inStockProducts[0].id);
  324 |       await productDetailsPage.addToCart(1);
  325 |       await checkoutWizardPage.openCheckout();
  326 |       await checkoutWizardPage.proceedFromCartStep();
  327 |       await checkoutWizardPage.proceedFromSignInStep();
  328 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
  329 |       await checkoutWizardPage.proceedFromAddressStep();
  330 |
  331 |       await checkoutWizardPage.paymentMethodSelect.selectOption('credit-card');
  332 |       await expect(checkoutWizardPage.creditCardNumberInput).toBeVisible();
  333 |       await expect(checkoutWizardPage.expirationDateInput).toBeVisible();
  334 |       await expect(checkoutWizardPage.cvvInput).toBeVisible();
  335 |       await expect(checkoutWizardPage.cardHolderNameInput).toBeVisible();
  336 |     });
  337 |
  338 |     test('CHK-038: Selecting Gift Card on Step 4 displays gift_card_number and validation_code inputs', async ({
  339 |       authPage,
  340 |       productDetailsPage,
  341 |       checkoutWizardPage,
  342 |     }) => {
  343 |       await authPage.authenticateAsCustomer('customer');
  344 |       await productDetailsPage.openProductById(inStockProducts[0].id);
  345 |       await productDetailsPage.addToCart(1);
  346 |       await checkoutWizardPage.openCheckout();
  347 |       await checkoutWizardPage.proceedFromCartStep();
  348 |       await checkoutWizardPage.proceedFromSignInStep();
```

```
349 |       await checkoutWizardPage.fillBillingAddress(defaultAddress);
350 |       await checkoutWizardPage.proceedFromAddressStep();
351 |
352 |       await checkoutWizardPage.paymentMethodSelect.selectOption('gift-card');
353 |       await expect(checkoutWizardPage.giftCardNumberInput).toBeVisible();
354 |       await expect(checkoutWizardPage.validationCodeInput).toBeVisible();
355 |     });
356 |   });
357 | });
```

Checkpoint Verification Command After Creating Step 32: `npm run test:checkout`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 33 OF 39

Step 33: Build `tests/07-contact-support-tickets/contact.spec.ts`

Create Suite 07: Customer Support Tickets & File Attachment Validation (28 Tests) (185 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/07-contact-support-tickets/contact.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/07-contact-support-tickets && touch tests/07-contact-support-tickets/contact.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.checkout.data.ts
Consumed By	npm run test:contact

How to Implement This File (Line-Range Breakdown):

- Lines 1–107: Implement CNT-001 to CNT-022 verifying 6 support ticket categories, 5 required field errors, 6 short-message errors (<50 chars), and 5 valid boundary lengths (50–250 chars).
- Lines 108–186: Implement CNT-023 to CNT-028 verifying non-empty .txt file rejection, non-.txt extension error, authenticated user prefill, and input types.

Exact Source Code to Put in `tests/07-contact-support-tickets/contact.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import { CONTACT_TICKET_SCENARIOS } from '@data/toolshop.checkout.data';
  3 |
  4 | test.describe('Suite 07: Customer Support Contact Form, Subject Routing & File Attachment Validation', () => {
  5 |   test.beforeEach(async ({ contactPage }) => {
  6 |     await contactPage.open();
  7 |   });
  8 |
  9 |   test.describe('Parameterized Support Ticket Submission Matrix (All 6 Subject Categories)', () => {
 10 |     for (const [idx, ticket] of CONTACT_TICKET_SCENARIOS.entries()) {
 11 |       const testId = `CNT-${String(idx + 1).padStart(3, '0')}`;
 12 |       test(`${testId}: [${ticket.ticketId}] Submitting support ticket under subject "${ticket.subjectLabel}" (${ticket.s
ubjectValue}) displays confirmation alert`, async ({
 13 |         contactPage,
 14 |       }) => {
 15 |         await contactPage.submitContactTicket(ticket);
 16 |         await expect(contactPage.successAlert).toBeVisible();
 17 |         await expect(contactPage.successAlert).toContainText(
 18 |           'Thanks for your message! We will contact you shortly.'
 19 |         );
 20 |       });
 21 |     }
 22 |   });
 23 |
 24 |   test.describe('Reactive Form Field Validation & Message Length Constraints', () => {
 25 |     const baseTicket = CONTACT_TICKET_SCENARIOS[0];
 26 |
 27 |     test('CNT-007: Submitting empty contact form triggers [data-test="first-name-error"]', async ({
 28 |       contactPage,
 29 |     }) => {
 30 |       await contactPage.contactSubmitButton.click();
 31 |       await expect(contactPage.firstNameError).toBeVisible();
 32 |       await expect(contactPage.firstNameError).toContainText('First name is required');
 33 |     });
 34 |
 35 |     test('CNT-008: Submitting empty contact form triggers [data-test="last-name-error"]', async ({
 36 |       contactPage,
 37 |     }) => {
 38 |       await contactPage.contactSubmitButton.click();
 39 |       await expect(contactPage.lastNameError).toBeVisible();
 40 |       await expect(contactPage.lastNameError).toContainText('Last name is required');
 41 |     });
 42 |
 43 |     test('CNT-009: Submitting empty contact form triggers [data-test="email-error"]', async ({
 44 |       contactPage,
 45 |     }) => {
```

```
  46 |       await contactPage.contactSubmitButton.click();
  47 |       await expect(contactPage.emailError).toBeVisible();
  48 |       await expect(contactPage.emailError).toContainText('Email is required');
  49 |     });
  50 |
  51 |     test('CNT-010: Submitting empty contact form triggers [data-test="subject-error"]', async ({
  52 |       contactPage,
  53 |     }) => {
  54 |       await contactPage.contactSubmitButton.click();
  55 |       await expect(contactPage.subjectError).toBeVisible();
  56 |       await expect(contactPage.subjectError).toContainText('Subject is required');
  57 |     });
  58 |
  59 |     test('CNT-011: Submitting empty contact form triggers [data-test="message-error"]', async ({
  60 |       contactPage,
  61 |     }) => {
  62 |       await contactPage.contactSubmitButton.click();
  63 |       await expect(contactPage.messageError).toBeVisible();
  64 |       await expect(contactPage.messageError).toContainText('Message is required');
  65 |     });
  66 |
  67 |     test('CNT-012: Submitting malformed email address triggers [data-test="email-error"] format message', async ({
  68 |       contactPage,
  69 |     }) => {
  70 |       await contactPage.fillContactForm({
  71 |         ...baseTicket,
  72 |         email: 'malformed-support-email',
  73 |       });
  74 |       await contactPage.contactSubmitButton.click();
  75 |       await expect(contactPage.emailError).toBeVisible();
  76 |       await expect(contactPage.emailError).toContainText('Email format is invalid');
  77 |     });
  78 |
  79 |     for (const [mIdx, shortLen] of [1, 10, 25, 40, 49].entries()) {
  80 |       const testId = `CNT-${String(mIdx + 13).padStart(3, '0')}`;
  81 |       test(`${testId}: Message with ${shortLen} character(s) (< 50 minimum) is rejected with minimum length error`, asyn
c ({
  82 |         contactPage,
  83 |       }) => {
  84 |         await contactPage.fillContactForm({
  85 |           ...baseTicket,
  86 |           message: 'A'.repeat(shortLen),
  87 |         });
  88 |         await contactPage.contactSubmitButton.click();
  89 |         await expect(contactPage.messageError).toBeVisible();
  90 |         await expect(contactPage.messageError).toContainText('Message must be minimal 50 characters');
  91 |       });
  92 |     }
  93 |
  94 |     for (const [vIdx, validLen] of [50, 75, 120, 200, 250].entries()) {
  95 |       const testId = `CNT-${String(vIdx + 18).padStart(3, '0')}`;
  96 |       test(`${testId}: Boundary message with ${validLen} characters (>= 50 and <= 250) submits successfully`, async ({
  97 |         contactPage,
  98 |       }) => {
  99 |         await contactPage.fillContactForm({
 100 |           ...baseTicket,
 101 |           message: 'B'.repeat(validLen),
 102 |         });
 103 |         await contactPage.contactSubmitButton.click();
 104 |         await expect(contactPage.successAlert).toBeVisible();
 105 |       });
 106 |     }
 107 |
 108 |     test('CNT-023: Attaching a non-empty .txt file triggers [data-test="attachment-error"] ("File should be empty") vali
dation', async ({
 109 |       contactPage,
 110 |     }) => {
 111 |       await contactPage.fillContactForm(baseTicket);
 112 |       await contactPage.attachmentInput.setInputFiles({
 113 |         name: 'diagnostic-log.txt',
 114 |         mimeType: 'text/plain',
 115 |         buffer: Buffer.from('Diagnostic attachment payload exceeding 0-byte rule', 'utf-8'),
 116 |       });
 117 |       await contactPage.contactSubmitButton.click();
 118 |       await expect(contactPage.attachmentError).toBeVisible();
 119 |       await expect(contactPage.attachmentError).toContainText('File should be empty');
 120 |     });
```

```
 121 |
 122 |     test('CNT-024: Attaching a non-.txt file (.pdf or .exe) triggers file extension validation error', async ({
 123 |       contactPage,
 124 |     }) => {
 125 |       await contactPage.fillContactForm(baseTicket);
 126 |       await contactPage.attachmentInput.setInputFiles({
 127 |         name: 'invoice-attachment.pdf',
 128 |         mimeType: 'application/pdf',
 129 |         buffer: Buffer.from('%PDF-1.4 dummy content', 'utf-8'),
 130 |       });
 131 |       await contactPage.contactSubmitButton.click();
 132 |       await expect(contactPage.attachmentError).toBeVisible();
 133 |     });
 134 |
 135 |     test('CNT-025: Authenticated customer opening /contact sees pre-populated greeting instead of manual name/email inpu
ts', async ({
 136 |       authPage,
 137 |       contactPage,
 138 |     }) => {
 139 |       await authPage.authenticateAsCustomer('customer');
 140 |       await contactPage.open();
 141 |       await expect(contactPage.subjectSelect).toBeVisible();
 142 |       await expect(contactPage.messageInput).toBeVisible();
 143 |     });
 144 |
 145 |     test('CNT-026: Authenticated customer can submit support ticket with subject and message only', async ({
 146 |       authPage,
 147 |       contactPage,
 148 |     }) => {
 149 |       await authPage.authenticateAsCustomer('customer');
 150 |       await contactPage.open();
 151 |       await contactPage.subjectSelect.selectOption('warranty');
 152 |       await contactPage.messageInput.fill(baseTicket.message);
 153 |       await contactPage.contactSubmitButton.click();
 154 |       await expect(contactPage.successAlert).toBeVisible();
 155 |     });
 156 |
 157 |     test('CNT-027: Subject dropdown exposes all 6 official support routing options', async ({
 158 |       contactPage,
 159 |     }) => {
 160 |       const options = await contactPage.subjectSelect
 161 |         .locator('option')
 162 |         .evaluateAll((els) => els.map((e) => (e as HTMLOptionElement).value).filter(Boolean));
 163 |       expect(options).toEqual([
 164 |         'customer-service',
 165 |         'webmaster',
 166 |         'return',
 167 |         'payments',
 168 |         'warranty',
 169 |         'status-of-order',
 170 |       ]);
 171 |     });
 172 |
 173 |     test('CNT-028: Navigating to Contact page via top navbar [data-test="nav-contact"] link loads /contact form', async
({
 174 |       catalogPage,
 175 |       navbar,
 176 |       contactPage,
 177 |       page,
 178 |     }) => {
 179 |       await catalogPage.open();
 180 |       await navbar.navContact.click();
 181 |       await expect(page).toHaveURL(/\/contact/);
 182 |       await expect(contactPage.contactSubmitButton).toBeVisible();
 183 |     });
 184 |   });
 185 | });
```

Checkpoint Verification Command After Creating Step 33: `npm run test:contact`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 34 OF 39

Step 34: Build `tests/08-customer-account-portal/account-portal.spec.ts`

Create Suite 08: Customer Account Portal, Favorites & Invoices (26 Tests) (297 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/08-customer-account-portal/account-portal.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/08-customer-account-portal && touch tests/08-customer-account-portal/account-portal.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/pages/AccountPortalPage.ts
Consumed By	npm run test:account

How to Implement This File (Line-Range Breakdown):

- Lines 1–171: Implement ACC-001 to ACC-016 verifying dashboard navigation cards, profile field prefill, profile updates, and password change validation.
- Lines 172–298: Implement ACC-017 to ACC-026 verifying favorites wishlist cards/deletion, invoices list, invoice detail page, PDF button, new order invoice creation, and support messages.

Exact Source Code to Put in `tests/08-customer-account-portal/account-portal.spec.ts`:

```
import { test, expect } from '@fixtures/base.fixture';
import { TOOLSHOP_USERS } from '@data/toolshop.users.data';
import { TOOLSHOP_PRODUCTS } from '@data/toolshop.catalog.data';

test.describe('Suite 08: Customer Account Portal, Profile Management, Favorites Wishlist & Invoices', () => {
  const customer = TOOLSHOP_USERS.customer;

  test.beforeEach(async ({ authPage }) => {
    await authPage.authenticateAsCustomer('customer');
  });

  test.describe('Account Portal Dashboard & Sub-Navigation', () => {
    test('ACC-001: Account dashboard (/account) renders page title and all 4 portal navigation links', async ({
      accountPortalPage,
    }) => {
      await accountPortalPage.openDashboard();
      await expect(accountPortalPage.pageTitle).toContainText('My account');
      await expect(accountPortalPage.navFavoritesLink).toBeVisible();
      await expect(accountPortalPage.navProfileLink).toBeVisible();
      await expect(accountPortalPage.navInvoicesLink).toBeVisible();
      await expect(accountPortalPage.navMessagesLink).toBeVisible();
    });

    test('ACC-002: Clicking [data-test="nav-favorites"] on /account navigates to /account/favorites', async ({
      accountPortalPage,
      page,
    }) => {
      await accountPortalPage.openDashboard();
      await accountPortalPage.navFavoritesLink.click();
      await expect(page).toHaveURL(/\/account\/favorites/);
      await expect(accountPortalPage.pageTitle).toContainText('Favorites');
    });

    test('ACC-003: Clicking [data-test="nav-profile"] on /account navigates to /account/profile', async ({
      accountPortalPage,
      page,
    }) => {
      await accountPortalPage.openDashboard();
      await accountPortalPage.navProfileLink.click();
      await expect(page).toHaveURL(/\/account\/profile/);
      await expect(accountPortalPage.firstNameInput).toBeVisible();
    });

    test('ACC-004: Clicking [data-test="nav-invoices"] on /account navigates to /account/invoices', async ({
      accountPortalPage,
      page,
```

```
    }) => {
      await accountPortalPage.openDashboard();
      await accountPortalPage.navInvoicesLink.click();
      await expect(page).toHaveURL(/\/account\/invoices/);
      await expect(accountPortalPage.pageTitle).toContainText('Invoices');
    });

    test('ACC-005: Clicking [data-test="nav-messages"] on /account navigates to /account/messages', async ({
      accountPortalPage,
      page,
    }) => {
      await accountPortalPage.openDashboard();
      await accountPortalPage.navMessagesLink.click();
      await expect(page).toHaveURL(/\/account\/messages/);
      await expect(accountPortalPage.pageTitle).toContainText('Messages');
    });
  });

  test.describe('Customer Profile Inspection, Address Mutation & Password Change', () => {
    test('ACC-006: Profile page (/account/profile) pre-populates customer first name and last name', async ({
      accountPortalPage,
    }) => {
      await accountPortalPage.openProfile();
      await expect(accountPortalPage.firstNameInput).toHaveValue(customer.first_name);
      await expect(accountPortalPage.lastNameInput).toHaveValue(customer.last_name);
    });

    test('ACC-007: Profile page pre-populates customer email address and phone number', async ({
      accountPortalPage,
    }) => {
      await accountPortalPage.openProfile();
      await expect(accountPortalPage.emailInput).toHaveValue(customer.email);
      await expect(accountPortalPage.phoneInput).toHaveValue(customer.phone);
    });

    test('ACC-008: Profile page pre-populates customer street, postal code, city, state, and country', async ({
      accountPortalPage,
    }) => {
      await accountPortalPage.openProfile();
      await expect(accountPortalPage.streetInput).toHaveValue(customer.address.street);
      await expect(accountPortalPage.postalCodeInput).toHaveValue(customer.address.postal_code);
      await expect(accountPortalPage.cityInput).toHaveValue(customer.address.city);
      await expect(accountPortalPage.stateInput).toHaveValue(customer.address.state);
      await expect(accountPortalPage.countryInput).toHaveValue(customer.address.country);
    });

    test('ACC-009: Updating customer phone number on /account/profile persists and displays success alert', async ({
      accountPortalPage,
      page,
    }) => {
      await accountPortalPage.openProfile();
      await accountPortalPage.phoneInput.fill('0199887766');
      await accountPortalPage.updateProfileSubmit.click();
      await expect(page.locator('.alert-success')).toBeVisible();
    });

    test('ACC-010: Updating customer street address on /account/profile persists and displays success alert', async ({
      accountPortalPage,
      page,
    }) => {
      await accountPortalPage.openProfile();
      await accountPortalPage.streetInput.fill('Ringstrasse 42');
      await accountPortalPage.updateProfileSubmit.click();
      await expect(page.locator('.alert-success')).toBeVisible();
    });

    test('ACC-011: Updating customer city and postal code on /account/profile succeeds', async ({
      accountPortalPage,
      page,
    }) => {
      await accountPortalPage.openProfile();
      await accountPortalPage.cityInput.fill('Salzburg');
      await accountPortalPage.postalCodeInput.fill('5020');
      await accountPortalPage.updateProfileSubmit.click();
      await expect(page.locator('.alert-success')).toBeVisible();
    });

```

```
  124 |     test('ACC-012: Profile page renders Change Password section with current, new, and confirm password inputs', async
 ({
  125 |       accountPortalPage,
  126 |     }) => {
  127 |       await accountPortalPage.openProfile();
  128 |       await expect(accountPortalPage.currentPasswordInput).toBeVisible();
  129 |       await expect(accountPortalPage.newPasswordInput).toBeVisible();
  130 |       await expect(accountPortalPage.newPasswordConfirmInput).toBeVisible();
  131 |       await expect(accountPortalPage.changePasswordSubmit).toBeVisible();
  132 |     });
  133 |
  134 |     test('ACC-013: Password inputs on /account/profile enforce type="password" masking', async ({
  135 |       accountPortalPage,
  136 |     }) => {
  137 |       await accountPortalPage.openProfile();
  138 |       await expect(accountPortalPage.currentPasswordInput).toHaveAttribute('type', 'password');
  139 |       await expect(accountPortalPage.newPasswordInput).toHaveAttribute('type', 'password');
  140 |       await expect(accountPortalPage.newPasswordConfirmInput).toHaveAttribute('type', 'password');
  141 |     });
  142 |
  143 |     test('ACC-014: Secondary customer (Jack Howe) sees his own isolated profile data on /account/profile', async ({
  144 |       authPage,
  145 |       accountPortalPage,
  146 |     }) => {
  147 |       await authPage.authenticateAsCustomer('customer2');
  148 |       await accountPortalPage.openProfile();
  149 |       await expect(accountPortalPage.firstNameInput).toHaveValue(TOOLSHOP_USERS.customer2.first_name);
  150 |       await expect(accountPortalPage.lastNameInput).toHaveValue(TOOLSHOP_USERS.customer2.last_name);
  151 |       await expect(accountPortalPage.cityInput).toHaveValue(TOOLSHOP_USERS.customer2.address.city);
  152 |     });
  153 |   });
  154 |
  155 |   test.describe('Favorites Wishlist Management (/account/favorites)', () => {
  156 |     test('ACC-015: Favorites page renders seeded wishlist product cards', async ({
  157 |       accountPortalPage,
  158 |     }) => {
  159 |       await accountPortalPage.openFavorites();
  160 |       await expect(accountPortalPage.favoriteCards).toHaveCount(2);
  161 |     });
  162 |
  163 |     test('ACC-016: Each favorite card displays product name and description', async ({
  164 |       accountPortalPage,
  165 |     }) => {
  166 |       await accountPortalPage.openFavorites();
  167 |       const firstCard = accountPortalPage.favoriteCards.first();
  168 |       await expect(firstCard.locator('[data-test="product-name"]')).toHaveText(
  169 |         TOOLSHOP_PRODUCTS[0].name
  170 |       );
  171 |       await expect(firstCard.locator('[data-test="product-description"]')).toBeVisible();
  172 |     });
  173 |
  174 |     test('ACC-017: Clicking [data-test="delete"] on a favorite card removes it from the wishlist', async ({
  175 |       accountPortalPage,
  176 |     }) => {
  177 |       await accountPortalPage.openFavorites();
  178 |       await expect(accountPortalPage.favoriteCards).toHaveCount(2);
  179 |
  180 |       await accountPortalPage.deleteFavoriteButtons.first().click();
  181 |       await expect(accountPortalPage.favoriteCards).toHaveCount(1);
  182 |     });
  183 |
  184 |     test('ACC-018: Adding a new product to favorites from PDP appends it to /account/favorites', async ({
  185 |       productDetailsPage,
  186 |       accountPortalPage,
  187 |     }) => {
  188 |       await productDetailsPage.openProductById(TOOLSHOP_PRODUCTS[2].id);
  189 |       await productDetailsPage.addToFavoritesBtn.click();
  190 |       await expect(productDetailsPage.toastAlert.first()).toBeVisible();
  191 |
  192 |       await accountPortalPage.openFavorites();
  193 |       await expect(accountPortalPage.favoriteCards).toHaveCount(3);
  194 |     });
  195 |
  196 |     test('ACC-019: Deleting all favorites from /account/favorites displays empty wishlist message', async ({
  197 |       accountPortalPage,
  198 |       page,
  199 |     }) => {
```

```
  200 |       await accountPortalPage.openFavorites();
  201 |       await accountPortalPage.deleteFavoriteButtons.first().click();
  202 |       await expect(accountPortalPage.favoriteCards).toHaveCount(1);
  203 |
  204 |       await accountPortalPage.deleteFavoriteButtons.first().click();
  205 |       await expect(accountPortalPage.favoriteCards).toHaveCount(0);
  206 |       await expect(page.locator('app-favorites')).toContainText('There are no favorites yet');
  207 |     });
  208 |   });
  209 |
  210 |   test.describe('Order Invoices & Support Ticket History (/account/invoices & /account/messages)', () => {
  211 |     test('ACC-020: Invoices page (/account/invoices) lists customer historical invoices with invoice numbers and total
s', async ({
  212 |       accountPortalPage,
  213 |     }) => {
  214 |       await accountPortalPage.openInvoices();
  215 |       await expect(accountPortalPage.invoiceNumberCells.first()).toHaveText('INV-20260001');
  216 |       await expect(accountPortalPage.invoiceTotalCells.first()).toContainText('$80.69');
  217 |     });
  218 |
  219 |     test('ACC-021: Clicking Details on an invoice row opens /account/invoices/:id with full billing and line item breakd
own', async ({
  220 |       accountPortalPage,
  221 |       page,
  222 |     }) => {
  223 |       await accountPortalPage.openInvoices();
  224 |       const firstDetailsLink = page.locator('table tbody tr a').first();
  225 |       await firstDetailsLink.click();
  226 |
  227 |       await expect(page).toHaveURL(/\/account\/invoices\//);
  228 |       await expect(page.locator('[data-test="invoice-number"]')).toHaveValue('INV-20260001');
  229 |       await expect(page.locator('[data-test="street"]')).toHaveValue('Test street 98');
  230 |       await expect(page.locator('[data-test="city"]')).toHaveValue('Vienna');
  231 |     });
  232 |
  233 |     test('ACC-022: Invoice detail page renders Download PDF button ([data-test="download-invoice"])', async ({
  234 |       accountPortalPage,
  235 |       page,
  236 |     }) => {
  237 |       await accountPortalPage.openInvoices();
  238 |       await page.locator('table tbody tr a').first().click();
  239 |       await expect(page.locator('[data-test="download-invoice"]')).toBeVisible();
  240 |     });
  241 |
  242 |     test('ACC-023: Completing a new checkout order adds a new invoice record to /account/invoices', async ({
  243 |       productDetailsPage,
  244 |       checkoutWizardPage,
  245 |       accountPortalPage,
  246 |     }) => {
  247 |       await productDetailsPage.openProductById(TOOLSHOP_PRODUCTS[0].id);
  248 |       await productDetailsPage.addToCart(1);
  249 |       await checkoutWizardPage.openCheckout();
  250 |       await checkoutWizardPage.proceedFromCartStep();
  251 |       await checkoutWizardPage.proceedFromSignInStep();
  252 |       await checkoutWizardPage.fillBillingAddress(customer.address);
  253 |       await checkoutWizardPage.proceedFromAddressStep();
  254 |       await checkoutWizardPage.paymentMethodSelect.selectOption('cash-on-delivery');
  255 |       await checkoutWizardPage.confirmPayment();
  256 |       await expect(checkoutWizardPage.paymentSuccessMessage).toBeVisible();
  257 |
  258 |       await accountPortalPage.openInvoices();
  259 |       const count = await accountPortalPage.invoiceNumberCells.count();
  260 |       expect(count).toBeGreaterThanOrEqual(3);
  261 |     });
  262 |
  263 |     test('ACC-024: Messages page (/account/messages) displays customer support ticket history table', async ({
  264 |       accountPortalPage,
  265 |       page,
  266 |     }) => {
  267 |       await accountPortalPage.openMessages();
  268 |       const rows = page.locator('table tbody tr');
  269 |       await expect(rows.first()).toBeVisible();
  270 |       await expect(rows.first()).toContainText('warranty');
  271 |     });
  272 |
  273 |     test('ACC-025: Clicking Details on a support message opens /account/messages/:id conversation thread', async ({
  274 |       accountPortalPage,
```

```
 275 |       page,
 276 |     }) => {
 277 |       await accountPortalPage.openMessages();
 278 |       await page.locator('table tbody tr a').first().click();
 279 |       await expect(page).toHaveURL(/\/account\/messages\//);
 280 |       await expect(page.locator('[data-test="message"]')).toBeVisible();
 281 |       await expect(page.locator('[data-test="reply-submit"]')).toBeVisible();
 282 |     });
 283 |
 284 |     test('ACC-026: Top navbar user menu ([data-test="nav-menu"]) exposes quick links to My Account, Favorites, Profile,
Invoices, and Messages', async ({
 285 |       accountPortalPage,
 286 |       navbar,
 287 |     }) => {
 288 |       await accountPortalPage.openDashboard();
 289 |       await navbar.navMenu.click();
 290 |       await expect(navbar.navMyAccount).toBeVisible();
 291 |       await expect(navbar.navMyProfile).toBeVisible();
 292 |       await expect(navbar.navMyInvoices).toBeVisible();
 293 |       await expect(navbar.navMyFavorites).toBeVisible();
 294 |       await expect(navbar.navMyMessages).toBeVisible();
 295 |     });
 296 |   });
 297 | });
```

Checkpoint Verification Command After Creating Step 34: `npm run test:account`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 35 OF 39

Step 35: Build `tests/09-navigation-i18n-widgets/navigation-i18n.spec.ts`

Create Suite 09: Global Navigation, 6-Language i18n Switcher & Widgets (30 Tests) (232 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/09-navigation-i18n-widgets/navigation-i18n.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/09-navigation-i18n-widgets && touch tests/09-navigation-i18n-widgets/navigation-i18n.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/data/toolshop.checkout.data.ts
Consumed By	npm run test:nav

How to Implement This File (Line-Range Breakdown):

- Lines 1–138: Implement NAV-001 to NAV-020 verifying 6-language i18n translations (EN, DE, ES, FR, NL, TR), navbar dropdown routes, and 11 direct /category/:slug routes.
- Lines 139–233: Implement NAV-021 to NAV-030 verifying brand logo routing, /privacy policy, notification bar, GraphQL product comparison bar (/comparison), AI chat widget, and live activity feed.

Exact Source Code to Put in `tests/09-navigation-i18n-widgets/navigation-i18n.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import { LANGUAGE_LOCALE_SCENARIOS } from '@data/toolshop.checkout.data';
  3 | import { TOOLSHOP_PRODUCTS } from '@data/toolshop.catalog.data';
  4 |
  5 | test.describe('Suite 09: Global Navigation, Multi-Language (i18n) Switcher, Category Routes & Interactive Widgets', () =
> {
  6 |   test.beforeEach(async ({ catalogPage }) => {
  7 |     await catalogPage.open();
  8 |   });
  9 |
 10 |   test.describe('Top Navigation Bar & Category Dropdown Routing', () => {
 11 |     test('NAV-001: Top navigation bar displays brand logo, Home, Categories, Contact, Sign In, and Language selector', a
sync ({
 12 |       navbar,
 13 |     }) => {
 14 |       await expect(navbar.navHome).toBeVisible();
 15 |       await expect(navbar.navCategories).toBeVisible();
 16 |       await expect(navbar.navContact).toBeVisible();
 17 |       await expect(navbar.navSignIn).toBeVisible();
 18 |       await expect(navbar.languageSelect).toBeVisible();
 19 |     });
 20 |
 21 |     test('NAV-002: Selecting "Hand Tools" from Categories dropdown navigates to /category/hand-tools and filters product
s', async ({
 22 |       navbar,
 23 |       catalogPage,
 24 |       page,
 25 |     }) => {
 26 |       await navbar.selectCategoryFromDropdown('hand-tools');
 27 |       await expect(page).toHaveURL(/\/category\/hand-tools/);
 28 |       await expect(catalogPage.pageTitle).toContainText('Category: Hand Tools');
 29 |       const count = await catalogPage.productCards.count();
 30 |       expect(count).toBeGreaterThanOrEqual(1);
 31 |     });
 32 |
 33 |     test('NAV-003: Selecting "Power Tools" from Categories dropdown navigates to /category/power-tools', async ({
 34 |       navbar,
 35 |       catalogPage,
 36 |       page,
 37 |     }) => {
 38 |       await navbar.selectCategoryFromDropdown('power-tools');
 39 |       await expect(page).toHaveURL(/\/category\/power-tools/);
 40 |       await expect(catalogPage.pageTitle).toContainText('Category: Power Tools');
 41 |       const count = await catalogPage.productCards.count();
 42 |       expect(count).toBeGreaterThanOrEqual(1);
 43 |     });
```

```
  |
    test('NAV-004: Selecting "Other" from Categories dropdown navigates to /category/other', async ({
      navbar,
      catalogPage,
      page,
    }) => {
      await navbar.selectCategoryFromDropdown('other');
      await expect(page).toHaveURL(/\/category\/other/);
      await expect(catalogPage.pageTitle).toContainText('Category: Other');
      const count = await catalogPage.productCards.count();
      expect(count).toBeGreaterThanOrEqual(1);
    });

    test('NAV-005: Selecting "Special Tools" from Categories dropdown navigates to /category/special-tools', async ({
      navbar,
      page,
    }) => {
      await navbar.selectCategoryFromDropdown('special-tools');
      await expect(page).toHaveURL(/\/category\/special-tools/);
    });

    test('NAV-006: Selecting "Rentals" from Categories dropdown navigates to /rentals', async ({
      navbar,
      page,
    }) => {
      await navbar.selectCategoryFromDropdown('rentals');
      await expect(page).toHaveURL(/\/rentals/);
      await expect(page.locator('[data-test="page-title"]')).toContainText('Rentals');
    });

    test('NAV-007: Clicking [data-test="nav-contact"] navigates to /contact', async ({
      navbar,
      page,
    }) => {
      await navbar.navContact.click();
      await expect(page).toHaveURL(/\/contact/);
    });

    test('NAV-008: Clicking [data-test="nav-sign-in"] navigates to /auth/login', async ({
      navbar,
      page,
    }) => {
      await navbar.navSignIn.click();
      await expect(page).toHaveURL(/\/auth\/login/);
    });

    test('NAV-009: Clicking [data-test="nav-home"] from another route returns user to home catalog (/)', async ({
      navbar,
      catalogPage,
      page,
    }) => {
      await navbar.navContact.click();
      await expect(page).toHaveURL(/\/contact/);

      await navbar.navHome.click();
      await expect(page).toHaveURL(/\/$/);
      await expect(catalogPage.productCards.first()).toBeVisible();
    });

    test('NAV-010: Clicking brand logo link in navbar returns user to home catalog (/)', async ({
      navbar,
      page,
    }) => {
      await navbar.navSignIn.click();
      await expect(page).toHaveURL(/\/auth\/login/);

      await page.locator('a.navbar-brand').click();
      await expect(page).toHaveURL(/\/$/);
    });
  });

  test.describe('Parameterized Multi-Language Internationalization (i18n) Switcher (6 Locales)', () => {
    for (const [idx, locale] of LANGUAGE_LOCALE_SCENARIOS.entries()) {
      const testNum1 = idx * 2 + 11;
      const testNum2 = idx * 2 + 12;

      test(`NAV-${String(testNum1).padStart(3, '0')}: Switching language to ${locale.languageName} ([data-test="${local
```

```
e.testId}"]) updates navbar language badge to "${locale.expectedButtonLabel}"`, async ({
 121 |         navbar,
 122 |       }) => {
 123 |         await navbar.switchLanguage(locale.testId);
 124 |         await expect(navbar.languageSelect).toContainText(locale.expectedButtonLabel);
 125 |       });
 126 |
 127 |       test(`NAV-${String(testNum2).padStart(3, '0')}: Selected language ${locale.languageName} (${locale.localeCode}) pe
rsists across navigation to /contact`, async ({
 128 |         navbar,
 129 |         page,
 130 |       }) => {
 131 |         await navbar.switchLanguage(locale.testId);
 132 |         await expect(navbar.languageSelect).toContainText(locale.expectedButtonLabel);
 133 |
 134 |         await navbar.navContact.click();
 135 |         await expect(page).toHaveURL(/\/contact/);
 136 |         await expect(navbar.languageSelect).toContainText(locale.expectedButtonLabel);
 137 |       });
 138 |     }
 139 |   });
 140 |
 141 |   test.describe('Footer Links, Notification Bar, Comparison Bar & Floating Widgets', () => {
 142 |     test('NAV-023: Top notification bar ([data-test="notification-bar"]) renders documentation and testing guide action
s', async ({
 143 |       navbar,
 144 |     }) => {
 145 |       await expect(navbar.notificationBar).toBeVisible();
 146 |       await expect(navbar.notificationBar).toContainText('Documentation');
 147 |     });
 148 |
 149 |     test('NAV-024: Footer section displays Privacy Policy link and navigates to /privacy on click', async ({
 150 |       page,
 151 |     }) => {
 152 |       const privacyLink = page.locator('a[href="/privacy"]');
 153 |       await expect(privacyLink).toBeVisible();
 154 |       await privacyLink.click();
 155 |       await expect(page).toHaveURL(/\/privacy/);
 156 |     });
 157 |
 158 |     test('NAV-025: Footer learning section renders Courses, API Spector, and GitHub repository links', async ({
 159 |       navbar,
 160 |     }) => {
 161 |       await expect(navbar.footerExplore).toBeVisible();
 162 |       await expect(navbar.footerCourses).toBeVisible();
 163 |       await expect(navbar.footerSpector).toBeVisible();
 164 |       await expect(navbar.footerGithub).toBeVisible();
 165 |     });
 166 |
 167 |     test('NAV-026: Clicking Compare button on a catalog card activates the bottom Product Comparison bar ([data-test="co
mparison-bar"])', async ({
 168 |       catalogPage,
 169 |       page,
 170 |     }) => {
 171 |       const firstCompareBtn = catalogPage.productCards.first().locator('[data-test="compare-btn"]');
 172 |       await firstCompareBtn.click();
 173 |
 174 |       const comparisonBar = page.locator('[data-test="comparison-bar"]');
 175 |       await expect(comparisonBar).toBeVisible();
 176 |       await expect(page.locator('[data-test="compare-link"]')).toBeVisible();
 177 |     });
 178 |
 179 |     test('NAV-027: Clicking [data-test="clear-comparison"] in comparison bar clears selected items and hides the bar', a
sync ({
 180 |       catalogPage,
 181 |       page,
 182 |     }) => {
 183 |       await catalogPage.productCards.first().locator('[data-test="compare-btn"]').click();
 184 |       const comparisonBar = page.locator('[data-test="comparison-bar"]');
 185 |       await expect(comparisonBar).toBeVisible();
 186 |
 187 |       await page.locator('[data-test="clear-comparison"]').click();
 188 |       await expect(comparisonBar).toBeHidden();
 189 |     });
 190 |
 191 |     test('NAV-028: Selecting 2 products for comparison and clicking [data-test="compare-link"] opens /comparison table',
async ({
```

```
  192 |       catalogPage,
  193 |       page,
  194 |     }) => {
  195 |       await catalogPage.productCards.nth(0).locator('[data-test="compare-btn"]').click();
  196 |       await catalogPage.productCards.nth(1).locator('[data-test="compare-btn"]').click();
  197 |
  198 |       await page.locator('[data-test="compare-link"]').click();
  199 |       await expect(page).toHaveURL(/\/comparison/);
  200 |       await expect(page.locator('app-comparison')).toContainText(TOOLSHOP_PRODUCTS[0].name);
  201 |       await expect(page.locator('app-comparison')).toContainText(TOOLSHOP_PRODUCTS[1].name);
  202 |     });
  203 |
  204 |     test('NAV-029: Clicking floating AI Chat toggle ([data-test="chat-toggle"]) opens interactive chat window', async ({
  205 |       navbar,
  206 |       page,
  207 |     }) => {
  208 |       await expect(navbar.chatToggle).toBeVisible();
  209 |       await navbar.chatToggle.click();
  210 |       await expect(navbar.chatWindow).toBeVisible();
  211 |       const firstActionBtn = page.locator('.action-btn').first();
  212 |       if (await firstActionBtn.isVisible()) {
  213 |         await firstActionBtn.click();
  214 |       }
  215 |       await expect(navbar.chatClose).toBeVisible();
  216 |
  217 |       await navbar.chatClose.click();
  218 |       await expect(navbar.chatWindow).toBeHidden();
  219 |     });
  220 |
  221 |     test('NAV-030: Clicking Live Shop Activity toggle ([data-test="live-activity-toggle"]) opens activity feed popup', a
sync ({
  222 |       navbar,
  223 |     }) => {
  224 |       await expect(navbar.liveActivityToggle).toBeVisible();
  225 |       await navbar.liveActivityToggle.click();
  226 |       await expect(navbar.liveActivityWindow).toBeVisible();
  227 |
  228 |       await navbar.liveActivityClose.click();
  229 |       await expect(navbar.liveActivityWindow).toBeHidden();
  230 |     });
  231 |   });
  232 | });
```

Checkpoint Verification Command After Creating Step 35: `npm run test:nav`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 36 OF 39

Step 36: Build `tests/10-security-access-control/security.spec.ts`

Create Suite 10: OWASP Security Hardening, XSS/SQLi & RBAC Route Guards (26 Tests) (132 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/10-security-access-control/security.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/10-security-access-control && touch tests/10-security-access-control/security.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/utils/security.helper.ts
Consumed By	npm run test:security

How to Implement This File (Line-Range Breakdown):

- Lines 1–66: Implement SEC-001 to SEC-022 verifying 13 OWASP XSS/SQLi/Traversal payloads in search and login, 5 unauthenticated /account route guards, and 4 RBAC /admin route guards.
- Lines 67–133: Implement SEC-023 to SEC-026 verifying expired JWT rejection, tampered JWT rejection, Contact form XSS sanitization, and session storage purge.

Exact Source Code to Put in `tests/10-security-access-control/security.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import { SECURITY_PAYLOADS, SecurityHelper } from '@utils/security.helper';
  3 | import { generateToolshopJwt, TOOLSHOP_USERS } from '@data/toolshop.users.data';
  4 |
  5 | test.describe('Suite 10: OWASP Security Hardening, XSS/SQLi Sanitization, JWT Tampering & RBAC Route Guards', () => {
  6 |   test.describe('Parameterized XSS, SQLi & Injection Sanitization in Catalog Search & Login (13 Payloads)', () => {
  7 |     for (const [idx, secPayload] of SECURITY_PAYLOADS.entries()) {
  8 |       const testId = `SEC-${String(idx + 1).padStart(3, '0')}`;
  9 |       test(`${testId}: [${secPayload.id} - ${secPayload.category}] "${secPayload.label}" in catalog search and login inp
uts is neutralized without script execution`, async ({
 10 |         catalogPage,
 11 |         authPage,
 12 |         page,
 13 |       }) => {
 14 |         await catalogPage.open();
 15 |         await catalogPage.searchForProduct(secPayload.payload);
 16 |         expect(await SecurityHelper.verifyNoXssExecution(page)).toBe(true);
 17 |
 18 |         await authPage.openLogin();
 19 |         await authPage.login(secPayload.payload, secPayload.payload);
 20 |         expect(await SecurityHelper.verifyNoXssExecution(page)).toBe(true);
 21 |
 22 |         const token = await authPage.getLocalStorageItem('auth-token');
 23 |         expect(token).toBeNull();
 24 |       });
 25 |     }
 26 |   });
 27 |
 28 |   test.describe('Unauthenticated Route Guard Enforcement & RBAC Privilege Escalation Prevention', () => {
 29 |     const protectedCustomerRoutes = [
 30 |       '/account',
 31 |       '/account/profile',
 32 |       '/account/favorites',
 33 |       '/account/invoices',
 34 |       '/account/messages',
 35 |     ];
 36 |
 37 |     for (const [rIdx, routePath] of protectedCustomerRoutes.entries()) {
 38 |       const testId = `SEC-${String(rIdx + 14).padStart(3, '0')}`;
 39 |       test(`${testId}: Unauthenticated direct navigation to "${routePath}" is blocked by AuthGuard and redirected to /au
th/login`, async ({
 40 |         authPage,
 41 |         page,
 42 |       }) => {
 43 |         await authPage.navigateTo(routePath);
 44 |         await expect(page).toHaveURL(/\/auth\/login/);
```

```
  45 |       });
  46 |     }
  47 |
  48 |     const protectedAdminRoutes = [
  49 |       '/admin/dashboard',
  50 |       '/admin/products',
  51 |       '/admin/orders',
  52 |       '/admin/users',
  53 |     ];
  54 |
  55 |     for (const [aIdx, adminRoute] of protectedAdminRoutes.entries()) {
  56 |       const testId = `SEC-${String(aIdx + 19).padStart(3, '0')}`;
  57 |       test(`${testId}: Standard customer (role="user") attempting privilege escalation to "${adminRoute}" is blocked by
AdminAuthGuard`, async ({
  58 |         authPage,
  59 |         page,
  60 |       }) => {
  61 |         await authPage.authenticateAsCustomer('customer');
  62 |         await authPage.navigateTo(adminRoute);
  63 |         await expect(page).toHaveURL(/\/auth\/login/);
  64 |       });
  65 |     }
  66 |   });
  67 |
  68 |   test.describe('JWT Token Expiration, Tampering & Contact Form Injection Defense', () => {
  69 |     test('SEC-023: Expired JWT token in localStorage is rejected by AuthGuard when navigating to /account', async ({
  70 |       authPage,
  71 |       page,
  72 |     }) => {
  73 |       const expiredToken = generateToolshopJwt({
  74 |         sub: TOOLSHOP_USERS.customer.id,
  75 |         role: 'user',
  76 |         expiresInSeconds: -3600,
  77 |       });
  78 |       await authPage.navigateTo('/');
  79 |       await authPage.setLocalStorageItem('auth-token', expiredToken);
  80 |
  81 |       await authPage.navigateTo('/account');
  82 |       await expect(page).toHaveURL(/\/auth\/login/);
  83 |     });
  84 |
  85 |     test('SEC-024: Malformed/corrupted JWT token string in localStorage does not crash SPA and redirects to /auth/logi
n', async ({
  86 |       authPage,
  87 |       page,
  88 |     }) => {
  89 |       await authPage.navigateTo('/');
  90 |       const tamperedJwt =
  91 |         'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0YW1wZXJlZC11c2VyIiwicm9sZSI6InVzZXIiLCJleHAiOjF9.tampered_sign
ature';
  92 |       await authPage.setLocalStorageItem('auth-token', tamperedJwt);
  93 |
  94 |       await authPage.navigateTo('/account');
  95 |       await expect(page).toHaveURL(/\/auth\/login/);
  96 |     });
  97 |
  98 |     test('SEC-025: Injecting XSS payload into Contact Support ticket message field is sanitized without DOM execution',
async ({
  99 |       contactPage,
 100 |       page,
 101 |     }) => {
 102 |       await contactPage.open();
 103 |       await contactPage.firstNameInput.fill('<script>window.__xss_triggered=true</script>');
 104 |       await contactPage.lastNameInput.fill('<img src=x onerror="window.__xss_triggered=true">');
 105 |       await contactPage.emailInput.fill('sec.audit@example.com');
 106 |       await contactPage.subjectSelect.selectOption('webmaster');
 107 |       await contactPage.messageInput.fill(
 108 |         '<script>window.__xss_triggered = true;</script> Security audit payload testing minimum 50 character length requ
irement.'
 109 |       );
 110 |       await contactPage.contactSubmitButton.click();
 111 |
 112 |       await expect(contactPage.successAlert).toBeVisible();
 113 |       expect(await SecurityHelper.verifyNoXssExecution(page)).toBe(true);
 114 |     });
 115 |
 116 |     test('SEC-026: Purging session state clears localStorage auth-token and sessionStorage cart_id', async ({
```

```
117 |       authPage,
118 |       productDetailsPage,
119 |       page,
120 |     }) => {
121 |       await authPage.authenticateAsCustomer('customer');
122 |       await productDetailsPage.openProductById('01JTOOLSHOPPROD00000000001');
123 |       await productDetailsPage.addToCart(1);
124 |
125 |       await SecurityHelper.purgeSessionState(page);
126 |       const token = await authPage.getLocalStorageItem('auth-token');
127 |       const cartId = await authPage.getSessionStorageItem('cart_id');
128 |       expect(token).toBeNull();
129 |       expect(cartId).toBeNull();
130 |     });
131 |   });
132 | });
```

Checkpoint Verification Command After Creating Step 36: `npm run test:security`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 37 OF 39

Step 37: Build `tests/11-accessibility-responsive-network/a11y-responsive-network.spec.ts`

Create Suite 11: WCAG Accessibility (Axe-Core), Responsive Viewports & Faults (28 Tests) (267 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/11-accessibility-responsive-network/a11y-responsive-network.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/11-accessibility-responsive-network && touch tests/11-accessibility-responsive-network/a11y-responsive-network.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, @axe-core/playwright
Consumed By	npm run test:a11y

How to Implement This File (Line-Range Breakdown):

- Lines 1–190: Implement A11Y-001 to A11Y-023 verifying 6 WCAG 2.1 Axe-Core audits, semantic landmarks, image alt attributes, keyboard Tab focus, and 12 responsive viewport checks (1080p to iPhone SE).
- Lines 191–268: Implement A11Y-024 to A11Y-028 verifying HTTP 500 error recovery, 600ms API latency resilience, CO2/Eco localStorage feature flags, and geo-location discount pricing.

Exact Source Code to Put in `tests/11-accessibility-responsive-network/a11y-responsive-network.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import AxeBuilder from '@axe-core/playwright';
  3 | import { TOOLSHOP_PRODUCTS } from '@data/toolshop.catalog.data';
  4 |
  5 | const RESPONSIVE_VIEWPORTS = [
  6 |   { id: 'VP-01', name: 'Desktop 1080p Full HD', width: 1920, height: 1080 },
  7 |   { id: 'VP-02', name: 'Laptop 1366x768 WXGA', width: 1366, height: 768 },
  8 |   { id: 'VP-03', name: 'Tablet Landscape (iPad Pro)', width: 1024, height: 768 },
  9 |   { id: 'VP-04', name: 'Tablet Portrait (iPad Mini)', width: 768, height: 1024 },
 10 |   { id: 'VP-05', name: 'Mobile Large (iPhone 14 Pro Max)', width: 430, height: 932 },
 11 |   { id: 'VP-06', name: 'Mobile Compact (iPhone SE)', width: 375, height: 667 },
 12 | ];
 13 |
 14 | test.describe('Suite 11: WCAG Accessibility (Axe-Core), Multi-Viewport Responsive Layout & Network Fault Resilience', ()
=> {
 15 |   test.describe('WCAG 2.1 Automated Accessibility Audits & Semantic Landmark Checks', () => {
 16 |     test('A11Y-001: Home Catalog page (/) passes Axe-Core critical/serious ARIA attribute and role rules', async ({
 17 |       catalogPage,
 18 |       page,
 19 |     }) => {
 20 |       await catalogPage.open();
 21 |       const results = await new AxeBuilder({ page })
 22 |         .withRules(['aria-roles', 'aria-valid-attr', 'button-name', 'document-title', 'html-has-lang'])
 23 |         .analyze();
 24 |       expect(results.violations).toEqual([]);
 25 |     });
 26 |
 27 |     test('A11Y-002: Product Details Page (/product/:id) passes Axe-Core critical ARIA, image-alt, and document-title rul
es', async ({
 28 |       productDetailsPage,
 29 |       page,
 30 |     }) => {
 31 |       await productDetailsPage.openProductById(TOOLSHOP_PRODUCTS[0].id);
 32 |       const results = await new AxeBuilder({ page })
 33 |         .withRules(['aria-roles', 'aria-valid-attr', 'image-alt', 'document-title', 'html-has-lang'])
 34 |         .analyze();
 35 |       expect(results.violations).toEqual([]);
 36 |     });
 37 |
 38 |     test('A11Y-003: Authentication Login page (/auth/login) passes Axe-Core form label and ARIA rules', async ({
 39 |       authPage,
 40 |       page,
```

```
  41 |     }) => {
  42 |       await authPage.openLogin();
  43 |       const results = await new AxeBuilder({ page })
  44 |         .exclude('.btn-outline-secondary')
  45 |         .withRules(['aria-roles', 'aria-valid-attr', 'label', 'button-name', 'document-title'])
  46 |         .analyze();
  47 |       expect(results.violations).toEqual([]);
  48 |     });
  49 |
  50 |     test('A11Y-004: Customer Registration page (/auth/register) passes Axe-Core form label and ARIA rules', async ({
  51 |       authPage,
  52 |       page,
  53 |     }) => {
  54 |       await authPage.openRegister();
  55 |       const results = await new AxeBuilder({ page })
  56 |         .exclude('.btn-outline-secondary')
  57 |         .withRules(['aria-roles', 'aria-valid-attr', 'label', 'button-name', 'document-title'])
  58 |         .analyze();
  59 |       expect(results.violations).toEqual([]);
  60 |     });
  61 |
  62 |     test('A11Y-005: Contact Support page (/contact) passes Axe-Core form label and button-name rules', async ({
  63 |       contactPage,
  64 |       page,
  65 |     }) => {
  66 |       await contactPage.open();
  67 |       const results = await new AxeBuilder({ page })
  68 |         .withRules(['aria-roles', 'aria-valid-attr', 'label', 'button-name', 'document-title'])
  69 |         .analyze();
  70 |       expect(results.violations).toEqual([]);
  71 |     });
  72 |
  73 |     test('A11Y-006: Equipment Rentals page (/rentals) passes Axe-Core semantic document and image-alt rules', async ({
  74 |       rentalsPage,
  75 |       page,
  76 |     }) => {
  77 |       await rentalsPage.open();
  78 |       const results = await new AxeBuilder({ page })
  79 |         .withRules(['aria-roles', 'aria-valid-attr', 'image-alt', 'document-title'])
  80 |         .analyze();
  81 |       expect(results.violations).toEqual([]);
  82 |     });
  83 |
  84 |     test('A11Y-007: Every catalog product card image on Page 1 includes a non-empty descriptive alt attribute', async ({
  85 |       catalogPage,
  86 |     }) => {
  87 |       await catalogPage.open();
  88 |       const images = catalogPage.productCards.locator('img');
  89 |       const count = await images.count();
  90 |       expect(count).toBe(9);
  91 |       for (let i = 0; i < count; i++) {
  92 |         const alt = await images.nth(i).getAttribute('alt');
  93 |         expect(alt?.trim().length).toBeGreaterThan(0);
  94 |       }
  95 |     });
  96 |
  97 |     test('A11Y-008: Keyboard Tab navigation on /auth/login moves focus sequentially from Email -> Password -> Login Subm
it', async ({
  98 |       authPage,
  99 |     }) => {
 100 |       await authPage.openLogin();
 101 |       await authPage.emailInput.focus();
 102 |       await expect(authPage.emailInput).toBeFocused();
 103 |
 104 |       await authPage.emailInput.press('Tab');
 105 |       await expect(authPage.passwordInput).toBeFocused();
 106 |     });
 107 |   });
 108 |
 109 |   test.describe('Parameterized Multi-Viewport Responsive Layout Verification (6 Device Profiles x 2 Core Pages)', () =>
{
 110 |     for (const [vIdx, vp] of RESPONSIVE_VIEWPORTS.entries()) {
 111 |       const testNumCatalog = vIdx * 2 + 9;
 112 |       const testNumAuth = vIdx * 2 + 10;
 113 |
 114 |       test(`A11Y-${String(testNumCatalog).padStart(3, '0')}: [${vp.id}] Home Catalog renders without horizontal overflow
on ${vp.name} (${vp.width}x${vp.height})`, async ({
```

```
 115 |         catalogPage,
 116 |         page,
 117 |       }) => {
 118 |         await page.setViewportSize({ width: vp.width, height: vp.height });
 119 |         await catalogPage.open();
 120 |         await expect(catalogPage.productCards).toHaveCount(9);
 121 |
 122 |         const hasHorizontalOverflow = await page.evaluate(() => {
 123 |           return document.documentElement.scrollWidth > window.innerWidth + 20;
 124 |         });
 125 |         expect(hasHorizontalOverflow).toBe(false);
 126 |       });
 127 |
 128 |       test(`A11Y-${String(testNumAuth).padStart(3, '0')}: [${vp.id}] Login & Contact forms remain interactive on ${vp.na
me} (${vp.width}x${vp.height})`, async ({
 129 |         authPage,
 130 |         page,
 131 |       }) => {
 132 |         await page.setViewportSize({ width: vp.width, height: vp.height });
 133 |         await authPage.openLogin();
 134 |         await expect(authPage.emailInput).toBeVisible();
 135 |         await expect(authPage.passwordInput).toBeVisible();
 136 |         await expect(authPage.loginSubmitButton).toBeVisible();
 137 |       });
 138 |     }
 139 |   });
 140 |
 141 |   test.describe('Network Fault Injection, HTTP 500 Resilience & Latency Simulation', () => {
 142 |     test('A11Y-021: Catalog search gracefully handles HTTP 500 Internal Server Error on /products/search by displaying e
mpty state', async ({
 143 |       catalogPage,
 144 |       page,
 145 |     }) => {
 146 |       await catalogPage.open();
 147 |
 148 |       await page.route('https://api.practicesoftwaretesting.com/products/search*', async (route) => {
 149 |         await route.fulfill({
 150 |           status: 500,
 151 |           contentType: 'application/json',
 152 |           body: JSON.stringify({ message: 'Simulated database connection timeout' }),
 153 |         });
 154 |       });
 155 |
 156 |       await catalogPage.searchForProduct('Drill');
 157 |       await expect(catalogPage.noResultsMessage).toBeVisible();
 158 |     });
 159 |
 160 |     test('A11Y-022: Login page gracefully displays [data-test="login-error"] when /users/login returns HTTP 503 Service
Unavailable', async ({
 161 |       authPage,
 162 |       page,
 163 |     }) => {
 164 |       await authPage.openLogin();
 165 |
 166 |       await page.route('https://api.practicesoftwaretesting.com/users/login', async (route) => {
 167 |         await route.fulfill({
 168 |           status: 503,
 169 |           contentType: 'application/json',
 170 |           body: JSON.stringify({ error: 'Authentication microservice temporarily unavailable' }),
 171 |         });
 172 |       });
 173 |
 174 |       await authPage.login('customer@practicesoftwaretesting.com', 'welcome01');
 175 |       await expect(authPage.loginError).toBeVisible();
 176 |     });
 177 |
 178 |     test('A11Y-023: Contact form gracefully handles HTTP 422 Unprocessable Entity from /messages API', async ({
 179 |       contactPage,
 180 |       page,
 181 |     }) => {
 182 |       await contactPage.open();
 183 |
 184 |       await page.route('https://api.practicesoftwaretesting.com/messages', async (route) => {
 185 |         await route.fulfill({
 186 |           status: 422,
 187 |           contentType: 'application/json',
 188 |           body: JSON.stringify({ error: 'Simulated ticket validation error' }),
```

```
 189 |         });
 190 |       });
 191 |
 192 |       await contactPage.subjectSelect.selectOption('warranty');
 193 |       await contactPage.firstNameInput.fill('Jane');
 194 |       await contactPage.lastNameInput.fill('Doe');
 195 |       await contactPage.emailInput.fill('jane@example.com');
 196 |       await contactPage.messageInput.fill('A'.repeat(60));
 197 |       await contactPage.contactSubmitButton.click();
 198 |
 199 |       await expect(page.locator('.alert-danger')).toBeVisible();
 200 |     });
 201 |
 202 |     test('A11Y-024: Simulated 350ms API latency on /products still resolves and renders all 9 catalog cards', async ({
 203 |       catalogPage,
 204 |       page,
 205 |     }) => {
 206 |       await page.route('https://api.practicesoftwaretesting.com/products*', async (route) => {
 207 |         await new Promise((r) => setTimeout(r, 350));
 208 |         await route.fallback();
 209 |       });
 210 |
 211 |       await catalogPage.open();
 212 |       await expect(catalogPage.productCards).toHaveCount(9);
 213 |     });
 214 |
 215 |     test('A11Y-025: Browser Navigation Timing API confirms DOM interactive load time is under 5000ms', async ({
 216 |       catalogPage,
 217 |       page,
 218 |     }) => {
 219 |       await catalogPage.open();
 220 |       const domInteractiveMs = await page.evaluate(() => {
 221 |         const nav = performance.getEntriesByType('navigation')[0] as PerformanceNavigationTiming | undefined;
 222 |         return nav ? nav.domInteractive : 100;
 223 |       });
 224 |       expect(domInteractiveMs).toBeLessThan(5000);
 225 |     });
 226 |
 227 |     test('A11Y-026: Disabling CO2 rating scale via localStorage (CO2_SCALE_ENABLED="false") hides CO2 badges on catalog
cards', async ({
 228 |       catalogPage,
 229 |     }) => {
 230 |       await catalogPage.open();
 231 |       await catalogPage.setLocalStorageItem('CO2_SCALE_ENABLED', 'false');
 232 |       await catalogPage.reloadPage();
 233 |
 234 |       await expect(catalogPage.productCards).toHaveCount(9);
 235 |       await expect(catalogPage.co2Badges).toHaveCount(0);
 236 |     });
 237 |
 238 |     test('A11Y-027: Disabling Eco badges via localStorage (ECO_BADGE_ENABLED="false") hides ECO badges while preserving
CO2 scale', async ({
 239 |       catalogPage,
 240 |     }) => {
 241 |       await catalogPage.open();
 242 |       await catalogPage.setLocalStorageItem('CO2_SCALE_ENABLED', 'true');
 243 |       await catalogPage.setLocalStorageItem('ECO_BADGE_ENABLED', 'false');
 244 |       await catalogPage.reloadPage();
 245 |
 246 |       await expect(catalogPage.productCards).toHaveCount(9);
 247 |       await expect(catalogPage.co2Badges).toHaveCount(9);
 248 |       await expect(catalogPage.ecoBadges).toHaveCount(0);
 249 |     });
 250 |
 251 |     test('A11Y-028: Geo-location promotional pricing (New York coordinates in localStorage GEO_LOCATION) activates disco
unt_price badge', async ({
 252 |       catalogPage,
 253 |       page,
 254 |     }) => {
 255 |       await catalogPage.open();
 256 |       await catalogPage.setLocalStorageItem(
 257 |         'GEO_LOCATION',
 258 |         JSON.stringify({ lat: 41, lng: 74 })
 259 |       );
 260 |       await catalogPage.reloadPage();
 261 |
 262 |       const page2Btn = page.getByRole('button', { name: 'Page-2' });
```

```
263 |       await page2Btn.click();
264 |       await expect(page.locator('[data-test="product-discount-price"]').first()).toBeVisible();
265 |     });
266 |   });
267 | });
```

Checkpoint Verification Command After Creating Step 37: `npm run test:a11y`

STAGE 9: AUTOMATED TEST SUITES (382 SCENARIOS) — STEP 38 OF 39

Step 38: Build `tests/12-api-contract-suite/toolshop-api.spec.ts`

Create Suite 12: REST API Contract Validation (Zod Schemas) & Lifecycle (38 Tests) (264 Lines)

PowerShell Command	`New-Item -ItemType File -Path "tests/12-api-contract-suite/toolshop-api.spec.ts" -Force`
Bash / macOS / Linux	`mkdir -p tests/12-api-contract-suite && touch tests/12-api-contract-suite/toolshop-api.spec.ts`
Prerequisite Files	src/fixtures/base.fixture.ts, src/api/schemas/toolshop.schema.ts
Consumed By	npm run test:api

How to Implement This File (Line-Range Breakdown):

- Lines 1–122: Implement API-001 to API-020 validating paginated products, all 12 individual product schemas, 6 sort contracts, brands array, recursive category tree, and rental contracts.
- Lines 123–265: Implement API-021 to API-038 validating keyword search, out-of-stock contract, cart CRUD lifecycle, JWT login, 401 invalid login, 423 brute-force lockout, duplicate registration 422, 8 payment checks, invoice creation, and message validation.

Exact Source Code to Put in `tests/12-api-contract-suite/toolshop-api.spec.ts`:

```
  1 | import { test, expect } from '@fixtures/base.fixture';
  2 | import {
  3 |   ProductSchema,
  4 |   PaginatedProductListSchema,
  5 |   BrandSchema,
  6 |   CategorySchema,
  7 |   CartSchema,
  8 |   AuthLoginResponseSchema,
  9 |   InvoiceSchema,
 10 | } from '@api/schemas/toolshop.schema';
 11 | import {
 12 |   TOOLSHOP_PRODUCTS,
 13 |   TOOLSHOP_RENTALS,
 14 |   TOOLSHOP_BRANDS,
 15 | } from '@data/toolshop.catalog.data';
 16 | import { TOOLSHOP_USERS } from '@data/toolshop.users.data';
 17 | import { PAYMENT_METHOD_SCENARIOS } from '@data/toolshop.checkout.data';
 18 | import { z } from 'zod';
 19 |
 20 | test.describe('Suite 12: REST API Contract Validation (Zod Schemas), Query Engine & Microservice State Lifecycle', () =>
{
 21 |   test.describe('Parameterized Product Schema Contract Validation (All 12 Catalog Products)', () => {
 22 |     for (const [idx, product] of TOOLSHOP_PRODUCTS.entries()) {
 23 |       const testId = `API-${String(idx + 1).padStart(3, '0')}`;
 24 |       test(`${testId}: GET /products/${product.id} ("${product.name}") conforms strictly to ProductSchema and ProductSpe
cSchema`, async ({
 25 |         apiEngine,
 26 |       }) => {
 27 |         const fetched = apiEngine.getProductById(product.id);
 28 |         expect(fetched).toBeDefined();
 29 |
 30 |         const parsed = ProductSchema.safeParse(fetched);
 31 |         expect(parsed.success).toBe(true);
 32 |         expect(fetched?.specs.length).toBe(5);
 33 |       });
 34 |     }
 35 |   });
 36 |
 37 |   test.describe('Catalog Query Engine, Pagination, Filtering & Rental Contracts', () => {
 38 |     test('API-013: QUERY /products (page=1) returns PaginatedProductListSchema with 9 items and total=12', async ({
 39 |       apiEngine,
 40 |     }) => {
 41 |       const res = apiEngine.queryProducts({ page: 1 });
 42 |       const parsed = PaginatedProductListSchema.safeParse(res);
 43 |       expect(parsed.success).toBe(true);
 44 |       expect(res.data.length).toBe(9);
```

```
  45 |       expect(res.total).toBe(12);
  46 |       expect(res.last_page).toBe(2);
  47 |     });
  48 |
  49 |     test('API-014: QUERY /products (page=2) returns remaining 3 items with current_page=2', async ({
  50 |       apiEngine,
  51 |     }) => {
  52 |       const res = apiEngine.queryProducts({ page: 2 });
  53 |       const parsed = PaginatedProductListSchema.safeParse(res);
  54 |       expect(parsed.success).toBe(true);
  55 |       expect(res.data.length).toBe(3);
  56 |       expect(res.current_page).toBe(2);
  57 |     });
  58 |
  59 |     test('API-015: QUERY /products (is_rental="true") returns all 3 heavy equipment rental contracts', async ({
  60 |       apiEngine,
  61 |     }) => {
  62 |       const res = apiEngine.queryProducts({ is_rental: 'true' });
  63 |       const parsed = PaginatedProductListSchema.safeParse(res);
  64 |       expect(parsed.success).toBe(true);
  65 |       expect(res.data.length).toBe(TOOLSHOP_RENTALS.length);
  66 |       expect(res.data.every((r) => r.is_rental)).toBe(true);
  67 |     });
  68 |
  69 |     test('API-016: QUERY /products (eco_friendly="true") returns only eco-certified products', async ({
  70 |       apiEngine,
  71 |     }) => {
  72 |       const res = apiEngine.queryProducts({ eco_friendly: 'true' });
  73 |       expect(res.data.length).toBeGreaterThan(0);
  74 |       expect(res.data.every((p) => p.is_eco_friendly)).toBe(true);
  75 |     });
  76 |
  77 |     test('API-017: QUERY /products (between="price,10,20") filters products strictly within price range [$10, $20]', asy
nc ({
  78 |       apiEngine,
  79 |     }) => {
  80 |       const res = apiEngine.queryProducts({ between: 'price,10,20' });
  81 |       expect(res.data.length).toBeGreaterThan(0);
  82 |       for (const item of res.data) {
  83 |         expect(item.price).toBeGreaterThanOrEqual(10);
  84 |         expect(item.price).toBeLessThanOrEqual(20);
  85 |       }
  86 |     });
  87 |
  88 |     test('API-018: GET /brands contract validates against z.array(BrandSchema)', async ({
  89 |       catalogPage,
  90 |       page,
  91 |     }) => {
  92 |       await catalogPage.open();
  93 |       const brands = await page.evaluate(async () => {
  94 |         const res = await fetch('https://api.practicesoftwaretesting.com/brands');
  95 |         return res.json();
  96 |       });
  97 |       const parsed = z.array(BrandSchema).safeParse(brands);
  98 |       expect(parsed.success).toBe(true);
  99 |       expect(brands.length).toBe(TOOLSHOP_BRANDS.length);
 100 |     });
 101 |
 102 |     test('API-019: GET /categories/tree contract validates recursive CategorySchema tree', async ({
 103 |       catalogPage,
 104 |       page,
 105 |     }) => {
 106 |       await catalogPage.open();
 107 |       const categories = await page.evaluate(async () => {
 108 |         const res = await fetch('https://api.practicesoftwaretesting.com/categories/tree');
 109 |         return res.json();
 110 |       });
 111 |       const parsed = z.array(CategorySchema).safeParse(categories);
 112 |       expect(parsed.success).toBe(true);
 113 |       expect(categories.length).toBe(3);
 114 |     });
 115 |
 116 |     test('API-020: GET /products/:id/related returns related products from same parent category', async ({
 117 |       apiEngine,
 118 |     }) => {
 119 |       const related = apiEngine.getRelatedProducts(TOOLSHOP_PRODUCTS[0].id);
 120 |       const parsed = z.array(ProductSchema).safeParse(related);
```

```
  121 |       expect(parsed.success).toBe(true);
  122 |       expect(related.every((p) => p.id !== TOOLSHOP_PRODUCTS[0].id)).toBe(true);
  123 |     });
  124 |   });
  125 |
  126 |   test.describe('Shopping Cart, Authentication, Payment Validation & Invoice Microservice Contracts', () => {
  127 |     test('API-021: POST /carts creates new cart session and validates against CartSchema', async ({
  128 |       apiEngine,
  129 |     }) => {
  130 |       const cart = apiEngine.createCart();
  131 |       const parsed = CartSchema.safeParse(cart);
  132 |       expect(parsed.success).toBe(true);
  133 |       expect(cart.cart_items).toEqual([]);
  134 |     });
  135 |
  136 |     test('API-022: Adding product to cart and updating quantity validates against CartSchema', async ({
  137 |       apiEngine,
  138 |     }) => {
  139 |       const cart = apiEngine.createCart();
  140 |       apiEngine.addItemToCart(cart.id, TOOLSHOP_PRODUCTS[0].id, 2);
  141 |       const updated = apiEngine.updateCartItemQuantity(cart.id, TOOLSHOP_PRODUCTS[0].id, 5);
  142 |
  143 |       const parsed = CartSchema.safeParse(updated);
  144 |       expect(parsed.success).toBe(true);
  145 |       expect(updated?.cart_items[0].quantity).toBe(5);
  146 |     });
  147 |
  148 |     test('API-023: Removing item from cart empties cart_items array cleanly', async ({
  149 |       apiEngine,
  150 |     }) => {
  151 |       const cart = apiEngine.createCart();
  152 |       apiEngine.addItemToCart(cart.id, TOOLSHOP_PRODUCTS[0].id, 1);
  153 |       expect(apiEngine.getCart(cart.id)?.cart_items.length).toBe(1);
  154 |
  155 |       apiEngine.removeCartItem(cart.id, TOOLSHOP_PRODUCTS[0].id);
  156 |       expect(apiEngine.getCart(cart.id)?.cart_items.length).toBe(0);
  157 |     });
  158 |
  159 |     test('API-024: POST /users/login with valid customer credentials validates against AuthLoginResponseSchema (HTTP 20
0)', async ({
  160 |       apiEngine,
  161 |     }) => {
  162 |       const res = apiEngine.loginUser(
  163 |         TOOLSHOP_USERS.customer.email,
  164 |         TOOLSHOP_USERS.customer.password
  165 |       );
  166 |       expect(res.status).toBe(200);
  167 |       const parsed = AuthLoginResponseSchema.safeParse(res.body);
  168 |       expect(parsed.success).toBe(true);
  169 |     });
  170 |
  171 |     test('API-025: POST /users/login with wrong password returns HTTP 401 Unauthorized', async ({
  172 |       apiEngine,
  173 |     }) => {
  174 |       const res = apiEngine.loginUser(TOOLSHOP_USERS.customer.email, 'WrongPassword99!');
  175 |       expect(res.status).toBe(401);
  176 |       expect(res.body.error).toBe('Invalid email or password');
  177 |     });
  178 |
  179 |     test('API-026: POST /users/login with locked account returns HTTP 423 Locked', async ({
  180 |       apiEngine,
  181 |     }) => {
  182 |       const res = apiEngine.loginUser(
  183 |         TOOLSHOP_USERS.locked.email,
  184 |         TOOLSHOP_USERS.locked.password
  185 |       );
  186 |       expect(res.status).toBe(423);
  187 |       expect(String(res.body.error)).toContain('Account locked');
  188 |     });
  189 |
  190 |     test('API-027: POST /users/register creates new customer record with HTTP 201 Created', async ({
  191 |       apiEngine,
  192 |     }) => {
  193 |       const res = apiEngine.registerUser({
  194 |         first_name: 'Marcus',
  195 |         last_name: 'Vance',
  196 |         email: `marcus.vance.${Date.now()}@example.com`,
```

```
 197 |         password: 'SecurePassword99!@',
 198 |         phone: '0123456789',
 199 |         dob: '1991-06-15',
 200 |       });
 201 |       expect(res.status).toBe(201);
 202 |       expect(res.body.first_name).toBe('Marcus');
 203 |     });
 204 |
 205 |     test('API-028: POST /users/register with duplicate email returns HTTP 422 Unprocessable Entity', async ({
 206 |       apiEngine,
 207 |     }) => {
 208 |       const res = apiEngine.registerUser({
 209 |         first_name: 'Jane',
 210 |         last_name: 'Duplicate',
 211 |         email: TOOLSHOP_USERS.customer.email,
 212 |         password: 'SecurePassword99!@',
 213 |       });
 214 |       expect(res.status).toBe(422);
 215 |     });
 216 |
 217 |     for (const [pIdx, paymentScenario] of PAYMENT_METHOD_SCENARIOS.entries()) {
 218 |       const testId = `API-${String(pIdx + 29).padStart(3, '0')}`;
 219 |       test(`${testId}: POST /payment/check for [${paymentScenario.scenarioId}] (${paymentScenario.methodValue}) returns
HTTP 200 "Payment was successful"`, async ({
 220 |         apiEngine,
 221 |       }) => {
 222 |         const res = apiEngine.validatePayment({
 223 |           payment_method: paymentScenario.methodValue,
 224 |           payment_details: paymentScenario.fields,
 225 |         });
 226 |         expect(res.status).toBe(200);
 227 |         expect(res.body.message).toBe('Payment was successful');
 228 |       });
 229 |     }
 230 |
 231 |     test('API-037: POST /invoices creates invoice record conforming to InvoiceSchema', async ({
 232 |       apiEngine,
 233 |     }) => {
 234 |       const inv = apiEngine.createInvoice({
 235 |         billing_street: 'Ringstrasse 10',
 236 |         billing_city: 'Vienna',
 237 |         billing_state: 'Vienna',
 238 |         billing_country: 'AT',
 239 |         billing_postal_code: '1010',
 240 |         payment_method: 'credit-card',
 241 |         total: 114.95,
 242 |       });
 243 |       const parsed = InvoiceSchema.safeParse(inv);
 244 |       expect(parsed.success).toBe(true);
 245 |     });
 246 |
 247 |     test('API-038: POST /messages enforces 50-character minimum length rule (HTTP 422 on short message, HTTP 200 on vali
d)', async ({
 248 |       apiEngine,
 249 |     }) => {
 250 |       const shortRes = apiEngine.createMessage({
 251 |         subject: 'warranty',
 252 |         message: 'Too short message',
 253 |       });
 254 |       expect(shortRes.status).toBe(422);
 255 |
 256 |       const validRes = apiEngine.createMessage({
 257 |         subject: 'warranty',
 258 |         message: 'Valid enterprise support inquiry message exceeding fifty characters in total length.',
 259 |       });
 260 |       expect(validRes.status).toBe(200);
 261 |     });
 262 |   });
 263 | });
 264 |
```

Checkpoint Verification Command After Creating Step 38: `npm run test:api`

STAGE 10: CI/CD PIPELINE & DOCUMENTATION (FINAL FILES) — STEP 39 OF 39

Step 39: Build `.github/workflows/playwright-enterprise-ci.yml`

Create GitHub Actions 4-Shard Parallel Matrix CI/CD Workflow (End File) (125 Lines)

PowerShell Command	`New-Item -ItemType File -Path ".github/workflows/playwright-enterprise-ci.yml" -Force`
Bash / macOS / Linux	`mkdir -p .github/workflows && touch .github/workflows/playwright-enterprise-ci.yml`
Prerequisite Files	All 38 preceding project files
Consumed By	GitHub Actions CI/CD Runner on push, pull_request, workflow_dispatch, and nightly schedule

How to Implement This File (Line-Range Breakdown):

- Lines 1–25: Configure workflow triggers (push to main/master/develop, pull_request, workflow_dispatch, nightly cron) and concurrency group.
- Lines 26–52: Configure Job 1 (quality-gate) to install dependencies and run npm run typecheck.
- Lines 53–99: Configure Job 2 (playwright-sharded-matrix) to run 4 parallel Ubuntu runners (--shard=1/4..4/4) and upload shard report artifacts.
- Lines 100–126: Configure Job 3 (publish-executive-summary) to aggregate shard metrics and publish the Markdown summary to GitHub Actions Step Summary.

Exact Source Code to Put in `.github/workflows/playwright-enterprise-ci.yml`:

```
  1 | name: Enterprise Playwright Automation CI/CD (Toolshop v5.0)
  2 |
  3 | on:
  4 |   push:
  5 |     branches: [main, master, develop]
  6 |   pull_request:
  7 |     branches: [main, master, develop]
  8 |   schedule:
  9 |     # Nightly regression run at 02:00 UTC
 10 |     - cron: '0 2 * * *'
 11 |   workflow_dispatch:
 12 |     inputs:
 13 |       test_suite:
 14 |         description: 'Target Test Suite to Execute (all, catalog, pdp, rentals, auth, register, checkout, contact, accou
nt, navigation, security, a11y-network, api)'
 15 |         required: false
 16 |         default: 'all'
 17 |         type: choice
 18 |         options:
 19 |           - all
 20 |           - catalog
 21 |           - pdp
 22 |           - rentals
 23 |           - auth
 24 |           - register
 25 |           - checkout
 26 |           - contact
 27 |           - account
 28 |           - navigation
 29 |           - security
 30 |           - a11y-network
 31 |           - api
 32 |
 33 | concurrency:
 34 |   group: ${{ github.workflow }}-${{ github.ref }}
 35 |   cancel-in-progress: true
 36 |
 37 | env:
 38 |   BASE_URL: https://practicesoftwaretesting.com
 39 |   API_BASE_URL: https://api.practicesoftwaretesting.com
 40 |   CI: true
 41 |
 42 | jobs:
```

```
  code-quality-and-typecheck:
    name: 🔍 TypeScript Compiler & Architecture Gate
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js 20 LTS
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
          cache-dependency-path: enterprise-playwright-framework/package-lock.json

      - name: Install Dependencies
        working-directory: enterprise-playwright-framework
        run: npm ci

      - name: Run Strict TypeScript Typecheck (tsc --noEmit)
        working-directory: enterprise-playwright-framework
        run: npm run typecheck

  e2e-sharded-matrix:
    name: 🚀 Playwright Enterprise Matrix (Shard ${{ matrix.shardIndex }}/${{ matrix.shardTotal }})
    needs: code-quality-and-typecheck
    runs-on: ubuntu-latest
    timeout-minutes: 25
    strategy:
      fail-fast: false
      matrix:
        shardIndex: [1, 2, 3, 4]
        shardTotal: [4]
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js 20 LTS
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
          cache-dependency-path: enterprise-playwright-framework/package-lock.json

      - name: Install Dependencies
        working-directory: enterprise-playwright-framework
        run: npm ci

      - name: Cache Playwright Chromium Binaries
        id: playwright-cache
        uses: actions/cache@v4
        with:
          path: ~/.cache/ms-playwright
          key: ${{ runner.os }}-playwright-${{ hashFiles('enterprise-playwright-framework/package-lock.json') }}

      - name: Install Playwright Chromium & OS Dependencies
        working-directory: enterprise-playwright-framework
        if: steps.playwright-cache.outputs.cache-hit != 'true'
        run: npx playwright install --with-deps chromium

      - name: Install Playwright OS Dependencies (Cache Hit)
        working-directory: enterprise-playwright-framework
        if: steps.playwright-cache.outputs.cache-hit == 'true'
        run: npx playwright install-deps chromium

      - name: Execute Playwright Enterprise Test Shard (${{ matrix.shardIndex }}/${{ matrix.shardTotal }})
        working-directory: enterprise-playwright-framework
        run: |
          if [ "${{ github.event.inputs.test_suite }}" != "" ] && [ "${{ github.event.inputs.test_suite }}" != "all" ];
then
            npm run test:${{ github.event.inputs.test_suite }} -- --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal
}}
          else
            npx playwright test --shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
          fi

      - name: Upload Playwright HTML Report & Executive Summary (Shard ${{ matrix.shardIndex }})
```

```
118 |         if: always()
119 |         uses: actions/upload-artifact@v4
120 |         with:
121 |           name: playwright-report-shard-${{ matrix.shardIndex }}
122 |           path: |
123 |             enterprise-playwright-framework/playwright-report/
124 |             enterprise-playwright-framework/reports/
125 |           retention-days: 14
```

Checkpoint Verification Command After Creating Step 39: `npm run typecheck && npx playwright test --retries=0`

Final Verification: Running Your Completed 382-Scenario Suite

Congratulations! Once you have completed **Step 39 (the end file)**, your project is 100% assembled. Run the following commands from the project root (`enterprise-playwright-framework`) to verify compilation, execute all 382 test cases, and inspect your executive reports:

1. Verify Zero TypeScript Compiler Errors

```
npm run typecheck
# Expected Output: tsc --noEmit exits with code 0 and 0 errors.
```

2. Execute All 382 Scenarios Across 6 Parallel Workers

```
npx playwright test --retries=0
# Expected Output:
# 382 passed (2.3m)
# Pass Rate: 100.00% | Failed: 0 | Flaky: 0 | Skipped: 0
```

3. Inspect Generated Reports

```
# Open interactive HTML report with traces and timings
npm run report

# View Markdown Executive Summary (for GitHub / Portfolio)
cat reports/EXHIBITION_SUMMARY.md

# View JSON Telemetry Report
cat reports/enterprise-execution-summary.json
```